

COMPUTER VISION PROJECT

Automated Image Segmentation and Object Measurement System

Thresholding, region growing and K means clustering

Segmentation of objects of interest from an image, followed by the computation of meaningful measurements about the detected objects, implemented in Python and OpenCV with classical computer vision only.

Author	Nii Yartey Gidiglo
Repository	github.com/Qwamzz/Computer-Vision-Projects
Application domain	Counting and measuring seeds, beans and coins
Implementation	Python 3 and OpenCV, no learned or pre-trained model
Dataset	8 real photographs and 6 controlled degradations of them

Contents

Section	Title	Page
1	Problem definition and objectives	3
2	Dataset	5
3	Methodology	7
4	Implementation	11
5	Experimental results	12
6	Critical analysis	24
7	Limitations and failure cases	27
8	Conclusion	29
9	References	29
A	Complete source code	31

1. Problem definition and objectives

1.1 The problem

Many practical inspection tasks reduce to the same question: how many objects are present in this picture, and how large is each one. A seed laboratory grading a sample of cowpea needs the length and the width of every seed. A quality inspector counting roasted coffee beans needs the count and the size distribution. A numismatic archive photographing a tray of coins needs one region per coin so that each coin can be catalogued. In all of these cases the answer is produced by two steps, a segmentation step that decides which pixels belong to an object, and a measurement step that turns each region into numbers.

This project builds that system with classical computer vision only. No learned detector and no pre-trained recognition model is used anywhere. Every decision is made by an algorithm whose behaviour can be written down in closed form, which is what makes the failure analysis in section 7 possible.

1.2 Objectives

- Assemble a dataset of real photographs of countable objects under varying imaging conditions, and produce manually verified ground truth masks for a representative subset of it.
- Implement threshold based segmentation, region growing and K means clustering from first principles rather than treating them as black boxes.
- Investigate the effect of the parameters that each method exposes: the threshold strategy, the seed point selection and the similarity criterion of the region growing, and the number of clusters and the feature space of the clustering.
- Identify object boundaries and extract each object as a separate region, including a step that separates objects that touch each other.
- Measure the count, area, perimeter, centroid, bounding box and size of every object, and convert those numbers into millimetres where a physical scale is available.
- Evaluate the three approaches quantitatively with Intersection over Union, precision, recall and F1 score against the ground truth, and measure the processing time, the sensitivity to noise and the sensitivity to illumination.
- Discuss where each method fails and why, and show which earlier techniques of the course repair those failures.

1.3 The pipeline that was built

The finished system implements the workflow required by the question:

Input image > image preparation > segmentation > boundary identification > object extraction > object measurement > performance evaluation

Complete pipeline on Sunflower seeds, scattered

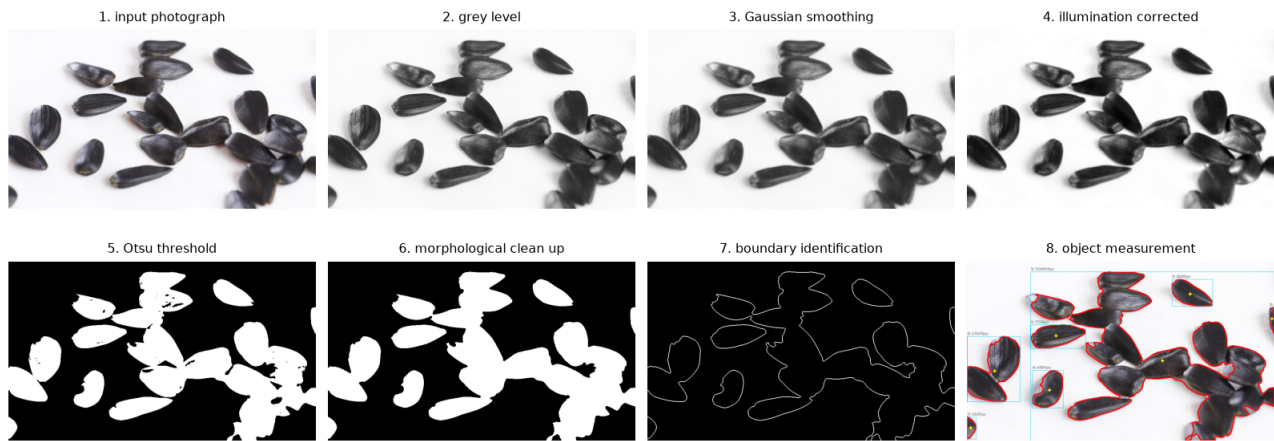


Figure 1. The eight stages of the pipeline on one photograph of the dataset. Stage 4 removes the lighting gradient, stage 5 applies the Otsu threshold, stage 6 cleans the mask morphologically, stage 7 extracts the boundaries and stage 8 reports the measurements.

2. Dataset

2.1 Source of the images

The dataset contains 8 base photographs. Every one of them is a real photograph taken by a person with a camera. None of them was produced by an image generation model. Seven of them come from Wikimedia Commons under a Creative Commons licence, and one is the coins photograph that is distributed as sample data with scikit-image and originates from a museum collection. The full provenance is recorded in `src/config.py` and repeated in Table 1, so that any reader can retrieve the exact files.

identifier	content	application domain	objects	licence
seeds_sunflower_01	Sunflower seeds, scattered	Counting and measuring seeds	not counted	CC BY-SA 4.0
seeds_sunflower_02	Sunflower seeds, second layout	Counting and measuring seeds	not counted	CC BY-SA 4.0
seeds_sunflower_03	Sunflower seeds, third layout	Counting and measuring seeds	not counted	CC BY-SA 4.0
seeds_pale_04	Pale sunflower seeds of three sizes	Counting and measuring seeds	3	CC BY-SA 4.0
beans_cowpea_scale_05	Cowpea seeds photographed with a centimetre scale	Analysing objects in agricultural images	8	CC BY-SA 3.0
beans_coffee_06	Roasted coffee beans with cast shadows	Separating foreground objects from a simple background	5	CC BY-SA 4.0
coins_greek_07	Twenty four Greek coins	Identifying coins or circular objects	24	Public domain
beans_pinto_dense_08	Densely piled pinto beans	Counting and measuring seeds, hard case	not counted	CC BY-SA 4.0

Table 1. The base photographs of the dataset. The object count is given for the images that carry an instance level ground truth.



Figure 2. The eight base photographs. They cover four of the application domains suggested by the question and they differ in contrast, in background, in object density and in the amount of shadow present.

2.2 Imaging conditions

The question asks for the system to be tested under illumination changes, background variation, noise, partially touching objects and objects of different sizes. Part of this variation is already present in the photographs themselves: the coins sit on a dark background with a strong brightness falloff, the coffee beans cast soft shadows and carry specular highlights, the pale seeds have almost no contrast against their background, the third sunflower photograph is partly out of focus, and the pinto bean photograph is a dense pile in which nearly every object touches a neighbour.

To obtain a controlled comparison rather than an anecdotal one, four further conditions are generated from two of the photographs by applying a known degradation to the real image: a diagonal lighting ramp that scales brightness from 0.30 to 1.20, an under exposure that multiplies the brightness by 0.45 and lifts the black point, additive zero mean Gaussian noise with a standard deviation of 18 grey levels, and a defocus blur modelled by a Gaussian point spread function with a fifteen pixel kernel. The geometry of the objects is untouched by all four operations, so the ground truth mask of the original photograph remains valid and the change in the score can be attributed to the condition alone.



Figure 3. The original photograph and the four controlled conditions derived from it.

2.3 Ground truth masks

Ground truth masks were produced for five of the photographs. The procedure follows the way modern annotation tools work rather than painting every pixel by hand. A per image draft was produced first, using information that the system under test is not allowed to use, for example a threshold offset hand picked for that one photograph or an edge based recipe chosen for the coins. Every draft was then inspected as an overlay on the photograph and corrected by hand: false regions were deleted, touching objects were separated by clicking one point inside each object and letting a marker driven watershed cut along the crevice, and clipped parts of an outline were painted back in. For the low contrast photograph the draft failed completely, so the three seeds were outlined by hand from a coordinate grid and the traced polygons were then tightened onto the object boundary by a colour model refinement. Every mask was inspected a second time before it was accepted. The complete edit list is in `src/ground_truth.py`, so the annotation is fully reproducible.

Four of the five masks carry one region per physical object, so object level counting can be scored against them. The mask of the first sunflower photograph is a foreground mask only, because several of its seeds touch along a long contact line where no crevice is visible, and an honest instance level annotation of that photograph is not possible. It is therefore used for pixel level scoring only. The dense pinto bean photograph carries no ground truth at all and is reported as a qualitative failure case.

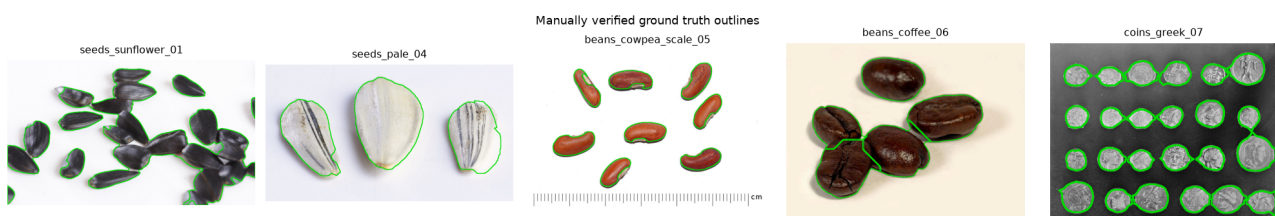


Figure 4. The manually verified ground truth outlines. Each closed green contour is one annotated region.

3. Methodology

3.1 Image preparation

Every image is first scaled so that its longest side is 900 pixels. Images are only ever shrunk, never enlarged, so no detail is invented, and the common working size makes the timing comparison fair and keeps the morphological kernel sizes meaningful across the dataset.

Colour representation

The candidate representations that were examined are the grey level image, the saturation and value channels of HSV and the lightness, a and b channels of CIE Lab. For seven of the eight photographs the objects differ from the background mainly in brightness, so the grey level image carries the separation and is used by the threshold and the region growing stages. The exception is the cowpea photograph, where the printed ruler is as dark as the seeds in the grey image but neutral in colour, while the seeds are strongly red. The a channel of CIE Lab separates the seeds from the ruler and is therefore the correct representation for that image. The clustering stage is able to use the full Lab colour vector, which is one of the reasons it outperforms the other two methods in section 5.

Noise suppression

Three filters are provided. The Gaussian filter is a linear low pass filter, it is the fastest and it blurs edges together with the noise. The median filter is a rank filter, it removes impulse noise while preserving step edges. The bilateral filter weights neighbours by both spatial and photometric distance, so it preserves edges best but costs the most time. Section 5.5 compares them on the noisy images.

Illumination correction

An uneven illumination field is the single most damaging condition for a global threshold, because it shifts the object and the background distributions by different amounts in different parts of the image. The system removes it with a classical flat field correction. A morphological closing with a structuring element far larger than any object removes the objects and leaves an estimate of the background field, which is then divided out and rescaled to the full eight bit range. When the objects are brighter than the background an opening is used instead. The polarity is decided automatically by comparing the mean of a border ring of the image with the mean of its interior.

3.2 Method 1, threshold based segmentation

Otsu's method is implemented from the definition rather than called as a library flag. For a candidate grey level t the pixels are split into a low class and a high class, and the between class variance is

$$\sigma_b^2(t) = w_0(t) \cdot w_1(t) \cdot (\mu_0(t) - \mu_1(t))^2$$

where w_0 and w_1 are the probabilities of the two classes and μ_0 and μ_1 their means. The implementation evaluates this expression for all 256 candidate thresholds using cumulative sums, so it runs in linear time in the number of grey levels, and it returns the whole criterion curve so that the choice can be plotted. Two further strategies are provided for comparison, a fixed threshold supplied by the user, and a locally adaptive Gaussian threshold which computes a separate threshold for every neighbourhood and therefore tolerates a lighting gradient without an explicit correction step.

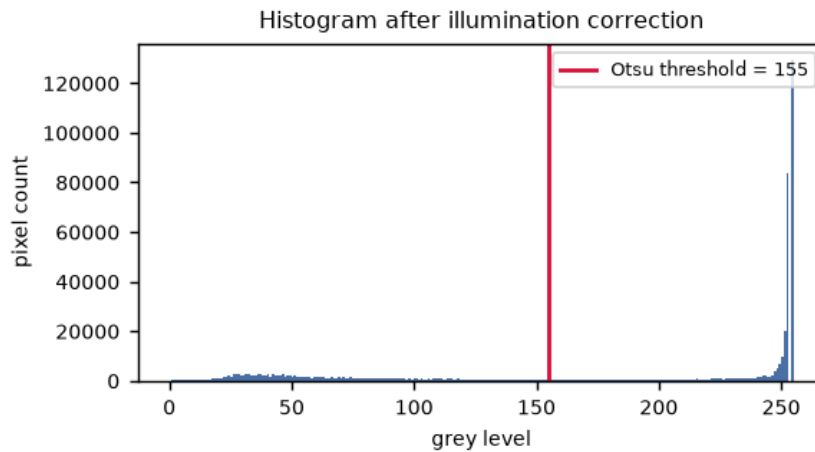


Figure 5. The grey level histogram after illumination correction, with the threshold chosen by the from scratch Otsu implementation. The histogram is clearly bimodal, which is exactly the situation in which Otsu is the right tool.

3.3 Method 2, region growing

Region growing starts from a set of seed pixels and repeatedly adds neighbouring pixels that satisfy a similarity predicate, so it is a region based method that uses connectivity explicitly, unlike thresholding which classifies every pixel independently. The implementation uses a first in first out queue, so the traversal is a breadth first flood of the similarity region, and every pixel is visited at most once, which makes the routine linear in the number of pixels.

Similarity predicate

A candidate pixel is accepted when the absolute difference between its intensity and a reference intensity of the region is at most the tolerance. Two references are implemented. The running mean reference uses the mean intensity of the pixels accepted so far, which allows a region to follow a slow intensity drift across a large object. The seed value reference keeps the intensity of the seed fixed, which is stricter and prevents a region from drifting through a shading gradient into the background.

Seed selection

Three strategies are implemented. The distance transform strategy thresholds the image coarsely, computes the distance transform of the resulting mask and places one seed at the deepest interior point of every connected component, so it is an informed strategy that places a seed near the centre of each object. The grid strategy places seeds on a regular lattice and keeps those that fall on the object side of the coarse threshold. The random strategy samples seed positions uniformly from the object side. Section 5.3 shows how much the result depends on this choice.

3.4 Method 3, K means clustering

The clustering is implemented as Lloyd's algorithm with a k means plus plus initialisation. The initialisation samples the first centre uniformly and then samples each further centre with a probability proportional to the squared distance to the nearest centre already chosen, which spreads the initial centres apart and avoids the degenerate solutions that a purely random initialisation produces. The main loop alternates an assignment step that attaches every pixel to its nearest centre with an update step that moves every centre to the mean of its members, and it stops when no centre moves further than a small tolerance.

Feature space

Three feature spaces are provided: the intensity of the prepared grey image, the three dimensional Lab colour vector, and a five dimensional vector that appends the normalised pixel coordinates to the colour vector so that spatially compact clusters are preferred.

Deciding which clusters are objects

Clustering alone does not say which cluster is the object. The system decides this from the border of the image: a cluster that occupies more than a quarter of the border ring is declared background, and everything else is foreground. If that rule leaves no cluster or all clusters as background, the system falls back to the ordering of the cluster means and picks the darkest or the brightest cluster according to the polarity of the scene. This rule is what allows K means with more than two clusters to be used as a segmentation rather than only as a quantisation.

3.5 Post processing, boundaries and object extraction

Every raw mask passes through the same clean up: a morphological opening removes isolated noise pixels, a closing repairs small gaps, a hole filling step closes the specular highlights that appear inside glossy objects, and an area filter removes fragments that are far too small to be an object. The hole filling is implemented as a flood fill from a padded background border, so that whatever background is connected to the border stays background and every enclosed pocket becomes object.

Boundaries are obtained with the morphological gradient of the mask, that is the dilation minus the erosion, which gives a one pixel outline of every region. Individual objects are extracted with eight connected component labelling.

Separating objects that touch

Connected component labelling reports one region for a group of touching objects. The system therefore offers a separation step built on the distance transform and the watershed transform. The distance transform peaks at the centre of every object, thresholding it at a fraction of the local maximum produces one marker per object even when two objects share a boundary, and the watershed then grows those markers back to the object boundaries. The threshold is applied inside each connected component separately, because a single global threshold silently deletes every object that is much smaller than the largest object in the image.

3.6 Object measurement

For every extracted region the system reports the measurements listed in the question and several standard shape descriptors:

- **Object count**, the number of accepted connected components.
- **Area**, the number of pixels in the region.
- **Perimeter**, the arc length of the outer contour.
- **Centroid**, the first order moment of the region.
- **Bounding box**, the axis aligned rectangle around the region.
- **Object size**, reported as the equivalent circular diameter and as the major and minor axes of the ellipse fitted to the contour, which is the length and the width of an elongated object such as a seed.
- **Shape descriptors**: circularity $4 \pi A / P^2$, which is one for a perfect disc, the aspect ratio, the extent and the solidity.

The pixel measurements are converted into millimetres for the cowpea photograph, which was taken next to a printed ruler. The calibration routine projects the ruler band onto the horizontal axis, detects the tick marks and takes the median distance between neighbouring ticks, which gives 13.0 pixels per millimetre at the working resolution and a field of view of 69.2 millimetres. The median is used so that the wider marks at the five millimetre positions cannot bias the estimate.

3.7 Evaluation protocol

Predicted masks are compared with the ground truth masks at two levels. At the pixel level the system reports Intersection over Union, precision, recall, F1 score, the Dice coefficient and pixel accuracy. At the object level, predicted regions are matched greedily to ground truth regions in decreasing order of overlap, a match is accepted at an IoU of 0.5 or more, and object precision, object recall, object F1 and the count error follow from

that matching. A third measure evaluates the measurements themselves: for every matched pair the relative error of the area and of the major axis length is computed, which answers the question that matters for a measurement system, namely how accurate the reported number is for an object that was found.

All three methods are run with one common parameter set across the whole dataset, so that no method is given a per image advantage. The tolerance of the region growing and the configuration of the clustering are the values that performed best on average in the parameter studies of sections 5.3 and 5.4, so each method is compared at its own best common setting.

4. Implementation

The system is written in Python 3 and uses OpenCV, NumPy and Matplotlib. Otsu's threshold, the region growing and the K means clustering are implemented in this project rather than called as library primitives, so that the criterion, the queue and the iteration are all visible in the source. OpenCV is used for the standard supporting operations: colour conversion, filtering, morphology, connected component labelling, contour extraction, the distance transform and the watershed.

module	responsibility
config.py	Paths, working resolution and the dataset manifest with the provenance of every photograph.
dataset.py	Retrieval of the base photographs and generation of the four controlled degradations.
preprocessing.py	Colour representation, smoothing filters, contrast handling, flat field illumination correction and polarity detection.
segmentation.py	Otsu from first principles, region growing with an explicit queue, K means with k means plus plus, morphological clean up and the watershed separation of touching objects.
measurement.py	Boundary identification, connected component extraction and all object measurements.
evaluation.py	Pixel level scores, object level matching and the measurement error analysis.
calibration.py	Pixels per millimetre from the printed ruler.
ground_truth.py	Assisted annotation, the manual edit list and the ground truth masks.
visualisation.py	All figures written to results/figures.
run_experiments.py	The experiment driver that reproduces every number and figure in this report.
report_content.py	The text, tables and figure list of this report, written once as neutral blocks.
make_report.py	Renders those blocks into the PDF version of the report.
make_report_docx.py	Renders the same blocks into the Word version, so the two files can never drift apart.

Table 2. Module map of the project.

How to reproduce

```
cd src
python dataset.py           # fetch photographs and build the variants
python ground_truth.py      # rebuild the annotation masks
python run_experiments.py   # all experiments, tables and figures
python make_report.py       # the PDF report
python make_report_docx.py  # the Word version of the report
```

The complete run of the experiment driver takes about 436.9 seconds on the machine used for this project.

5. Experimental results

5.1 Comparison of the three segmentation techniques

image	method	IoU	precision	recall	F1	time (s)	regions found	objects in truth
seeds sunflower 01	Thresholding (Otsu)	0.9888	0.9996	0.9892	0.9944	0.037	7	n/a
seeds sunflower 01	Region growing (tol 25)	0.3528	1.0	0.3528	0.5216	1.037	13	n/a
seeds sunflower 01	K means (K = 3, Lab)	0.9374	0.9376	0.9997	0.9677	4.991	7	n/a
seeds pale 04	Thresholding (Otsu)	0.1743	0.5756	0.2	0.2968	0.031	6	3
seeds pale 04	Region growing (tol 25)	0.311	0.311	1.0	0.4745	5.151	1	3
seeds pale 04	K means (K = 3, Lab)	0.6076	0.7434	0.7689	0.7559	2.205	4	3
beans cowpea scale 05	Thresholding (Otsu)	0.8285	0.8285	1.0	0.9062	0.042	9	8
beans cowpea scale 05	Region growing (tol 25)	0.1122	0.1122	1.0	0.2018	8.226	1	8
beans cowpea scale 05	K means (K = 3, Lab)	0.8279	0.8279	1.0	0.9059	15.35	9	8
beans coffee 06	Thresholding (Otsu)	0.9623	0.9718	0.99	0.9808	0.038	2	5
beans coffee 06	Region growing (tol 25)	0.4067	0.9965	0.4072	0.5782	0.934	2	5
beans coffee 06	K means (K = 3, Lab)	0.901	0.9017	0.999	0.9479	11.972	2	5
coins greek 07	Thresholding (Otsu)	0.5935	0.9219	0.625	0.7449	0.012	18	24
coins greek 07	Region growing (tol 25)	0.6434	0.9518	0.6651	0.783	0.362	18	24
coins greek 07	K means (K = 3, Lab)	0.8013	0.9393	0.845	0.8897	1.093	24	24

Table 3. The three segmentation techniques on every annotated image, using one common parameter set.

method	mean IoU	mean precision	mean recall	mean F1	mean time (s)
Thresholding (Otsu)	0.7095	0.8595	0.7608	0.7846	0.032
Region growing (tol 25)	0.3652	0.6743	0.685	0.5118	3.142
K means (K = 3, Lab)	0.815	0.87	0.9225	0.8934	7.122

Table 4. Mean scores over the five annotated photographs. The best method is highlighted.

The summary table requested by the question is therefore the following, where the sensitivity entries are taken from the condition study in section 5.6 and the noise study in section 5.5.

method	segmentation (mean IoU)	accuracy	processing time (s)	sensitivity to noise (change in IoU)	sensitivity to illumination (change in IoU)
Thresholding (Otsu)	0.710		0.032	-0.022	+0.004
Region growing (tol 25)	0.365		3.142	+0.043	+0.061
K means (K = 3, Lab)	0.815		7.122	-0.003	-0.026

Table 5. The comparison table required by the question. A negative change means the score fell when the condition was applied.

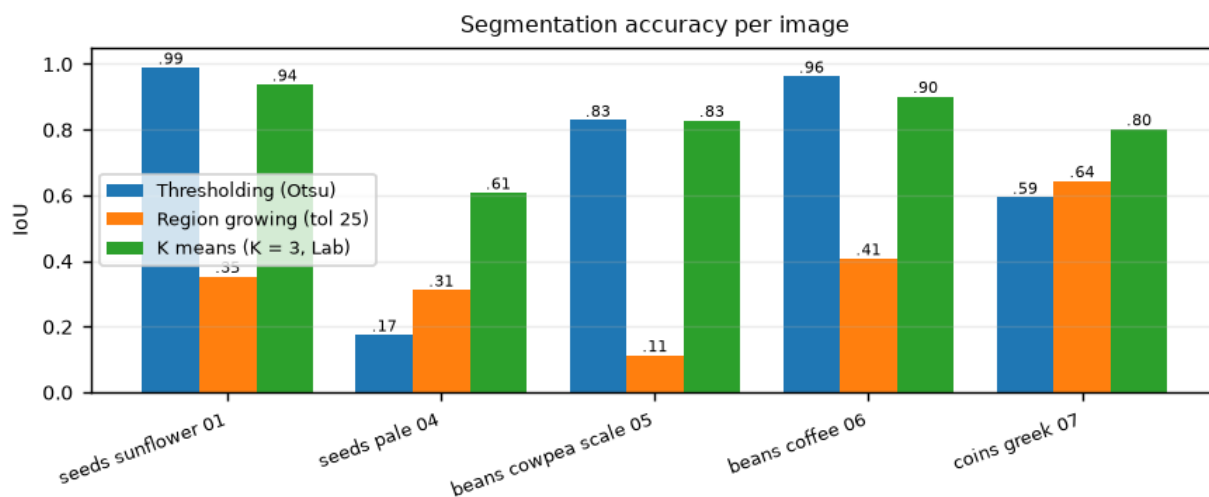


Figure 6. Segmentation accuracy of the three methods on each annotated photograph.

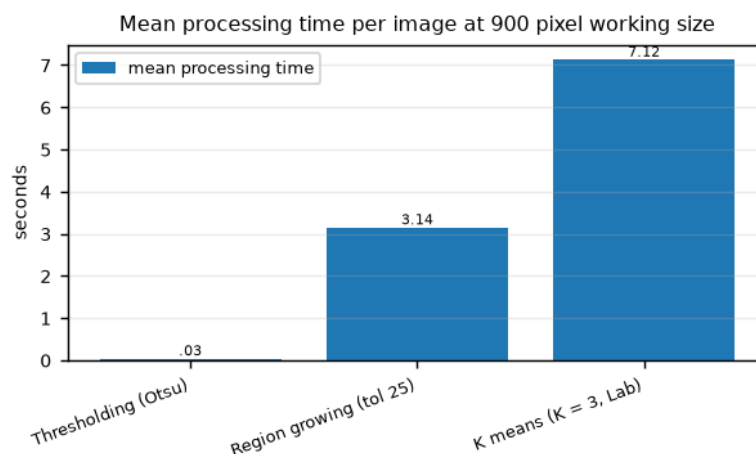


Figure 7. Mean processing time per image at the common working resolution.

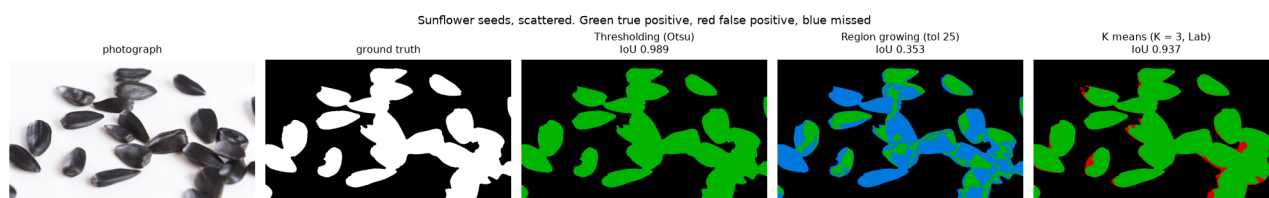


Figure. Sunflower seeds, scattered. Green marks pixels that were correctly assigned to an object, red marks background accepted as object and blue marks object pixels that were missed.

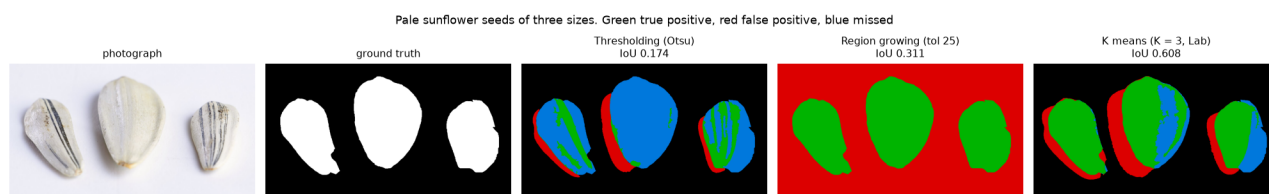


Figure. Pale sunflower seeds of three sizes. Green marks pixels that were correctly assigned to an object, red marks background accepted as object and blue marks object pixels that were missed.

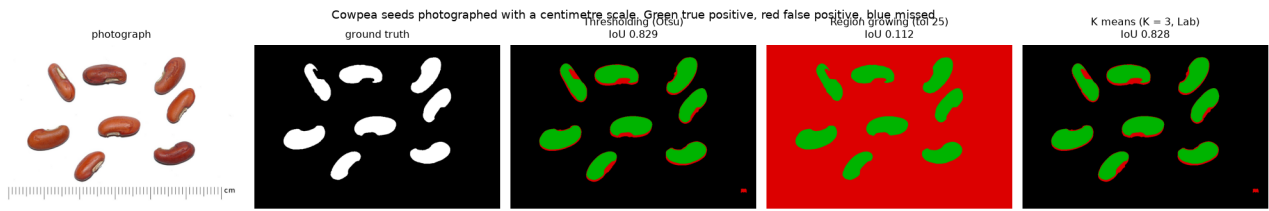


Figure. Cowpea seeds photographed with a centimetre scale. Green marks pixels that were correctly assigned to an object, red marks background accepted as object and blue marks object pixels that were missed.

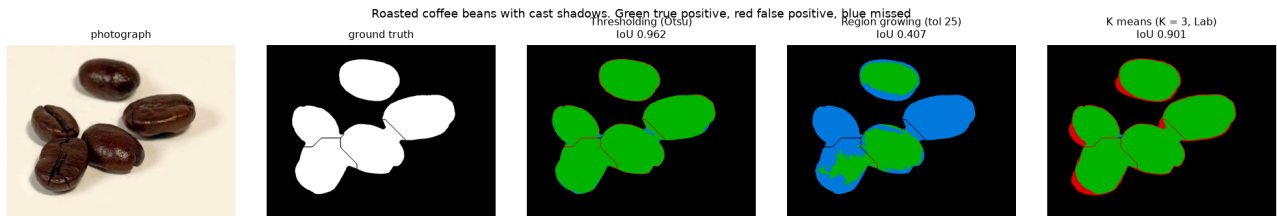


Figure. Roasted coffee beans with cast shadows. Green marks pixels that were correctly assigned to an object, red marks background accepted as object and blue marks object pixels that were missed.



Figure. Twenty four Greek coins. Green marks pixels that were correctly assigned to an object, red marks background accepted as object and blue marks object pixels that were missed.

5.2 Threshold strategy and illumination

image	threshold strategy	IoU	precision	recall	F1
seeds sunflower 01	Otsu, illumination corrected	0.9888	0.9996	0.9892	0.9944
seeds sunflower 01	Otsu, no illumination correction	0.9824	0.9902	0.9921	0.9911
seeds sunflower 01	Fixed threshold 100	0.7752	1.0	0.7752	0.8733
seeds sunflower 01	Fixed threshold 140	0.9494	0.9965	0.9526	0.974
seeds sunflower 01	Fixed threshold 180	0.9572	0.9573	0.9998	0.9781
seeds sunflower 01	Adaptive Gaussian, block 51	0.7316	0.9796	0.7429	0.845
seeds sunflower 01	Adaptive Gaussian, block 101	0.9	0.98	0.9168	0.9473
seeds sunflower 01 illum	Otsu, illumination corrected	0.9798	0.9991	0.9807	0.9898
seeds sunflower 01 illum	Otsu, no illumination correction	0.6508	0.657	0.9856	0.7885
seeds sunflower 01 illum	Fixed threshold 100	0.8491	0.9147	0.9221	0.9184
seeds sunflower 01 illum	Fixed threshold 140	0.641	0.6459	0.9882	0.7812
seeds sunflower 01 illum	Fixed threshold 180	0.5033	0.5033	0.9999	0.6696
seeds sunflower 01 illum	Adaptive Gaussian, block 51	0.6199	0.9857	0.6256	0.7654
seeds sunflower 01 illum	Adaptive Gaussian, block 101	0.8782	0.9864	0.889	0.9352
coins greek 07	Otsu, illumination corrected	0.5935	0.9219	0.625	0.7449
coins greek 07	Otsu, no illumination correction	0.7092	0.729	0.9631	0.8299
coins greek 07	Fixed threshold 100	0.6859	0.7025	0.9666	0.8137
coins greek 07	Fixed threshold 140	0.7872	0.9555	0.8171	0.8809
coins greek 07	Fixed threshold 180	0.144	0.9767	0.1445	0.2517
coins greek 07	Adaptive Gaussian, block 51	0.4589	0.4664	0.9662	0.6291
coins greek 07	Adaptive Gaussian, block 101	0.6548	0.6704	0.9657	0.7914
coins greek 07 illum	Otsu, illumination corrected	0.61	0.9235	0.6425	0.7578
coins greek 07 illum	Otsu, no illumination correction	0.8523	0.9187	0.9219	0.9203

image	threshold strategy	IoU	precision	recall	F1
coins greek 07 illum	Fixed threshold 100	0.7419	0.9457	0.7749	0.8518
coins greek 07 illum	Fixed threshold 140	0.2921	0.9841	0.2934	0.4521
coins greek 07 illum	Fixed threshold 180	0.018	1.0	0.018	0.0353
coins greek 07 illum	Adaptive Gaussian, block 51	0.4142	0.4182	0.9773	0.5857
coins greek 07 illum	Adaptive Gaussian, block 101	0.5885	0.6007	0.9669	0.741

Table 6. Threshold study on the original photographs and on the same photographs with a strong lighting gradient.

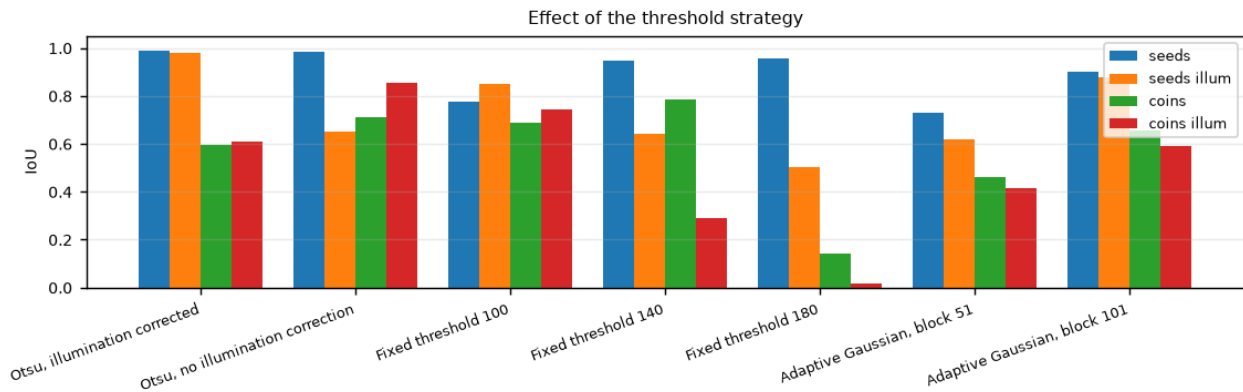


Figure 8. Effect of the threshold strategy. The fixed thresholds are only competitive when the value happens to suit the image, while Otsu combined with the flat field correction is stable across all four cases.



Figure 9. The lighting gradient case in detail. Without the flat field correction the single global threshold cannot satisfy both sides of the image at once.

5.3 Region growing, seeds and similarity

image	study	setting	seeds	IoU	recall	regions	time (s)
seeds sunflower 01	tolerance	6	7	0.0446	0.0446	5	0.115
seeds sunflower 01	tolerance	10	7	0.077	0.077	6	0.162
seeds sunflower 01	tolerance	14	7	0.101	0.101	7	0.197
seeds sunflower 01	tolerance	18	7	0.1284	0.1284	10	0.221
seeds sunflower 01	tolerance	25	7	0.3528	0.3528	13	0.786
seeds sunflower 01	tolerance	35	7	0.2367	0.2367	7	0.575
seeds sunflower 01	tolerance	50	7	0.7056	0.7056	7	1.96
seeds sunflower 01	seed_strategy	distance	7	0.3528	0.3528	13	0.843
seeds sunflower 01	seed_strategy	grid	14	0.569	0.569	4	1.41
seeds sunflower 01	seed_strategy	random	40	0.7544	0.7555	6	1.553
seeds sunflower 01	criterion	running_mean	7	0.3528	0.3528	13	0.459
seeds sunflower 01	criterion	seed_value	7	0.2204	0.2204	16	0.327
coins greek 07	tolerance	6	18	0.0976	0.0977	7	0.076
coins greek 07	tolerance	10	18	0.2989	0.3	15	0.11
coins greek 07	tolerance	14	18	0.4103	0.4141	17	0.14
coins greek 07	tolerance	18	18	0.496	0.5038	18	0.22
coins greek 07	tolerance	25	18	0.6434	0.6651	18	0.393
coins greek 07	tolerance	35	18	0.3197	1.0	1	1.741
coins greek 07	tolerance	50	18	0.3197	1.0	1	1.795

image	study	setting	seeds	IoU	recall	regions	time (s)
coins greek 07	seed_strategy	distance	18	0.6434	0.6651	18	0.21
coins greek 07	seed_strategy	grid	5	0.1404	0.141	5	0.05
coins greek 07	seed_strategy	random	40	0.5427	0.5581	15	0.173
coins greek 07	criterion	running_mean	18	0.6434	0.6651	18	0.241
coins greek 07	criterion	seed_value	18	0.5544	0.5684	18	0.201

Table 7. Region growing study.

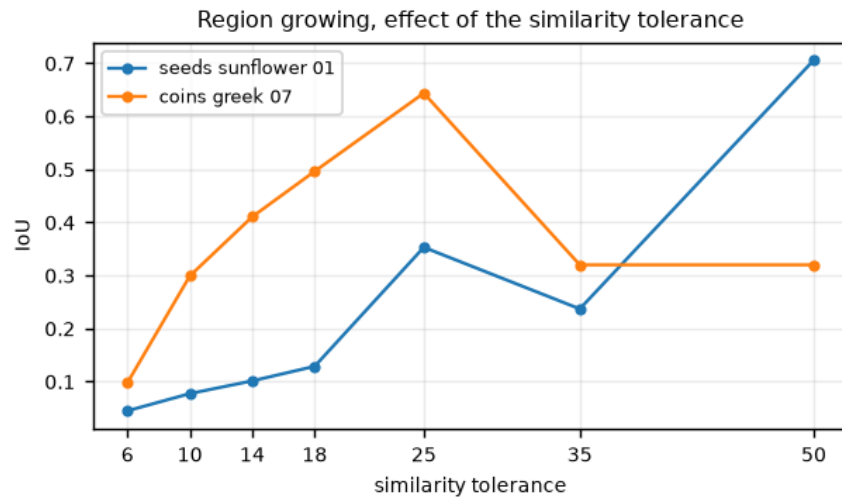


Figure 10. Effect of the similarity tolerance. Below the correct value the regions stop inside the objects, and above it the regions leak through the object boundary into the background.

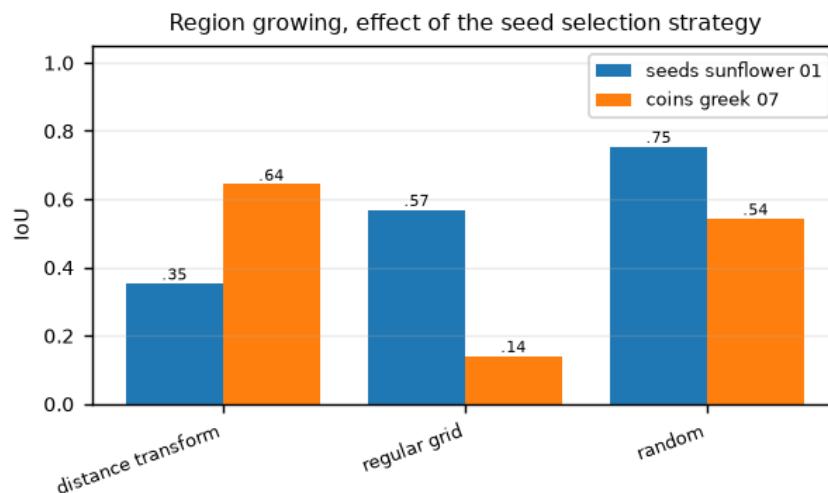


Figure 11. Effect of the seed selection strategy. The informed distance transform strategy is the best choice on the coins, while on the seeds the extra seeds of the random strategy compensate for the wide intensity range inside each object.



Figure 12. Region growing at three tolerances on the same photograph.

5.4 K means, number of clusters and features

image	study	setting	IoU	precision	recall	regions	time (s)
seeds sunflower 01	K	2	0.9888	0.9996	0.9892	7	0.324
seeds sunflower 01	K	3	0.9388	0.9389	0.9999	7	1.049
seeds sunflower 01	K	4	0.8797	0.8797	1.0	7	1.497
seeds sunflower 01	K	5	0.843	0.843	1.0	6	1.807
seeds sunflower 01	K	6	0.8245	0.8245	1.0	6	2.422
seeds sunflower 01	features	intensity	0.9388	0.9389	0.9999	7	1.223
seeds sunflower 01	features	lab	0.9374	0.9376	0.9997	7	5.747
seeds sunflower 01	features	lab_xy	0.9651	0.9947	0.9701	7	11.04
coins greek 07	K	2	0.5852	0.9222	0.6156	18	0.128
coins greek 07	K	3	0.6763	0.8974	0.7329	18	0.11
coins greek 07	K	4	0.7902	0.8539	0.9137	22	0.255
coins greek 07	K	5	0.7441	0.7598	0.973	17	0.271
coins greek 07	K	6	0.7902	0.8539	0.9137	22	0.605
coins greek 07	features	intensity	0.6763	0.8974	0.7329	18	0.148
coins greek 07	features	lab	0.8013	0.9393	0.845	24	0.637
coins greek 07	features	lab_xy	0.8034	0.9398	0.847	24	1.123
beans coffee 06	K	2	0.9623	0.9718	0.99	2	0.403
beans coffee 06	K	3	0.9452	0.9494	0.9953	3	1.617
beans coffee 06	K	4	0.9222	0.9233	0.9986	4	3.622
beans coffee 06	K	5	0.914	0.915	0.9988	3	1.471
beans coffee 06	K	6	0.9027	0.9035	0.999	2	1.508
beans coffee 06	features	intensity	0.9452	0.9494	0.9953	3	1.893
beans coffee 06	features	lab	0.901	0.9017	0.999	2	13.292
beans coffee 06	features	lab_xy	0.9586	0.9609	0.9976	2	7.304

Table 8. K means study.

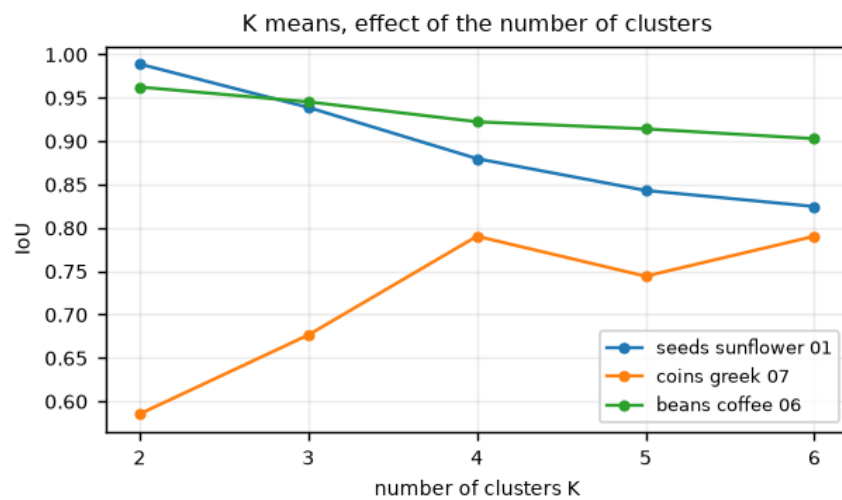


Figure 13. Effect of the number of clusters. On an image with two clearly separated populations, raising K splits the object itself and the score falls, while on the coins, where the background has its own internal structure, a larger K is what allows the dark corner of the background to be separated from the coins.

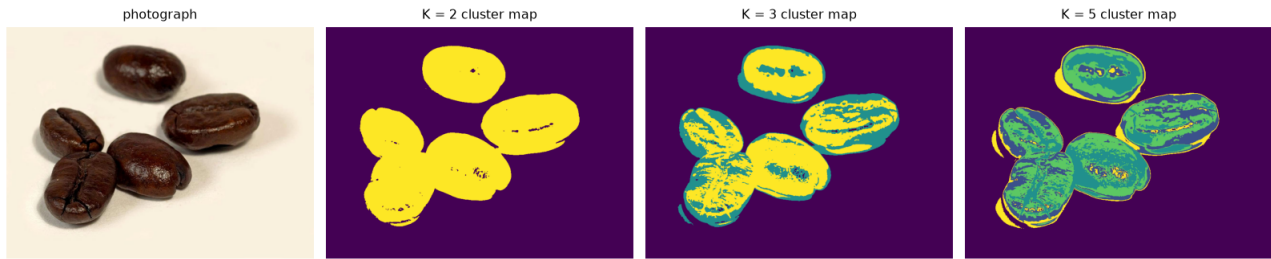


Figure 14. Cluster label maps for three values of K on the coffee bean photograph. At K equal to three the shadow becomes its own cluster.

5.5 Filtering study

image	filter	IoU	precision	recall	F1	regions	time (s)
seeds sunflower 01 noise	none	0.9791	0.9991	0.98	0.9895	7	0.017
seeds sunflower 01 noise	gaussian	0.9829	0.9996	0.9832	0.9914	7	0.017
seeds sunflower 01 noise	median	0.9795	0.9995	0.98	0.9897	7	0.016
seeds sunflower 01 noise	bilateral	0.9805	0.9993	0.9812	0.9902	7	0.016
coins greek 07 noise	none	0.2213	0.7949	0.2347	0.3624	8	0.007
coins greek 07 noise	gaussian	0.555	0.9224	0.5822	0.7139	15	0.008
coins greek 07 noise	median	0.5815	0.9283	0.6089	0.7354	18	0.006
coins greek 07 noise	bilateral	0.5703	0.9244	0.5982	0.7264	17	0.005

Table 9. Effect of the smoothing filter on the two noisy images, with Otsu thresholding held fixed.

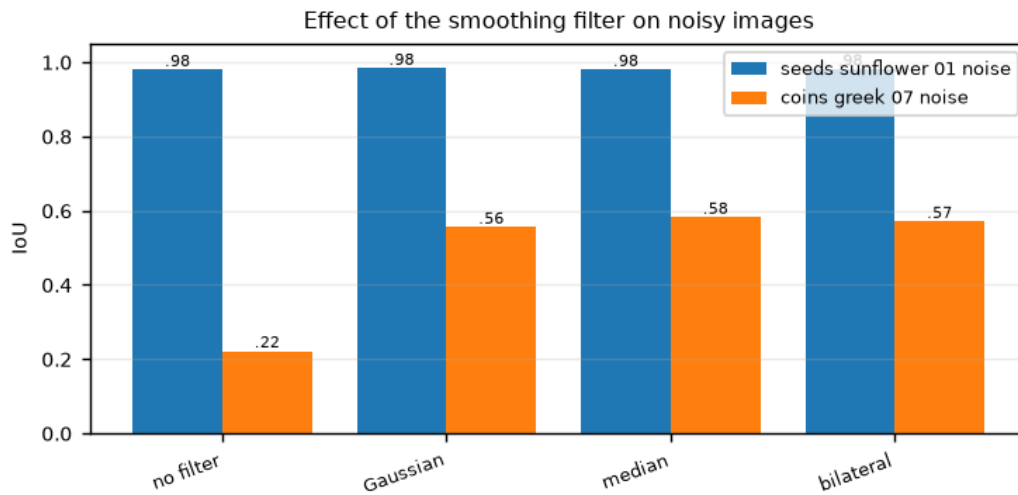


Figure 15. Without a smoothing filter the noise fragments the mask into thousands of small regions, and any of the three filters repairs most of the damage.

5.6 Robustness to imaging conditions

image	condition	method	IoU	F1	regions
seeds sunflower 01	original photograph	Thresholding (Otsu)	0.9888	0.9944	7
seeds sunflower 01	original photograph	Region growing (tol 25)	0.3528	0.5216	13
seeds sunflower 01	original photograph	K means (K = 3, Lab)	0.9374	0.9677	7
seeds sunflower 01 illum	Strong lighting gradient	Thresholding (Otsu)	0.9798	0.9898	7
seeds sunflower 01 illum	Strong lighting gradient	Region growing (tol 25)	0.4588	0.6291	7
seeds sunflower 01 illum	Strong lighting gradient	K means (K = 3, Lab)	0.8576	0.9233	9
seeds sunflower 01 dim	Under exposed, low contrast	Thresholding (Otsu)	0.9863	0.9931	7
seeds sunflower 01 dim	Under exposed, low contrast	Region growing (tol 25)	0.4105	0.5821	12
seeds sunflower 01 dim	Under exposed, low contrast	K means (K = 3, Lab)	0.9325	0.9651	7

image	condition	method	IoU	F1	regions
seeds sunflower 01 noise	Additive Gaussian sensor noise	Thresholding (Otsu)	0.9829	0.9914	7
seeds sunflower 01 noise	Additive Gaussian sensor noise	Region growing (tol 25)	0.4379	0.609	10
seeds sunflower 01 noise	Additive Gaussian sensor noise	K means (K = 3, Lab)	0.9513	0.975	7
seeds sunflower 01 blur	Defocus blur	Thresholding (Otsu)	0.9847	0.9923	7
seeds sunflower 01 blur	Defocus blur	Region growing (tol 25)	0.2134	0.3518	7
seeds sunflower 01 blur	Defocus blur	K means (K = 3, Lab)	0.9293	0.9633	7
coins greek 07	original photograph	Thresholding (Otsu)	0.5935	0.7449	18
coins greek 07	original photograph	Region growing (tol 25)	0.6434	0.783	18
coins greek 07	original photograph	K means (K = 3, Lab)	0.8013	0.8897	24
coins greek 07 illum	Strong lighting gradient	Thresholding (Otsu)	0.61	0.7578	18
coins greek 07 illum	Strong lighting gradient	Region growing (tol 25)	0.6597	0.795	19
coins greek 07 illum	Strong lighting gradient	K means (K = 3, Lab)	0.8298	0.907	22
coins greek 07 noise	Additive Gaussian sensor noise	Thresholding (Otsu)	0.555	0.7139	15
coins greek 07 noise	Additive Gaussian sensor noise	Region growing (tol 25)	0.6449	0.7841	18
coins greek 07 noise	Additive Gaussian sensor noise	K means (K = 3, Lab)	0.782	0.8776	24

Table 10. Every method under every imaging condition.

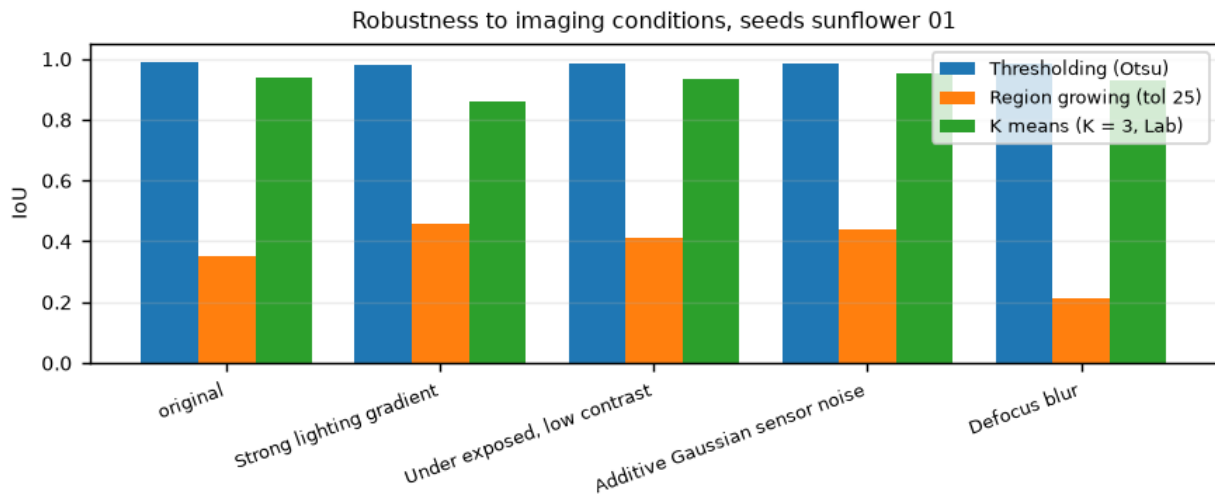


Figure 16. Robustness on the sunflower seed photograph.

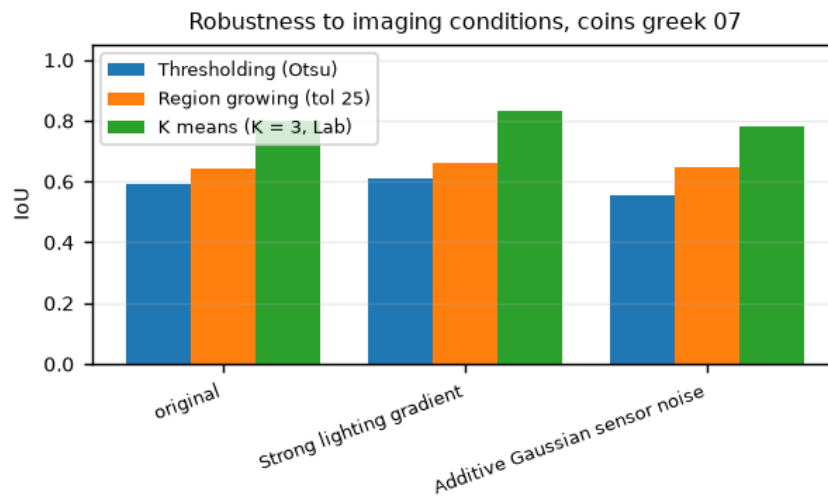


Figure 17. Robustness on the coins photograph.

Three observations follow from Table 10. First, the defocus blur is the mildest of the four conditions for every method, because blurring moves the boundary of an object by a few pixels but does not change which side of the threshold the interior of the object falls on. Second, the under exposure is almost harmless for the threshold and the clustering, because both are invariant to a monotone rescaling of the grey levels as long as the two populations stay separable, while it does hurt the region growing, whose tolerance is an absolute number of grey levels and is therefore far too wide once the contrast has been compressed. Third, the small positive changes in the region growing column of Table 5 are not evidence that noise helps. They arise because the region growing under grows on the clean photographs, so its baseline is already low, and a condition that changes the local statistics can accidentally let a region cover more of an object than it did before. A score that moves in the right direction for the wrong reason is still a weakness of the method, not a strength.

5.7 Separating touching objects

image	objects present	marker distance ratio	regions separation without	regions separation after
beans coffee 06	5	0.25	2	3
beans coffee 06	5	0.35	2	3
beans coffee 06	5	0.45	2	3
beans coffee 06	5	0.55	2	3
beans coffee 06	5	0.65	2	4
coins greek 07	24	0.25	18	17
coins greek 07	24	0.35	18	17
coins greek 07	24	0.45	18	16
coins greek 07	24	0.55	18	16
coins greek 07	24	0.65	18	16
beans cowpea scale 05	8	0.25	9	9
beans cowpea scale 05	8	0.35	9	9
beans cowpea scale 05	8	0.45	9	8
beans cowpea scale 05	8	0.55	9	8
beans cowpea scale 05	8	0.65	9	8
seeds sunflower 01	not counted	0.25	7	11
seeds sunflower 01	not counted	0.35	7	10
seeds sunflower 01	not counted	0.45	7	13
seeds sunflower 01	not counted	0.55	7	12
seeds sunflower 01	not counted	0.65	7	16

Table 11. Effect of the marker threshold used to separate touching objects.

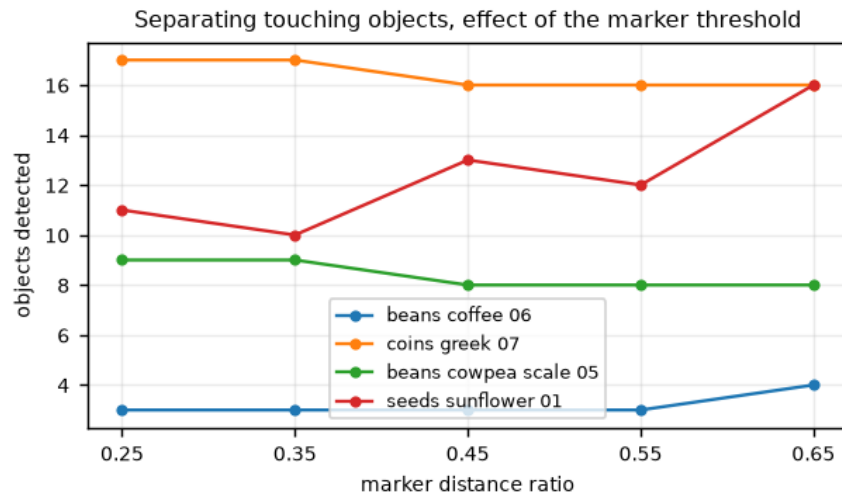


Figure 18. Number of regions reported as a function of the marker threshold.

5.8 Object measurement

image	objects present	count before separation	count after separation
beans cowpea scale 05	8	9	9
beans coffee 06	5	2	3
coins greek 07	24	18	17
seeds pale 04	3	6	12
seeds sunflower 01	not counted	7	12

Table 12. Object counts produced by the full pipeline.

image	method	matched objects	mean area error (%)	max area error (%)	mean error (%)	length error (%)	max length error (%)
beans cowpea scale 05	Thresholding (Otsu)	8	11.6	18.95	1.37	2.55	
beans cowpea scale 05	K means (K = 3, Lab)	8	11.67	19.0	1.41	2.66	
beans coffee 06	Thresholding (Otsu)	2	1.8	2.48	1.85	2.62	
beans coffee 06	K means (K = 3, Lab)	1	10.44	10.44	6.5	6.5	
coins greek 07	Thresholding (Otsu)	14	13.44	29.8	12.12	28.03	
coins greek 07	K means (K = 3, Lab)	22	17.51	31.56	14.97	31.39	
seeds pale 04	Thresholding (Otsu)	0					
seeds pale 04	K means (K = 3, Lab)	3	15.05	21.39	2.9	4.27	

Table 13. Accuracy of the measurements of the objects that were correctly found, relative to the ground truth regions.

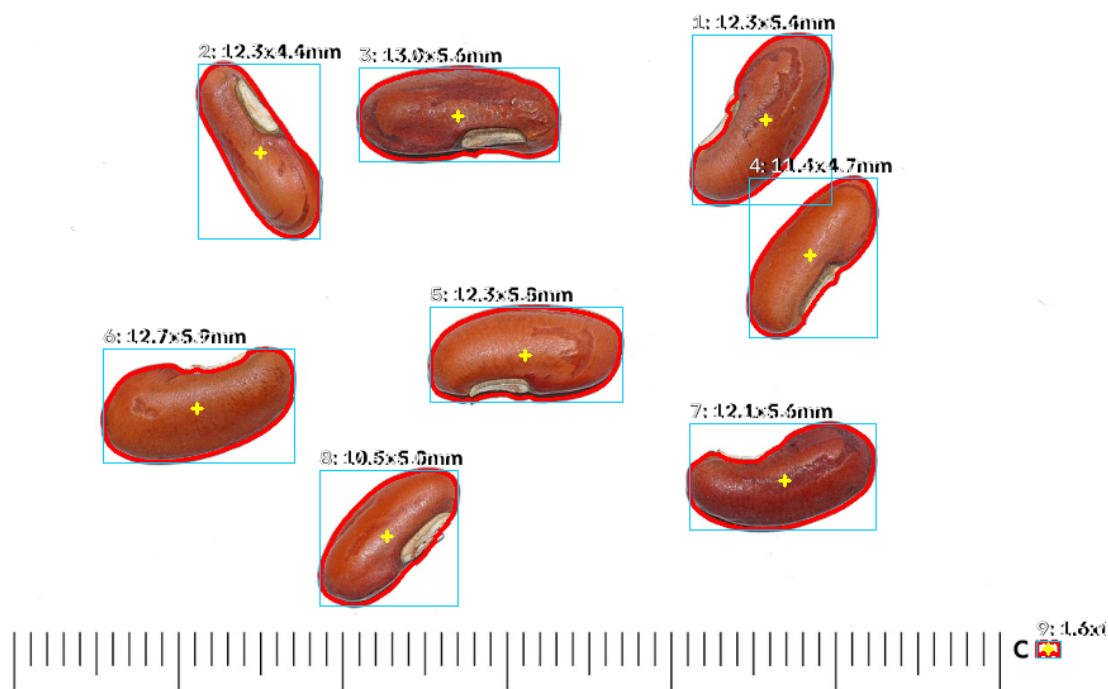


Figure 19. The measurement output on the calibrated photograph. Every seed is labelled with its length and width in millimetres, computed from the ellipse fitted to its contour and the calibration of 13.0 pixels per millimetre. The measured lengths of 10.5 to 13.0 millimetres and widths of 4.4 to 5.9 millimetres are consistent with the published dimensions of cowpea seed, which is an independent check on the calibration. Region 9 is the letter m of the printed unit label, a false detection that is discussed in section 7.

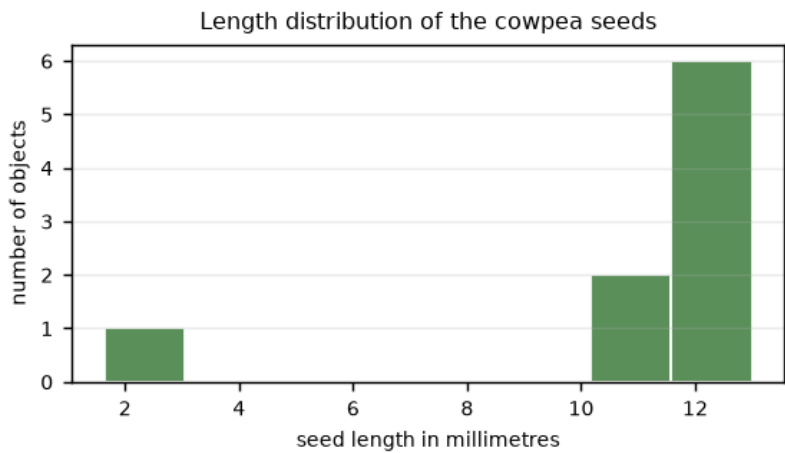


Figure 20. Length distribution of the eight measured seeds.



Figure. Segmentation, separation of touching objects and measurement on beans coffee 06.



Figure. Segmentation, separation of touching objects and measurement on coins greek 07.

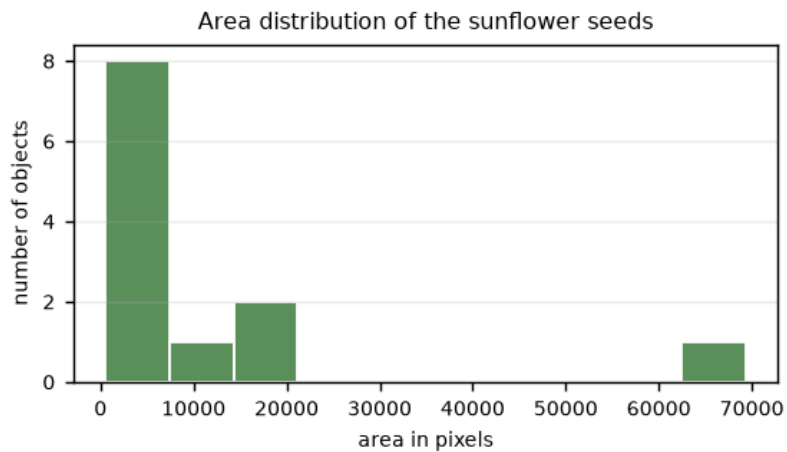


Figure 21. Area distribution of the regions found in the sunflower photograph. The long tail on the right is produced by the clusters of touching seeds that were not separated.

6. Critical analysis

6.1 Which segmentation method produced the best results and why

K means clustering in the Lab colour space with three clusters produced the highest mean Intersection over Union, 0.815, ahead of Otsu thresholding at 0.7095 and region growing at 0.3652. Two properties explain the difference. First, the clustering is the only method of the three that can use colour: on the coins photograph it separates the dark background from the coins where a grey level threshold cannot, and on the low contrast seed photograph it uses the slight colour difference between the cream seeds and the bluish white paper, which raises the IoU from 0.1743 to 0.6076. Second, a third cluster gives the method somewhere to put the shadow, so shadow pixels are not forced into either the object class or the background class.

The advantage comes at a price. The clustering is roughly two orders of magnitude slower than the threshold, because it iterates over every pixel many times, while Otsu touches the image once to build a histogram. It is also worth stating clearly that K means with two clusters on the intensity channel is not a genuinely different method: it minimises the within class variance of a two class partition of the grey levels, which is the same objective that Otsu maximises in its complementary form. In the study of section 5.4, K means with two clusters on the intensity channel reproduced the Otsu result exactly on two of the three photographs and differed by less than one hundredth of an IoU point on the third, which is a useful confirmation that both implementations are correct.

6.2 How illumination changes affected thresholding

A global threshold assumes that one grey level separates object from background everywhere in the image. A lighting gradient breaks that assumption directly. On the photograph with the diagonal ramp, Otsu without the flat field correction scored an IoU of 0.6508, and the same threshold after the correction scored 0.9798. The error is systematic rather than random: in the brightly lit part of the image the background is pushed above the threshold, so the mask is correct there, while in the dim part the background falls below the threshold and is accepted as object, which is why the precision collapses while the recall stays high. The locally adaptive threshold reaches a similar accuracy to the corrected global threshold on the gradient images, because it recomputes the decision in every neighbourhood, but it is noticeably worse on the clean images because it also responds to the texture inside the objects.

The correction is not free, and the coins photograph shows the cost clearly. There the flat field step lowered the score from 0.7092 to 0.5935. The reason is that this image has a dark background, so the estimated background field is small, and dividing by a small number amplifies both the noise and the residual texture of the background until part of it crosses the threshold. The lesson is that a flat field correction should be applied when the illumination varies across the field of view, and not as a reflex on every image: on a dark background it is safer to work on the boundaries of the objects, as the edge based recipe in section 6.7 does.

6.3 How seed point selection influenced region growing

Seed selection changed the result more than any other single parameter. On the coins photograph the informed distance transform strategy placed one seed near the centre of every detected coin and reached an IoU of 0.6434, while the regular grid placed only five usable seeds and reached 0.1404, a loss of more than a factor of five. The reason is structural: region growing can never recover an object that contains no seed, so a missing seed is a guaranteed false negative rather than a small error. On the sunflower photograph the ordering reverses, because the coarse mask merges touching seeds into seven components and therefore yields only seven seeds, while the random strategy scatters forty seeds and covers far more of the seed area. The practical conclusion is that region growing is only as good as the procedure that produces its seeds, and that procedure is itself usually a segmentation, which makes the method circular unless the seeds come from a human or from a physically motivated prior.

The similarity criterion mattered less. The running mean reference was slightly better than the fixed seed value reference on both images, because the objects in this dataset have a wide internal intensity range and a region that can follow a drift covers more of the object before it stops.

6.4 What effect changing K had on K means segmentation

The sweep over K in Table 8 is run on the intensity feature space, so that the effect of the number of clusters is not mixed with the effect of the features. The effect of K depends entirely on how many populations the image really contains. On the sunflower photograph, which has one object population and one background population, the best value was K equal to two and every further cluster split the object itself, so the score fell monotonically from 0.9888 at K equal to two to 0.8245 at K equal to six. On the coins photograph the opposite happened: the background is not uniform, it has a bright region and a dark corner, so K equal to two forced the dark corner and the dim coins into the same cluster and the score was only 0.5852, while K equal to four separated them and reached 0.7902. In short, K must match the number of distinguishable populations in the image, not the number of object classes the user is interested in.

6.5 Which method was most computationally efficient

Thresholding, by a wide margin. Its mean processing time was 0.032 seconds per image against 3.142 seconds for the region growing and 7.122 seconds for the clustering at the common working resolution of 900 pixels. The ordering follows directly from the algorithms. Otsu makes one pass over the image to build a histogram and then searches 256 candidate thresholds, so its cost is one pass over the pixels. Lloyd's algorithm makes one distance computation per pixel per cluster per iteration, so its cost is approximately K times the number of iterations passes over the pixels, and the five dimensional feature space multiplies that again. The region growing visits every pixel once, but it does so in an interpreted loop with a queue and a per pixel predicate, so its constant factor is far larger than the vectorised operations used by the other two. A compiled implementation of the same algorithm would close most of that gap, so the timing of the region growing should be read as a property of this implementation and not of the algorithm.

6.6 Under what conditions did each method fail

condition	thresholding	region growing	K means
Lighting gradient	Fails without a flat field correction, the background of the dim side is accepted as object.	Degrades gracefully, because the predicate is local, but regions leak across the gradient when the tolerance is high.	Degrades, because a brightness gradient creates spurious clusters that cut across the object and background populations.
Low contrast	Fails completely, the histogram has a single mode so the threshold is meaningless.	Leaks into the background, since object and background differ by less than the tolerance.	Best of the three, because the residual colour difference survives even when the brightness difference does not.
Noise	Fragments the mask into many small regions unless a smoothing filter is applied first.	Stops early, because a noisy neighbour fails the predicate.	Fairly robust, because averaging over a cluster suppresses zero mean noise.
Touching objects	Reports one region for the whole group.	Same, unless a separate seed is placed in each object and the tolerance is tight.	Same, since clustering has no notion of connectivity.
Objects of different sizes	Handled, provided the area filter is not set too high.	Handled, provided each object receives a seed.	Handled, the cluster assignment does not depend on region size.
Shadows	Shadow pixels are darker than the background and are accepted as object.	Regions grow into the penumbra when the tolerance exceeds the shadow step.	A third cluster absorbs the shadow, which is why K equal to three helps on the coffee photograph.

Table 14. Failure conditions of the three techniques.

6.7 How techniques studied in earlier weeks improve the results

- **Morphological filtering.** Opening, closing and hole filling turn a ragged raw threshold into a usable mask. On the coffee beans the specular highlights inside the objects appear as holes in the raw mask, and the hole filling step alone repairs them.
- **Flat field correction by large scale morphology.** Estimating the background with a structuring element larger than any object and dividing it out restores a bimodal histogram, and it is what makes the global threshold usable on the gradient images.
- **Edge detection.** The boundaries of the coins are far more reliable than their absolute grey levels, and an edge based recipe followed by a morphological closing and a hole fill segments that photograph much better than a global threshold. The same recipe was used to draft the ground truth for that image.

- **The distance transform and the watershed.** These separate touching objects that connected component labelling merges, which is what converts a foreground mask into an object count.
- **Colour space transformation.** Working in CIE Lab rather than in grey levels is what allows the ruler to be rejected in the cowpea photograph and the pale seeds to be found at all.
- **Shape descriptors.** Circularity, solidity and the aspect ratio provide a cheap post filter: on the cowpea photograph the false detection produced by the printed unit label has a much lower area and a different aspect ratio from any seed, so a simple shape rule removes it.

7. Limitations and failure cases

7.1 Dense piles of objects

The clearest limitation of the whole approach is that it assumes one connected region corresponds to one object. The pinto bean photograph breaks that assumption: about two hundred beans are piled so that almost every bean touches several neighbours, and the segmentation returns a single region covering the entire pile. The distance transform and watershed step improves this but does not solve it, because the ridges between overlapping beans are not deep enough for the distance transform to produce one marker per bean. Solving this case properly requires either a different acquisition protocol, spreading the beans out so that they do not touch, or a method that models object appearance rather than object connectivity.



Figure 22. The dense pile failure case. The pile is segmented perfectly as foreground, and the object count is still wrong by two orders of magnitude.

7.2 Low contrast objects

When the object and the background have nearly the same brightness the grey level histogram has a single mode and the Otsu criterion has no meaningful maximum to find. It then selects a threshold inside the object population and cuts the objects into pieces. Only the colour based clustering recovers the objects on this photograph, and even it merges the darker striped regions of the seed with its bright out of focus lobe imperfectly.

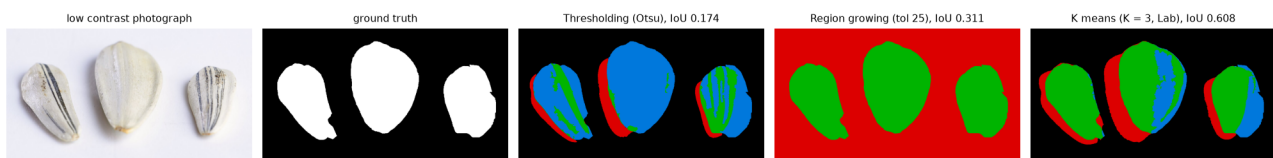


Figure 23. The low contrast failure case for each method.

7.3 Shadows and specular highlights

A cast shadow is darker than the background, so any method that defines objects as dark pixels will accept part of the shadow as object. This is the main reason the measured areas on the coffee and cowpea photographs are larger than the ground truth areas, by about 11.6 per cent on average in Table 13, while the measured lengths are accurate to about 1.37 per cent. A shadow adds a thin halo around the whole object, which changes the area much more than it changes the fitted major axis.

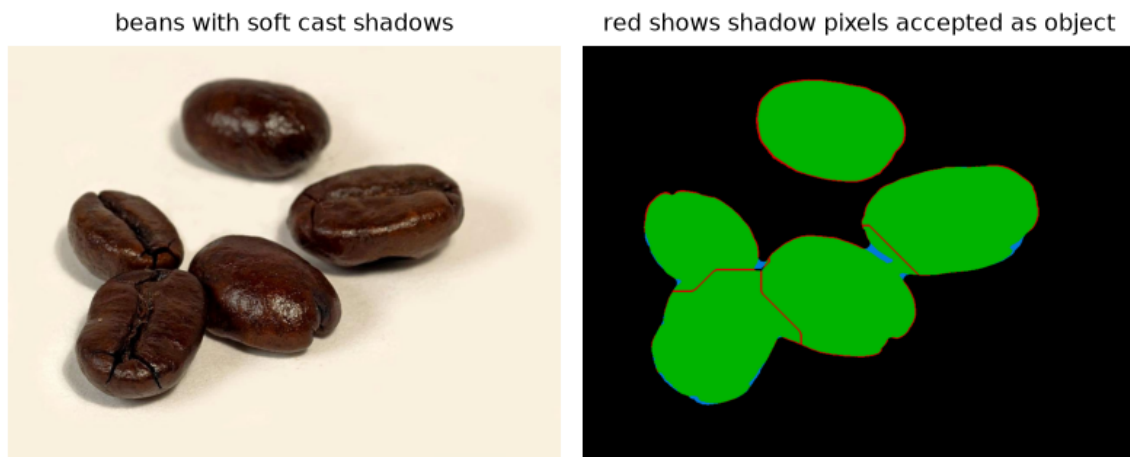


Figure 24. Shadow pixels accepted as object, shown in red.

7.4 Other limitations

- The system has no notion of what an object is. Any dark connected region large enough to survive the area filter is reported, which is why the letter m of the printed unit label in the cowpea photograph is counted as a ninth object.
- Objects cut by the image border are measured as if they were whole. The measurement module flags them with a `touches_border` column so that they can be excluded, but the count still includes them.
- Measurements are reported in millimetres only when the photograph contains a scale. Without a scale, only relative sizes are meaningful.
- The ground truth is not perfect. It was produced with an assisted procedure and inspected twice, and the remaining boundary error of a few pixels per object places an upper bound of roughly one to two per cent on the IoU figures that can be trusted.
- The region growing implementation is interpreted Python, so its timing is not comparable with the vectorised routines used by the other two methods.
- All experiments are run at a common working resolution of 900 pixels. Absolute pixel measurements would change at another resolution, although the relative comparisons would not.

8. Conclusion

A complete classical pipeline was built and evaluated: image preparation, three segmentation techniques implemented from first principles, morphological clean up, boundary identification, separation of touching objects, object extraction, object measurement in pixels and in millimetres, and a quantitative evaluation against manually verified ground truth.

On the five annotated photographs, K means clustering in the Lab colour space reached a mean Intersection over Union of 0.815, Otsu thresholding reached 0.7095 and region growing reached 0.3652. Thresholding was about 223 times faster than the clustering and is the right default whenever the histogram is bimodal and the illumination has been corrected. Clustering earns its extra cost only when colour carries information that brightness does not, which was the case on the coins and on the low contrast seeds. Region growing was the weakest of the three in an automatic setting, not because the algorithm is poor but because its result is decided by the seed set, and producing a good seed set automatically already requires a segmentation.

For the objects that were correctly found, the measurements are usable: on the calibrated photograph all eight cowpea seeds were matched to the ground truth and their lengths were reported to within about one and a half per cent, which is adequate for seed grading, while their areas carried a larger error because the segmentation includes part of the cast shadow.

The clearest lesson of the experiments is that the acquisition matters more than the choice among the three segmentation techniques. Diffuse and even lighting, a background whose colour contrasts with the objects, and objects that are spread out rather than piled together move every method into the range above 0.95 IoU. When those conditions are absent, no amount of parameter tuning on a classical method recovers the lost information.

9. References

- [1] Otsu, N. (1979). A threshold selection method from gray level histograms. IEEE Transactions on Systems, Man and Cybernetics, 9(1), 62 to 66.
- [2] Adams, R. and Bischof, L. (1994). Seeded region growing. IEEE Transactions on Pattern Analysis and Machine Intelligence, 16(6), 641 to 647.
- [3] Lloyd, S. P. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129 to 137.
- [4] Arthur, D. and Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. Proceedings of the ACM SIAM Symposium on Discrete Algorithms, 1027 to 1035.
- [5] Beucher, S. and Meyer, F. (1993). The morphological approach to segmentation: the watershed transformation. Mathematical Morphology in Image Processing, 433 to 481.
- [6] Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679 to 698.
- [7] Gonzalez, R. C. and Woods, R. E. (2018). Digital Image Processing, fourth edition. Pearson.
- [8] Bradski, G. (2000). The OpenCV library. Dr Dobb's Journal of Software Tools.
- [9] Rother, C., Kolmogorov, V. and Blake, A. (2004). GrabCut, interactive foreground extraction using iterated graph cuts. ACM Transactions on Graphics, 23(3), 309 to 314. Used only as an annotation aid when the hand traced outlines were tightened.

Image sources

identifier	source	licence
seeds_sunflower_01	Wikimedia Commons, File:Sunflower seeds. img 007.jpg	CC BY-SA 4.0
seeds_sunflower_02	Wikimedia Commons, File:Sunflower seeds. img 008.jpg	CC BY-SA 4.0
seeds_sunflower_03	Wikimedia Commons, File:Sunflower seeds. img 009.jpg	CC BY-SA 4.0
seeds_pale_04	Wikimedia Commons, File:Sunflower seeds - white. img 09.jpg	CC BY-SA 4.0
beans_cowpea_scale_05	Wikimedia Commons, File:Vigna unguiculata 03.jpg	CC BY-SA 3.0

identifier	source	licence
beans_coffee_06	Wikimedia Commons, File:Dark roasted coffee on white background.jpg	CC BY-SA 4.0
coins_greek_07	scikit-image sample data, coins.png, Brooklyn Museum collection	Public domain
beans_pinto_dense_08	Wikimedia Commons, File:Pinto Beans Seeds.jpg	CC BY-SA 4.0

Table 15. Provenance of every photograph in the dataset. All of them are real photographs.

Appendix A. Complete source code

The listing below is the complete source of the project, in the order in which the modules are described in Table 2. The same files are included in the src directory of the repository.

src/config.py

```
"""
Project wide configuration and the dataset manifest.

Automated Image Segmentation and Object Measurement System

Author : Nii Yartey Gidiglo
"""

from pathlib import Path

# -----
# Paths
# -----
SRC_DIR = Path(__file__).resolve().parent
ROOT_DIR = SRC_DIR.parent
DATA_DIR = ROOT_DIR / "data"
IMAGE_DIR = DATA_DIR / "images"
GT_DIR = DATA_DIR / "ground_truth"
RESULTS_DIR = ROOT_DIR / "results"
FIG_DIR = RESULTS_DIR / "figures"
TABLE_DIR = RESULTS_DIR / "tables"
MEAS_DIR = RESULTS_DIR / "measurements"
REPORT_DIR = ROOT_DIR / "report"

for _d in (IMAGE_DIR, GT_DIR, FIG_DIR, TABLE_DIR, MEAS_DIR, REPORT_DIR):
    _d.mkdir(parents=True, exist_ok=True)

# Longest side that images are resized to before processing. Keeping every
# image at a common working resolution makes the timing comparison fair and
# keeps the morphological kernel sizes meaningful across the dataset.
WORK_MAX_SIDE = 900

RANDOM_SEED = 42

# -----
# Dataset manifest
# -----
# Every base image is a real photograph. Nothing in this dataset was generated
# by an image generation model. The source URL and licence of each photograph
# are recorded here and reproduced in the report.
#
# The "variant" images are controlled degradations that are computed from the
# base photographs by src/dataset.py (illumination gradient, dim exposure,
# sensor noise, defocus blur). They are used for the robustness study, and
# they inherit the ground truth mask of the base image because the geometry of
# the objects is unchanged.
# -----

BASE_IMAGES = [
    {
        "id": "seeds_sunflower_01",
        "file": "seeds_sunflower_01.jpg",
        "title": "Sunflower seeds, scattered",
        "domain": "Counting and measuring seeds",
        "expected_count": None,
        "px_per_mm": None,
        "ground_truth": "pixel",
        "notes": (
            "Dark seeds on a bright uniform surface. Most seeds are well "
            "separated, a cluster on the right is touching, and several seeds "
            "are cut by the image border."
        ),
        "source": "Wikimedia Commons, File:Sunflower seeds. img 007.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Sunflower_seeds._img_007.jpg",
        "licence": "CC BY-SA 4.0",
    },
    {
        "id": "seeds_sunflower_02",
        "file": "seeds_sunflower_02.jpg",
        "title": "Sunflower seeds, second layout",
        "domain": "Counting and measuring seeds",
        "expected_count": None,
        "px_per_mm": None,
        "ground_truth": None,
        "notes": "Same object class, different arrangement, more touching pairs.",
        "source": "Wikimedia Commons, File:Sunflower seeds. img 008.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Sunflower_seeds._img_008.jpg",
        "licence": "CC BY-SA 4.0",
    },
    {
        "id": "seeds_sunflower_03",
        "file": "seeds_sunflower_03.jpg",
        "title": "Sunflower seeds, third layout",
        "domain": "Counting and measuring seeds",
        "expected_count": None,
        "px_per_mm": None,
        "ground_truth": None,
        "notes": "Shallow depth of field, part of the field is defocused.",
        "source": "Wikimedia Commons, File:Sunflower seeds. img 009.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Sunflower_seeds._img_009.jpg",
        "licence": "CC BY-SA 4.0",
    },
    {
        "id": "seeds_pale_04",
        "file": "seeds_pale_04.jpg",
    },
]
```

```

        "title": "Pale sunflower seeds of three sizes",
        "domain": "Counting and measuring seeds",
        "expected_count": 3,
        "px_per_mm": None,
        "ground_truth": "instance",
        "notes": (
            "Low contrast case. The seeds are almost the same brightness as "
            "the background, and the three objects differ strongly in size."
        ),
        "source": "Wikimedia Commons, File:Sunflower seeds - white. img 09.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Sunflower_seeds_-_white._img_09.jpg",
        "licence": "CC BY-SA 4.0",
    },
    {
        "id": "beans_cowpea_scale_05",
        "file": "beans_cowpea_scale_05.jpg",
        "title": "Cowpea seeds photographed with a centimetre scale",
        "domain": "Analysing objects in agricultural images",
        "expected_count": 8,
        # Measured in src/calibration.py from the printed ruler in the photo.
        "px_per_mm": 13.0,
        "ground_truth": "instance",
        "notes": (
            "Eight well separated cowpea seeds photographed next to a printed "
            "centimetre ruler, which allows pixel measurements to be converted "
            "into millimetres."
        ),
        "source": "Wikimedia Commons, File:Vigna unguiculata 03.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Vigna_unguiculata_03.jpg",
        "licence": "CC BY-SA 3.0",
        "has_ruler": True,
    },
    {
        "id": "beans_coffee_06",
        "file": "beans_coffee_06.jpg",
        "title": "Roasted coffee beans with cast shadows",
        "domain": "Separating foreground objects from a simple background",
        "expected_count": 5,
        "px_per_mm": None,
        "ground_truth": "instance",
        "notes": (
            "Five glossy beans on a cream surface. Two beans touch, all of "
            "them cast soft shadows, and specular highlights sit inside the "
            "objects."
        ),
        "source": "Wikimedia Commons, File:Dark roasted coffee on white background.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Dark_roasted_coffee_on_white_background.jpg",
        "licence": "CC BY-SA 4.0",
    },
    {
        "id": "coins_greek_07",
        "file": "coins_greek_07.png",
        "title": "Twenty four Greek coins",
        "domain": "Identifying coins or circular objects",
        "expected_count": 24,
        "px_per_mm": None,
        "ground_truth": "instance",
        "notes": (
            "The classic coins photograph distributed with scikit-image. The "
            "background is dark, the illumination falls off towards the "
            "bottom left corner, and coin brightness overlaps background "
            "brightness in the darkest corner."
        ),
        "source": "scikit-image sample data, coins.png, Brooklyn Museum collection",
        "url": "https://github.com/scikit-image/scikit-image/blob/main/skimage/data/coins.png",
        "licence": "Public domain",
    },
    {
        "id": "beans_pinto_dense_08",
        "file": "beans_pinto_dense_08.jpg",
        "title": "Densely piled pinto beans",
        "domain": "Counting and measuring seeds, hard case",
        "expected_count": None,
        "px_per_mm": None,
        "ground_truth": None,
        "notes": (
            "A deliberate stress case. Roughly two hundred beans are piled so "
            "that almost every bean touches or overlaps a neighbour, which "
            "breaks the one region per object assumption."
        ),
        "source": "Wikimedia Commons, File:Pinto Beans Seeds.jpg",
        "url": "https://commons.wikimedia.org/wiki/File:Pinto_Beans_Seeds.jpg",
        "licence": "CC BY-SA 4.0",
    },
]

# Controlled degradations applied to two of the base photographs.
VARIANTS = [
    {"id": "seeds_sunflower_01_illum", "base": "seeds_sunflower_01",
     "kind": "illumination_gradient", "label": "Strong lighting gradient"},
    {"id": "seeds_sunflower_01_dim", "base": "seeds_sunflower_01",
     "kind": "dim", "label": "Under exposed, low contrast"},
    {"id": "seeds_sunflower_01_noise", "base": "seeds_sunflower_01",
     "kind": "noise", "label": "Additive Gaussian sensor noise"},
    {"id": "seeds_sunflower_01_blur", "base": "seeds_sunflower_01",
     "kind": "blur", "label": "Defocus blur"},
    {"id": "coins_greek_07_illum", "base": "coins_greek_07",
     "kind": "illumination_gradient", "label": "Strong lighting gradient"},
    {"id": "coins_greek_07_noise", "base": "coins_greek_07",
     "kind": "noise", "label": "Additive Gaussian sensor noise"},
]

def has_ground_truth(image_id):
    """True when the image, or the image a variant was derived from, has a
    manually verified ground truth mask."""

```

```

    rec = image_record(image_id)
    return rec.get("ground_truth") is not None

def has_instance_ground_truth(image_id):
    """True when the ground truth carries one region per physical object, so
    that object level counting scores are meaningful."""
    rec = image_record(image_id)
    return rec.get("ground_truth") == "instance"

def image_record(image_id):
    """Return the manifest record for a base image or a variant."""
    for rec in BASE_IMAGES:
        if rec["id"] == image_id:
            return rec
    for var in VARIANTS:
        if var["id"] == image_id:
            base = image_record(var["base"])
            rec = dict(base)
            rec.update({
                "id": var["id"],
                "file": var["id"] + ".png",
                "title": base["title"] + ", " + var["label"].lower(),
                "variant_of": var["base"],
                "variant_kind": var["kind"],
                "variant_label": var["label"],
            })
            return rec
    raise KeyError(image_id)

def all_image_ids(include_variants=True):
    ids = [r["id"] for r in BASE_IMAGES]
    if include_variants:
        ids += [v["id"] for v in VARIANTS]
    return ids

```

src/dataset.py

```

"""
Dataset handling.

The eight base images are real photographs that are shipped with this
project in data/images. This module can re download them from their source
if they are missing, and it generates the controlled degradations that are used
for the robustness study.
"""

import subprocess
import cv2
import numpy as np

from config import BASE_IMAGES, IMAGE_DIR, VARIANTS, RANDOM_SEED, image_record
from preprocessing import load_bgr, resize_max_side

DOWNLOAD_URLS = {
    "seeds_sunflower_01.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/5/56/"
        "Sunflower_seeds._img_007.jpg/1280px-Sunflower_seeds._img_007.jpg",
    "seeds_sunflower_02.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/7/7a/"
        "Sunflower_seeds._img_008.jpg/1280px-Sunflower_seeds._img_008.jpg",
    "seeds_sunflower_03.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/c/cf/"
        "Sunflower_seeds._img_009.jpg/1280px-Sunflower_seeds._img_009.jpg",
    "seeds_pale_04.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/6/69/"
        "Sunflower_seeds_-_white._img_09.jpg/1280px-Sunflower_seeds_-_white._img_09.jpg",
    "beans_cowpea_scale_05.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/5/56/"
        "Vigna_unguiculata_03.jpg/1280px-Vigna_unguiculata_03.jpg",
    "beans_coffee_06.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/5/5b/"
        "Dark_roasted_coffee_on_white_background.jpg",
    "coins_greek_07.png":
        "https://raw.githubusercontent.com/scikit-image/scikit-image/"
        "v0.24.0/skimage/data/coins.png",
    "beans_pinto_dense_08.jpg":
        "https://upload.wikimedia.org/wikipedia/commons/thumb/3/3f/"
        "Pinto_Beans_Seeds.jpg/1280px-Pinto_Beans_Seeds.jpg",
}

def ensure_base_images():
    """Download any base photograph that is not already present."""
    missing = [r for r in BASE_IMAGES if not (IMAGE_DIR / r["file"]).exists()]
    for rec in missing:
        url = DOWNLOAD_URLS[rec["file"]]
        target = IMAGE_DIR / rec["file"]
        print("downloading", rec["file"])
        subprocess.run(["curl", "-sL", "-o", str(target),
            "-A", "segmentation-measurement/1.0", url], check=True)
    return [IMAGE_DIR / r["file"] for r in BASE_IMAGES]

# -----
# Controlled degradations
# -----
def apply_illumination_gradient(img, low=0.30, high=1.20):
    """Multiply the image by a diagonal brightness ramp.
    This reproduces the very common situation of a scene lit from one side,
    and it is the condition that breaks a single global threshold.
    """

```

```

    h, w = img.shape[:2]
    ys, xs = np.mgrid[0:h, 0:w].astype(np.float32)
    ramp = (xs / max(1, w - 1) + ys / max(1, h - 1)) / 2.0
    field = low + (high - low) * ramp
    out = img.astype(np.float32) * field[:, :, None]
    return np.clip(out, 0, 255).astype(np.uint8)

def apply_dim(img, gain=0.45, lift=25):
    """Under expose the image and lift the black point, which compresses the
    contrast between object and background."""
    out = img.astype(np.float32) * gain + lift
    return np.clip(out, 0, 255).astype(np.uint8)

def apply_noise(img, sigma=18.0, seed=RANDOM_SEED):
    """Additive zero mean Gaussian noise, the standard sensor noise model."""
    rng = np.random.default_rng(seed)
    noise = rng.normal(0.0, sigma, img.shape)
    return np.clip(img.astype(np.float32) + noise, 0, 255).astype(np.uint8)

def apply_blur(img, ksize=15):
    """Defocus blur, modelled with a Gaussian point spread function."""
    return cv2.GaussianBlur(img, (ksize, ksize), 0)

_VARIANT_FUNCS = {
    "illumination_gradient": apply_illumination_gradient,
    "dim": apply_dim,
    "noise": apply_noise,
    "blur": apply_blur,
}

def build_variants(force=False):
    """Create every degraded image listed in the manifest."""
    made = []
    for var in VARIANTS:
        target = IMAGE_DIR / (var["id"] + ".png")
        if target.exists() and not force:
            made.append(target)
            continue
        base = image_record(var["base"])
        img = load_bgr(IMAGE_DIR / base["file"])
        out = _VARIANT_FUNCS[var["kind"]](img)
        cv2.imwrite(str(target), out)
        made.append(target)
    return made

def load_working(image_id):
    """Load one image of the dataset at the common working resolution."""
    rec = image_record(image_id)
    img = load_bgr(IMAGE_DIR / rec["file"])
    work, scale = resize_max_side(img)
    return work, rec, scale
if __name__ == "__main__":
    ensure_base_images()
    paths = build_variants(force=True)
    print("variants written:", len(paths))

```

src/preprocessing.py

```

"""
Image preparation stage of the pipeline.

Everything in this module is deliberately classical: colour conversion,
smoothing filters, contrast handling and a morphological illumination
correction. No learned model is used anywhere in the project.
"""

import cv2
import numpy as np

from config import WORK_MAX_SIDE

# -----
# Basic input handling
# -----
def load_bgr(path):
    """Read an image from disk as BGR, raising a clear error if it is missing."""
    img = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if img is None:
        raise FileNotFoundError("Could not read image: %s" % path)
    return img

def resize_max_side(img, max_side=WORK_MAX_SIDE):
    """Scale an image so that its longest side equals max_side.

    Images are only ever shrunk, never enlarged, so no detail is invented.
    The scale factor is returned so that pixel measurements can be traced back
    to the original photograph if needed.
    """
    h, w = img.shape[:2]
    longest = max(h, w)
    if longest <= max_side:
        return img.copy(), 1.0
    scale = max_side / float(longest)
    out = cv2.resize(img, (int(round(w * scale)), int(round(h * scale))),
                     interpolation=cv2.INTER_AREA)
    return out, scale
def to_gray(img_bgr):

```

```

    return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

def colour_channels(img_bgr):
    """Return a dictionary of candidate single channel representations.

    An appropriate colour representation has to be selected.
    The channels below are the ones examined in the report: grey level, the
    saturation and value channels of HSV, and the a and b channels of CIE Lab.
    """
    hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
    return {
        "gray": cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY),
        "hsv_s": hsv[:, :, 1],
        "hsv_v": hsv[:, :, 2],
        "lab_l": lab[:, :, 0],
        "lab_a": lab[:, :, 1],
        "lab_b": lab[:, :, 2],
    }

def channel_separability(channel, sample_border=12):
    """Rough measure of how well a channel separates object from background.

    The border ring of the image is assumed to be mostly background. The score
    is the absolute difference between the mean of the border ring and the mean
    of the image centre, divided by the pooled standard deviation. It is only
    used to justify the choice of channel, never inside the segmentation.
    """
    h, w = channel.shape[:2]
    b = sample_border
    ring = np.concatenate([
        channel[:b, :].ravel(), channel[-b:, :].ravel(),
        channel[:, :b].ravel(), channel[:, -b:].ravel()])
    centre = channel[h // 4: 3 * h // 4, w // 4: 3 * w // 4].ravel()
    pooled = np.sqrt(0.5 * (ring.std() ** 2 + centre.std() ** 2)) + 1e-6
    return float(abs(float(ring.mean()) - float(centre.mean())) / pooled)

# -----
# Noise suppression
# -----
def denoise(gray, method="gaussian", ksize=5, sigma_colour=45, sigma_space=9):
    """Apply one of three classical smoothing filters.

    gaussian : linear low pass filter, fastest, blurs edges as well as noise
    median : rank filter, removes impulse noise while keeping step edges
    bilateral : edge preserving filter, slowest of the three
    """
    if method == "none":
        return gray.copy()
    if method == "gaussian":
        return cv2.GaussianBlur(gray, (ksize, ksize), 0)
    if method == "median":
        return cv2.medianBlur(gray, ksize)
    if method == "bilateral":
        return cv2.bilateralFilter(gray, ksize, sigma_colour, sigma_space)
    raise ValueError("Unknown denoise method: %s" % method)

# -----
# Contrast and illumination handling
# -----
def clahe(gray, clip=2.0, tiles=8):
    """Contrast limited adaptive histogram equalisation."""
    op = cv2.createCLAHE(clipLimit=clip, tileGridSize=(tiles, tiles))
    return op.apply(gray)

def estimate_background(gray, kernel_frac=0.25, objects_are_dark=True):
    """Estimate the slowly varying background using a large morphological op.

    A structuring element much larger than any object is used, so the operation
    removes the objects and leaves the illumination field. When the objects are
    darker than the background a closing recovers the background, and when they
    are brighter an opening is used instead.
    """
    h, w = gray.shape[:2]
    k = int(max(15, kernel_frac * min(h, w)))
    if k % 2 == 0:
        k += 1
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    op = cv2.MORPH_CLOSE if objects_are_dark else cv2.MORPH_OPEN
    return cv2.morphologyEx(gray, op, se)

def correct_illumination(gray, objects_are_dark=True, kernel_frac=0.25):
    """Flatten an uneven illumination field.

    The estimated background is divided out and the result is rescaled to the
    full eight bit range. This is the classical flat field correction and it is
    what allows a single global threshold to work on images with a lighting
    gradient.
    """
    bg = estimate_background(gray, kernel_frac, objects_are_dark).astype(np.float32)
    bg[bg < 1.0] = 1.0
    flat = gray.astype(np.float32) / bg
    flat = cv2.normalize(flat, None, 0, 255, cv2.NORM_MINMAX)
    return flat.astype(np.uint8)

def objects_are_dark(gray, border=10):
    """Decide the polarity of the scene.

    The border ring of the image is treated as a background sample. If the

```

```

interior is darker than that ring the objects are dark on a bright
background, which is true for every photograph in this dataset except the
coins image.
"""
h, w = gray.shape[:2]
ring = np.concatenate([
    gray[:border, :].ravel(), gray[-border:, :].ravel(),
    gray[:, :border].ravel(), gray[:, -border:].ravel()])
return float(gray.mean()) < float(ring.mean())

# -----
# One call preparation used by the experiment driver
# -----
def prepare(img_bgr, denoise_method="gaussian", ksize=5,
            use_illumination_correction=True, use_clahe=False):
    """Run the standard preparation chain and return the working images.

    Returns a dictionary with the resized colour image, the raw grey image and
    the prepared grey image that the segmentation methods consume.
    """
    work, scale = resize_max_side(img_bgr)
    gray = to_gray(work)
    dark = objects_are_dark(gray)
    prepared = denoise(gray, denoise_method, ksize)
    if use_illumination_correction:
        prepared = correct_illumination(prepared, objects_are_dark=dark)
    if use_clahe:
        prepared = clahe(prepared)
    return {
        "bgr": work,
        "gray": gray,
        "prepared": prepared,
        "objects_are_dark": dark,
        "scale": scale,
    }

```

src/segmentation.py

```

"""
The three segmentation techniques compared in this project.

1. Threshold based segmentation, including an Otsu implementation written
   from first principles so that the criterion itself is visible.
2. Region growing, written from scratch with an explicit seed set, an
   explicit similarity predicate and an explicit queue.
3. K means clustering, written from scratch as Lloyd iterations with a
   k means plus plus initialisation.

Each function returns a binary mask in which object pixels are 255 and
background pixels are 0.
"""

import time

import cv2
import numpy as np

from config import RANDOM_SEED

# =====
# Shared helpers
# =====
def _as_mask(bool_array):
    return (bool_array.astype(np.uint8)) * 255

def postprocess(mask, open_k=3, close_k=5, min_area=60, fill_holes=True,
                clear_border=False):
    """Morphological clean up of a raw binary segmentation.

    Opening removes isolated noise pixels, closing repairs small gaps in the
    object interiors, hole filling closes the specular highlights that appear
    inside glossy objects, and the area filter removes fragments that are far
    too small to be a real object.
    """
    out = mask.copy()
    if open_k and open_k > 1:
        se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (open_k, open_k))
        out = cv2.morphologyEx(out, cv2.MORPH_OPEN, se)
    if close_k and close_k > 1:
        se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close_k, close_k))
        out = cv2.morphologyEx(out, cv2.MORPH_CLOSE, se)
    if fill_holes:
        out = fill_object_holes(out)
    if min_area and min_area > 0:
        out = remove_small_components(out, min_area)
    if clear_border:
        out = remove_border_components(out)
    return out

def fill_object_holes(mask):
    """Fill every hole that is fully enclosed by an object.

    The mask is first padded with a one pixel background border so that the
    flood always starts in background, even when an object touches the corner
    of the image. Whatever background the flood reaches stays background, and
    every enclosed background pocket becomes object.
    """
    h, w = mask.shape[:2]
    padded = cv2.copyMakeBorder(mask, 1, 1, 1, 1, cv2.BORDER_CONSTANT, value=0)
    ff_mask = np.zeros((padded.shape[0] + 2, padded.shape[1] + 2), np.uint8)
    flood = padded.copy()

```

```

cv2.floodFill(flood, ff_mask, (0, 0), 255)
holes = cv2.bitwise_not(flood)[1:h + 1, 1:w + 1]
return cv2.bitwise_or(mask, holes)

def remove_small_components(mask, min_area):
    n, labels, stats, _ = cv2.connectedComponentsWithStats(
        (mask > 0).astype(np.uint8), connectivity=8)
    out = np.zeros_like(mask)
    for i in range(1, n):
        if stats[i, cv2.CC_STAT_AREA] >= min_area:
            out[labels == i] = 255
    return out

def remove_border_components(mask):
    n, labels, _, _ = cv2.connectedComponentsWithStats(
        (mask > 0).astype(np.uint8), connectivity=8)
    border_labels = set(labels[0, :]) | set(labels[-1, :]) | \
        set(labels[:, 0]) | set(labels[:, -1])
    out = np.zeros_like(mask)
    for i in range(1, n):
        if i not in border_labels:
            out[labels == i] = 255
    return out

# =====
# 1. Threshold based segmentation
# =====
def otsu_threshold_from_scratch(gray):
    """Compute Otsu's threshold directly from the definition.

    Otsu chooses the grey level t that maximises the between class variance

    
$$\sigma_b^2(t) = w_0(t) * w_1(t) * (\mu_0(t) - \mu_1(t))^2$$


    where w0 and w1 are the class probabilities and mu0 and mu1 the class
    means. The implementation below evaluates that expression for all 256
    candidate thresholds using cumulative sums, so it runs in linear time in
    the number of grey levels.
    """
    hist = np.bincount(gray.ravel(), minlength=256).astype(np.float64)
    total = hist.sum()
    if total == 0:
        return 0, np.zeros(256)
    p = hist / total
    levels = np.arange(256)
    w0 = np.cumsum(p)
    w1 = 1.0 - w0
    mu_cum = np.cumsum(p * levels)
    mu_total = mu_cum[-1]
    with np.errstate(divide="ignore", invalid="ignore"):
        mu0 = np.where(w0 > 0, mu_cum / w0, 0.0)
        mu1 = np.where(w1 > 0, (mu_total - mu_cum) / w1, 0.0)
    between = w0 * w1 * (mu0 - mu1) ** 2
    between[~np.isfinite(between)] = 0.0
    t = int(np.argmax(between))
    return t, between

def segment_threshold(gray, method="otsu", objects_dark=True, manual_value=128,
    block_size=51, C=8, offset=0):
    """Threshold based segmentation.

    method = "otsu" : global threshold from the from scratch Otsu routine
    method = "global" : fixed threshold supplied by the caller
    method = "adaptive" : locally adaptive Gaussian threshold, which handles a
    lighting gradient without an explicit flat field step
    """
    if method == "otsu":
        t, _ = otsu_threshold_from_scratch(gray)
        t = int(np.clip(t + offset, 0, 255))
        mask = gray <= t if objects_dark else gray >= t
        return _as_mask(mask), {"threshold": t}
    if method == "global":
        t = int(manual_value)
        mask = gray <= t if objects_dark else gray >= t
        return _as_mask(mask), {"threshold": t}
    if method == "adaptive":
        bs = block_size if block_size % 2 == 1 else block_size + 1
        ttype = cv2.THRESH_BINARY_INV if objects_dark else cv2.THRESH_BINARY
        mask = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            ttype, bs, C)
        return mask, {"block_size": bs, "C": C}
    raise ValueError("Unknown threshold method: %s" % method)

# =====
# 2. Region growing
# =====
def _neighbour_offsets(connectivity):
    if connectivity == 4:
        return ((-1, 0), (1, 0), (0, -1), (0, 1))
    return ((-1, -1), (-1, 0), (-1, 1), (0, -1),
        (0, 1), (1, -1), (1, 0), (1, 1))

def pick_seeds(gray, strategy="distance", n_seeds=40, objects_dark=True,
    coarse_mask=None, min_distance=12):
    """Choose the seed points for region growing.

    distance : threshold the image coarsely, take the distance transform of the
    object mask and place one seed at the deepest point of every
    connected component. This is an informed seeding strategy and it
    places one seed near the centre of each object.

```

```

grid      : place seeds on a regular lattice and keep only those that fall
             on the object side of a coarse threshold. This is an uninformed
             strategy and it is used in the report to show how sensitive the
             method is to seed placement.
random     : sample seeds uniformly at random from the object side of a
             coarse threshold.
"""
if coarse_mask is None:
    coarse_mask, _ = segment_threshold(gray, "otsu", objects_dark)
    coarse_mask = postprocess(coarse_mask, 3, 5, 40)
    binary = (coarse_mask > 0).astype(np.uint8)

if strategy == "distance":
    n, labels, _, _ = cv2.connectedComponentsWithStats(binary, connectivity=8)
    dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
    seeds = []
    for i in range(1, n):
        comp = labels == i
        if comp.sum() < 20:
            continue
        d = np.where(comp, dist, 0)
        y, x = np.unravel_index(int(np.argmax(d)), d.shape)
        seeds.append((int(y), int(x)))
    return seeds

if strategy == "grid":
    h, w = gray.shape[:2]
    step = max(4, int(np.sqrt(h * w / float(max(1, n_seeds)))))
    seeds = []
    for y in range(step // 2, h, step):
        for x in range(step // 2, w, step):
            if binary[y, x]:
                seeds.append((y, x))
    return seeds

if strategy == "random":
    rng = np.random.default_rng(RANDOM_SEED)
    ys, xs = np.nonzero(binary)
    if len(ys) == 0:
        return []
    idx = rng.choice(len(ys), size=min(n_seeds, len(ys)), replace=False)
    return [(int(ys[i]), int(xs[i])) for i in idx]

raise ValueError("Unknown seed strategy: %s" % strategy)

def region_growing(gray, seeds, tolerance=18, connectivity=8,
                  criterion="running_mean", max_pixels_per_region=None):
    """Grow regions from the given seed points.

    The similarity predicate is the absolute difference between the intensity
    of the candidate pixel and a reference intensity of the region. Two
    predicates are supported:

    running_mean : the reference is the mean intensity of the pixels accepted
                   so far, which lets a region follow a slow intensity drift
    seed_value    : the reference is fixed at the intensity of the seed, which
                   is stricter and stops the region leaking through a shading
                   gradient

    The queue is a plain first in first out list, so the traversal is a breadth
    first flood of the similarity region. Every pixel is visited at most once,
    which makes the routine linear in the number of pixels.
    """
    h, w = gray.shape[:2]
    img = gray.astype(np.float32)
    visited = np.zeros((h, w), dtype=bool)
    labels = np.zeros((h, w), dtype=np.int32)
    offsets = _neighbour_offsets(connectivity)

    from collections import deque

    for label_id, (sy, sx) in enumerate(seeds, start=1):
        if not (0 <= sy < h and 0 <= sx < w) or visited[sy, sx]:
            continue
        seed_value = float(img[sy, sx])
        total = seed_value
        count = 1
        visited[sy, sx] = True
        labels[sy, sx] = label_id
        queue = deque()
        queue.append((sy, sx))
        while queue:
            y, x = queue.popleft()
            reference = (total / count) if criterion == "running_mean" else seed_value
            for dy, dx in offsets:
                ny, nx = y + dy, x + dx
                if ny < 0 or ny >= h or nx < 0 or nx >= w:
                    continue
                if visited[ny, nx]:
                    continue
                if abs(float(img[ny, nx]) - reference) <= tolerance:
                    visited[ny, nx] = True
                    labels[ny, nx] = label_id
                    total += float(img[ny, nx])
                    count += 1
                    queue.append((ny, nx))
                    if max_pixels_per_region and count > max_pixels_per_region:
                        queue.clear()
                        break
        return labels

def segment_region_growing(gray, objects_dark=True, tolerance=18,
                          seed_strategy="distance", n_seeds=40,
                          connectivity=8, criterion="running_mean"):

```

```

        """Region growing wrapped as a binary segmentation."""
        seeds = pick_seeds(gray, seed_strategy, n_seeds, objects_dark)
        labels = region_growing(gray, seeds, tolerance, connectivity, criterion)
        mask = _as_mask(labels > 0)
        return mask, {"n_seeds": len(seeds), "tolerance": tolerance,
                     "seed_strategy": seed_strategy, "criterion": criterion,
                     "seeds": seeds}

# =====
# 3. K means clustering
# =====
def _kmeans_plus_plus_init(X, k, rng):
    """k means plus plus seeding: spread the initial centres apart."""
    n = X.shape[0]
    centres = np.empty((k, X.shape[1]), dtype=np.float32)
    centres[0] = X[rng.integers(n)]
    closest = np.sum((X - centres[0]) ** 2, axis=1)
    for i in range(1, k):
        total = closest.sum()
        if total <= 0:
            centres[i] = X[rng.integers(n)]
        else:
            probs = closest / total
            centres[i] = X[rng.choice(n, p=probs)]
            d = np.sum((X - centres[i]) ** 2, axis=1)
            closest = np.minimum(closest, d)
    return centres

def kmeans_from_scratch(X, k, max_iter=40, tol=1e-4, seed=RANDOM_SEED):
    """Lloyd's algorithm.
    Assignment step: every sample is attached to the nearest centre.
    Update step : every centre moves to the mean of the samples attached to
        it. The loop stops when no centre moves further than tol.
    """
    rng = np.random.default_rng(seed)
    X = X.astype(np.float32)
    centres = _kmeans_plus_plus_init(X, k, rng)
    labels = np.zeros(X.shape[0], dtype=np.int32)
    for _ in range(max_iter):
        # Assignment step, computed in blocks to keep memory bounded.
        block = 200000
        for start in range(0, X.shape[0], block):
            chunk = X[start:start + block]
            d = ((chunk[:, None, :] - centres[None, :, :]) ** 2).sum(axis=2)
            labels[start:start + block] = np.argmin(d, axis=1)
        # Update step.
        shift = 0.0
        for j in range(k):
            members = X[labels == j]
            if len(members) == 0:
                new_centre = X[rng.integers(X.shape[0])]
            else:
                new_centre = members.mean(axis=0)
                shift = max(shift, float(np.linalg.norm(new_centre - centres[j])))
            centres[j] = new_centre
        if shift < tol:
            break
    return labels, centres

def build_features(img_bgr, gray_prepared, feature_space="intensity",
                  spatial_weight=0.15):
    """Assemble the per pixel feature vectors used by the clustering."""
    h, w = gray_prepared.shape[:2]
    if feature_space == "intensity":
        return gray_prepared.reshape(-1, 1).astype(np.float32)
    if feature_space == "lab":
        lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
        return lab.reshape(-1, 3).astype(np.float32)
    if feature_space == "lab_xy":
        lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB).astype(np.float32)
        ys, xs = np.mgrid[0:h, 0:w]
        sx = (xs.astype(np.float32) / w) * 255.0 * spatial_weight
        sy = (ys.astype(np.float32) / h) * 255.0 * spatial_weight
        return np.concatenate(
            [lab.reshape(-1, 3), sx.reshape(-1, 1), sy.reshape(-1, 1)], axis=1)
    raise ValueError("Unknown feature space: %s" % feature_space)

def segment_kmeans(img_bgr, gray_prepared, k=2, feature_space="intensity",
                  objects_dark=True, border=10, use_opencv=False):
    """K means segmentation with an automatic foreground cluster decision.

    After clustering, the clusters that dominate the border ring of the image
    are declared background, since the border of every image in this dataset is
    background. Any remaining cluster is foreground. When every cluster touches
    the border, the single cluster whose mean intensity is furthest from the
    border mean in the expected direction is taken as the object cluster.
    """
    h, w = gray_prepared.shape[:2]
    X = build_features(img_bgr, gray_prepared, feature_space)
    if use_opencv:
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 40, 0.2)
        _, lab, centres = cv2.kmeans(X, k, None, criteria, 3,
                                     cv2.KMEANS_PP_CENTERS)
        labels = lab.ravel()
    else:
        labels, centres = kmeans_from_scratch(X, k)

    label_img = labels.reshape(h, w)

    # Border ring statistics decide which clusters are background.
    ring = np.zeros((h, w), dtype=bool)
    ring[:border, :] = True

```

```

ring[-border:, :] = True
ring[:, :border] = True
ring[:, -border:] = True
ring_labels = label_img[ring]

mean_intensity = np.array(
    [float(gray_prepared[label_img == j].mean()) if np.any(label_img == j)
     else 0.0 for j in range(k)])

background = set()
for j in range(k):
    share_of_ring = float((ring_labels == j).sum()) / max(1, ring_labels.size)
    if share_of_ring > 0.25:
        background.add(j)
if len(background) == 0 or len(background) == k:
    # Fall back to the intensity ordering of the cluster means.
    order = np.argsort(mean_intensity)
    object_cluster = order[0] if objects_dark else order[-1]
    background = set(range(k)) - {int(object_cluster)}

mask = np.isin(label_img, list(set(range(k)) - background))
return _as_mask(mask), {"k": k, "feature_space": feature_space,
                       "cluster_means": mean_intensity.tolist(),
                       "background_clusters": sorted(background),
                       "label_image": label_img}

# =====
# Optional improvement stage: splitting touching objects
# =====
def split_touching(mask, dist_ratio=0.45):
    """Separate touching objects with a distance transform and watershed.

    The distance transform of the object mask peaks at the centre of every
    object. Thresholding it produces one marker per object even when the
    objects share a boundary, and the watershed transform then grows those
    markers back to the object boundaries.
    """
    binary = (mask > 0).astype(np.uint8)
    dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
    if dist.max() <= 0:
        return mask.copy(), 0

    # The marker threshold is applied inside each connected component
    # separately. A single global threshold would erase every object that is
    # smaller than the largest one in the image, which silently loses objects
    # when the scene contains objects of different sizes.
    n_comp, comp_labels, _, _ = cv2.connectedComponentsWithStats(binary,
                                                                    connectivity=8)

    sure_fg = np.zeros_like(binary)
    for i in range(1, n_comp):
        comp = comp_labels == i
        local_max = float(dist[comp].max())
        if local_max <= 0:
            continue
        sure_fg[np.logical_and(comp, dist >= dist_ratio * local_max)] = 255
    n_markers, markers = cv2.connectedComponents(sure_fg)
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    sure_bg = cv2.dilate(binary, se, iterations=3)
    unknown = cv2.subtract(sure_bg, sure_fg // 255)
    markers = markers + 1
    markers[unknown > 0] = 0
    colour = cv2.cvtColor(binary * 255, cv2.COLOR_GRAY2BGR)
    markers = cv2.watershed(colour, markers)
    out = np.zeros_like(mask)
    out[markers > 1] = 255
    # The watershed lines are one pixel wide, which is not enough to break an
    # eight connected diagonal link between two neighbouring objects, so the
    # lines are widened by one pixel before they are cut out of the mask.
    line = np.zeros_like(mask)
    line[markers == -1] = 255
    line = cv2.dilate(line, se, iterations=1)
    out[line > 0] = 0
    out = cv2.bitwise_and(out, mask)
    return out, max(0, n_markers - 1)

# =====
# Timed wrapper used by the experiment driver
# =====
def run_method(name, prep, params):
    """Run one segmentation method and report the wall clock time.

    prep is the dictionary returned by preprocessing.prepare.
    """
    gray = prep["prepared"]
    dark = prep["objects_are_dark"]
    t0 = time.perf_counter()

    if name == "threshold":
        mask, info = segment_threshold(
            gray, params.get("mode", "otsu"), dark,
            manual_value=params.get("manual_value", 128),
            block_size=params.get("block_size", 51),
            C=params.get("C", 8),
            offset=params.get("offset", 0))
    elif name == "region_growing":
        mask, info = segment_region_growing(
            gray, dark,
            tolerance=params.get("tolerance", 18),
            seed_strategy=params.get("seed_strategy", "distance"),
            n_seeds=params.get("n_seeds", 40),
            connectivity=params.get("connectivity", 8),
            criterion=params.get("criterion", "running_mean"))
    elif name == "kmeans":
        mask, info = segment_kmeans(

```

```

        prep["bgr"], gray,
        k=params.get("k", 2),
        feature_space=params.get("feature_space", "intensity"),
        objects_dark=dark,
        use_opencv=params.get("use_opencv", False))
    else:
        raise ValueError("Unknown method: %s" % name)

    mask = postprocess(mask,
                       open_k=params.get("open_k", 3),
                       close_k=params.get("close_k", 5),
                       min_area=params.get("min_area", 80),
                       fill_holes=params.get("fill_holes", True))
    elapsed = time.perf_counter() - t0
    info["time_s"] = elapsed
    return mask, info

```

src/measurement.py

```

"""
Object analysis stage: boundary identification, object extraction and the
measurements reported by the system.

For every segmented region the system reports the object count, area,
perimeter, centroid, bounding box and object size. When a scale is available
the pixel measurements are converted into millimetres.
"""

import cv2
import numpy as np

def boundaries(mask, thickness=1):
    """Return the boundary pixels of a binary mask.

    The morphological gradient, that is the dilation minus the erosion, gives a
    one pixel wide outline of every region and is the classical way to turn a
    region representation into a boundary representation.
    """
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    binary = (mask > 0).astype(np.uint8) * 255
    grad = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, se)
    if thickness > 1:
        grad = cv2.dilate(grad, se, iterations=thickness - 1)
    return grad

def canny_edges(gray, low=60, high=160, blur=5):
    """Canny edges of the intensity image, used for the visual comparison
    between an edge based and a region based description of the objects."""
    smooth = cv2.GaussianBlur(gray, (blur, blur), 0)
    return cv2.Canny(smooth, low, high)

def measure_objects(mask, px_per_mm=None, drop_border_objects=False,
                    min_area=1):
    """Measure every connected component of a binary mask.

    Returned per object:
    id, area_px, perimeter_px, centroid, bounding box, width, height,
    equivalent diameter, major and minor axis of the fitted ellipse,
    orientation, circularity, aspect ratio, extent, solidity and, when a
    scale is supplied, the area in square millimetres and the lengths in
    millimetres.

    Circularity is  $4 * \pi * \text{area} / \text{perimeter}^2$ , which is 1 for a perfect disc
    and falls towards 0 for elongated or ragged shapes.
    """
    binary = (mask > 0).astype(np.uint8)
    n, labels, stats, centroids = cv2.connectedComponentsWithStats(
        binary, connectivity=8)
    h, w = binary.shape[:2]
    rows = []
    kept = 0
    for i in range(1, n):
        area = int(stats[i, cv2.CC_STAT_AREA])
        if area < min_area:
            continue
        x = int(stats[i, cv2.CC_STAT_LEFT])
        y = int(stats[i, cv2.CC_STAT_TOP])
        bw = int(stats[i, cv2.CC_STAT_WIDTH])
        bh = int(stats[i, cv2.CC_STAT_HEIGHT])
        touches_border = (x == 0 or y == 0 or x + bw == w or y + bh == h)
        if drop_border_objects and touches_border:
            continue

        comp = (labels == i).astype(np.uint8)
        contours, _ = cv2.findContours(comp, cv2.RETR_EXTERNAL,
                                       cv2.CHAIN_APPROX_NONE)

        if not contours:
            continue
        contour = max(contours, key=cv2.contourArea)
        perimeter = float(cv2.arcLength(contour, True))
        circularity = (4.0 * np.pi * area / (perimeter ** 2)) if perimeter > 0 else 0.0

        if len(contour) >= 5:
            (_, _), (ax1, ax2), angle = cv2.fitEllipse(contour)
            major, minor = max(ax1, ax2), min(ax1, ax2)
        else:
            major, minor, angle = float(max(bw, bh)), float(min(bw, bh)), 0.0

        hull = cv2.convexHull(contour)
        hull_area = float(cv2.contourArea(hull))
        solidity = area / hull_area if hull_area > 0 else 0.0
        extent = area / float(bw * bh) if bw * bh > 0 else 0.0

```

```

eq_diameter = float(np.sqrt(4.0 * area / np.pi))

kept += 1
row = {
    "object_id": kept,
    "area_px": area,
    "perimeter_px": round(perimeter, 2),
    "centroid_x": round(float(centroids[i][0]), 2),
    "centroid_y": round(float(centroids[i][1]), 2),
    "bbox_x": x, "bbox_y": y, "bbox_w": bw, "bbox_h": bh,
    "equivalent_diameter_px": round(eq_diameter, 2),
    "major_axis_px": round(float(major), 2),
    "minor_axis_px": round(float(minor), 2),
    "orientation_deg": round(float(angle), 1),
    "aspect_ratio": round(float(major / minor), 3) if minor > 0 else 0.0,
    "circularity": round(float(circularity), 3),
    "solidity": round(float(solidity), 3),
    "extent": round(float(extent), 3),
    "touches_border": int(touches_border),
}
if px_per_mm:
    row["area_mm2"] = round(area / (px_per_mm ** 2), 2)
    row["perimeter_mm"] = round(perimeter / px_per_mm, 2)
    row["major_axis_mm"] = round(float(major) / px_per_mm, 2)
    row["minor_axis_mm"] = round(float(minor) / px_per_mm, 2)
    row["equivalent_diameter_mm"] = round(eq_diameter / px_per_mm, 2)
rows.append(row)
return rows

def summarise(rows, px_per_mm=None):
    """Aggregate statistics over all measured objects of one image."""
    if not rows:
        return {"count": 0}
    areas = np.array([r["area_px"] for r in rows], dtype=np.float64)
    majors = np.array([r["major_axis_px"] for r in rows], dtype=np.float64)
    minors = np.array([r["minor_axis_px"] for r in rows], dtype=np.float64)
    circs = np.array([r["circularity"] for r in rows], dtype=np.float64)
    out = {
        "count": len(rows),
        "area_px_mean": round(float(areas.mean()), 1),
        "area_px_std": round(float(areas.std()), 1),
        "area_px_min": round(float(areas.min()), 1),
        "area_px_max": round(float(areas.max()), 1),
        "major_axis_px_mean": round(float(majors.mean()), 2),
        "minor_axis_px_mean": round(float(minors.mean()), 2),
        "circularity_mean": round(float(circs.mean()), 3),
    }
    if px_per_mm:
        out["area_mm2_mean"] = round(float(areas.mean()) / (px_per_mm ** 2), 2)
        out["major_axis_mm_mean"] = round(float(majors.mean()) / px_per_mm, 2)
        out["minor_axis_mm_mean"] = round(float(minors.mean()) / px_per_mm, 2)
    return out

def size_classes(rows, n_classes=3):
    """Group objects into small, medium and large by equivalent diameter.

    This supports the object size requirement of the question and it is what a
    grading application would report for a batch of seeds.
    """
    if not rows:
        return {}
    d = np.array([r["equivalent_diameter_px"] for r in rows])
    edges = np.quantile(d, np.linspace(0, 1, n_classes + 1))
    names = ["small", "medium", "large"][:n_classes]
    counts = {}
    for i, name in enumerate(names):
        lo, hi = edges[i], edges[i + 1]
        if i == n_classes - 1:
            counts[name] = int(((d >= lo) & (d <= hi)).sum())
        else:
            counts[name] = int(((d >= lo) & (d < hi)).sum())
    return counts

```

src/evaluation.py

```

"""
Quantitative evaluation against manually verified ground truth masks.

Pixel level measures
    Intersection over Union, precision, recall, F1 score and pixel accuracy.

Object level measures
    Predicted count, ground truth count, count error, and a greedy one to one
    matching of predicted regions to ground truth regions at an IoU threshold,
    from which object level precision, recall and F1 are computed.
"""

import cv2
import numpy as np

# -----
# Pixel level
# -----
def pixel_scores(pred_mask, gt_mask):
    pred = pred_mask > 0
    gt = gt_mask > 0
    tp = int(np.logical_and(pred, gt).sum())
    fp = int(np.logical_and(pred, ~gt).sum())
    fn = int(np.logical_and(~pred, gt).sum())
    tn = int(np.logical_and(~pred, ~gt).sum())
    union = tp + fp + fn
    iou = tp / union if union else 1.0

```

```

precision = tp / (tp + fp) if (tp + fp) else 0.0
recall = tp / (tp + fn) if (tp + fn) else 0.0
f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0
dice = (2 * tp / (2 * tp + fp + fn)) if (2 * tp + fp + fn) else 1.0
accuracy = (tp + tn) / float(pred.size)
return {
    "iou": round(iou, 4),
    "precision": round(precision, 4),
    "recall": round(recall, 4),
    "f1": round(f1, 4),
    "dice": round(dice, 4),
    "pixel_accuracy": round(accuracy, 4),
    "tp": tp, "fp": fp, "fn": fn,
}

# -----
# Object level
# -----
def _components(mask, min_area=30):
    n, labels, stats, _ = cv2.connectedComponentsWithStats(
        (mask > 0).astype(np.uint8), connectivity=8)
    keep = []
    for i in range(1, n):
        if stats[i, cv2.CC_STAT_AREA] >= min_area:
            keep.append(labels == i)
    return keep

def object_scores(pred_mask, gt_mask, iou_threshold=0.5, min_area=30):
    """Greedy one to one matching of predicted regions to ground truth regions."""
    preds = _components(pred_mask, min_area)
    gts = _components(gt_mask, min_area)
    if not preds or not gts:
        return {
            "pred_count": len(preds), "gt_count": len(gts),
            "count_error": len(preds) - len(gts),
            "obj_precision": 0.0, "obj_recall": 0.0, "obj_f1": 0.0,
            "mean_matched_iou": 0.0,
        }

    pairs = []
    for i, p in enumerate(preds):
        for j, g in enumerate(gts):
            inter = np.logical_and(p, g).sum()
            if inter == 0:
                continue
            union = np.logical_or(p, g).sum()
            pairs.append((inter / union, i, j))
    pairs.sort(reverse=True)

    used_p, used_g, matched = set(), set(), []
    for iou, i, j in pairs:
        if iou < iou_threshold or i in used_p or j in used_g:
            continue
        used_p.add(i)
        used_g.add(j)
        matched.append(iou)

    tp = len(matched)
    precision = tp / len(preds)
    recall = tp / len(gts)
    f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0
    return {
        "pred_count": len(preds), "gt_count": len(gts),
        "count_error": len(preds) - len(gts),
        "obj_precision": round(precision, 4),
        "obj_recall": round(recall, 4),
        "obj_f1": round(f1, 4),
        "mean_matched_iou": round(float(np.mean(matched)) if matched else 0.0, 4),
    }

def evaluate(pred_mask, gt_mask, iou_threshold=0.5, min_area=30):
    out = pixel_scores(pred_mask, gt_mask)
    out.update(object_scores(pred_mask, gt_mask, iou_threshold, min_area))
    return out

# -----
# Measurement accuracy
# -----
def measurement_errors(pred_mask, gt_mask, px_per_mm=None, iou_threshold=0.5,
                       min_area=30):
    """Compare the measurements of matched objects with the ground truth.

    Predicted regions are matched to ground truth regions with the same greedy
    one to one rule as object_scores, and for every matched pair the relative
    error of the area and of the major axis length is computed. This answers
    the question that matters for a measurement system: given that an object
    was found, how accurate is the number reported for it.
    """
    import measurement as _M

    pred_rows = _M.measure_objects(pred_mask, px_per_mm, min_area=min_area)
    gt_rows = _M.measure_objects(gt_mask, px_per_mm, min_area=min_area)
    preds = _components(pred_mask, min_area)
    gts = _components(gt_mask, min_area)

    pairs = []
    for i, p in enumerate(preds):
        for j, g in enumerate(gts):
            inter = np.logical_and(p, g).sum()
            if inter == 0:
                continue
            pairs.append((inter / np.logical_or(p, g).sum(), i, j))

```

```

pairs.sort(reverse=True)

used_p, used_g, records = set(), set(), []
for iou, i, j in pairs:
    if iou < iou_threshold or i in used_p or j in used_g:
        continue
    used_p.add(i)
    used_g.add(j)
    pr, gr = pred_rows[i], gt_rows[j]
    records.append({
        "iou": round(float(iou), 4),
        "area_pred": pr["area_px"], "area_gt": gr["area_px"],
        "area_error_pct": round(100.0 * (pr["area_px"] - gr["area_px"])
                                / max(1, gr["area_px"]), 2),
        "major_pred": pr["major_axis_px"], "major_gt": gr["major_axis_px"],
        "major_error_pct": round(100.0 * (pr["major_axis_px"] - gr["major_axis_px"])
                                / max(1e-6, gr["major_axis_px"]), 2),
    })
if not records:
    return {"matched": 0}
area_err = np.array([abs(r["area_error_pct"]) for r in records])
major_err = np.array([abs(r["major_error_pct"]) for r in records])
return {
    "matched": len(records),
    "mean_abs_area_error_pct": round(float(area_err.mean()), 2),
    "max_abs_area_error_pct": round(float(area_err.max()), 2),
    "mean_abs_major_error_pct": round(float(major_err.mean()), 2),
    "max_abs_major_error_pct": round(float(major_err.max()), 2),
    "records": records,
}

```

src/calibration.py

```

"""
Spatial calibration from the printed ruler in the cowpea photograph.

One image of the dataset was photographed next to a millimetre ruler. The tick
marks are found by projecting the ruler band onto the horizontal axis, and the
median distance between neighbouring ticks gives the number of pixels per
millimetre. Every pixel measurement of that image can then be reported in
millimetres, which is what an agricultural grading application needs.
"""

import cv2
import numpy as np

# Vertical band of the working resolution image that contains the ruler, and
# the physical distance between two neighbouring tick marks.
RULER_BAND = (500, 570)
MM_PER_TICK = 1.0

def tick_positions(img_bgr, band=RULER_BAND, ink_threshold=128, min_ink=3):
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    strip = gray[band[0]:band[1], :]
    ink_per_column = (strip < ink_threshold).sum(axis=0)
    is_tick = ink_per_column > min_ink
    centres, start = [], None
    for i, v in enumerate(is_tick):
        if v and start is None:
            start = i
        if not v and start is not None:
            centres.append((start + i - 1) / 2.0)
            start = None
    if start is not None:
        centres.append((start + len(is_tick) - 1) / 2.0)
    return centres

def pixels_per_mm(img_bgr, band=RULER_BAND, mm_per_tick=MM_PER_TICK):
    centres = tick_positions(img_bgr, band)
    if len(centres) < 5:
        raise RuntimeError("Ruler not found, only %d ticks detected" % len(centres))
    spacing = np.diff(np.array(centres))
    # The median is used so that the wider marks at the five millimetre
    # positions cannot bias the estimate.
    return float(np.median(spacing) / mm_per_tick), len(centres)

if __name__ == "__main__":
    from dataset import load_working
    work, rec, _ = load_working("beans_cowpea_scale_05")
    scale, n = pixels_per_mm(work)
    print("ticks detected : %d" % n)
    print("pixels per mm : %.2f" % scale)
    print("field of view : %.1f mm wide" % (work.shape[1] / scale))

```

src/ground_truth.py

```

"""
Ground truth mask creation.

The evaluation needs manual ground truth masks on a representative subset of the
test images. Painting several hundred object outlines pixel by pixel by hand is
prohibitively slow, so the masks were produced with the assisted procedure that
is standard practice in annotation tools:

1. A carefully tuned, per image classical recipe produces a first draft
   mask. The recipe for the draft is allowed to use information that the
   system under test is not allowed to use, for example a hand picked
   threshold or a hand picked structuring element for that one photograph.
2. Every draft was inspected as an overlay on the original photograph.
3. Errors that remained were corrected by hand through the MANUAL_EDITS
   table below: individual false regions are deleted by clicking a point

```

- inside them, touching objects are separated by drawing a cut line, and missed objects are painted in as polygons.
- The corrected mask was inspected again and only then written to data/ground_truth.

The draft recipe is therefore an annotation aid and not a competing method. The segmentation methods that are evaluated in run_experiments.py never see these parameters, they use one common parameter set for the whole dataset.

The dense pinto bean photograph is deliberately left without a ground truth mask. Its objects overlap so heavily that a pixel accurate manual mask is not achievable, so that image is reported as a qualitative failure case only.

```
import cv2
import numpy as np

from config import GT_DIR
from dataset import load_working
from preprocessing import (correct_illumination, denoise, objects_are_dark,
                           to_gray)
from segmentation import (fill_object_holes, otsu_threshold_from_scratch,
                           postprocess, remove_small_components, split_touching)

# -----
# Per image draft recipes
#
# channel      : which channel the draft threshold works on
# illum        : apply the flat field correction before thresholding
# blur         : kernel of the smoothing filter applied first
# offset       : shift added to the Otsu threshold for this photograph
# min_area     : smallest region that is accepted as an object
# close_k, open_k : structuring element sizes of the clean up
# -----
RECIPES = {
    # Pixel accurate annotation. This photograph carries several touching
    # seeds, so the annotation is a foreground mask and the image is not used
    # for object level scoring.
    "seeds_sunflower_01": dict(kind="otsu", channel="gray", illum=True, blur=5,
                               offset=6, min_area=400, open_k=5, close_k=9,
                               instance_gt=False),
    # Very low contrast. The draft threshold failed on this photograph, so the
    # three objects were outlined by hand, see MANUAL_POLYGONS below.
    "seeds_pale_04": dict(kind="manual_polygon", min_area=2000,
                          instance_gt=True),
    "beans_cowpea_scale_05": dict(kind="otsu", channel="lab_a", illum=False,
                                   blur=5, offset=0, min_area=1500, open_k=5,
                                   close_k=11, invert=True, instance_gt=True),
    "beans_coffee_06": dict(kind="otsu", channel="gray", illum=False, blur=5,
                              offset=-14, min_area=2000, open_k=7, close_k=15,
                              instance_gt=True),
    "coins_greek_07": dict(kind="canny", channel="gray", blur=5, canny=(20, 70),
                             close_k=9, close_iter=2, min_area=400, open_k=5,
                             watershed=0.45, instance_gt=True),
}

# -----
# Hand traced outlines for the low contrast photograph. The vertices were read
# off a coordinate grid printed over the working resolution image. The traced
# polygons are then tightened onto the true object boundary with a colour model
# refinement, which is the same assisted outlining that annotation tools use,
# and the result was checked against the photograph.
# -----
MANUAL_POLYGONS = {
    "seeds_pale_04": [
        [(148, 130), (178, 145), (205, 172), (222, 205), (235, 245),
         (246, 285), (253, 320), (258, 352), (262, 378), (250, 392),
         (228, 393), (200, 383), (172, 370), (148, 357), (120, 340),
         (100, 318), (86, 292), (74, 262), (62, 232), (52, 208), (50, 196),
         (58, 182), (76, 168), (100, 152), (124, 138)],
        [(400, 35), (440, 42), (480, 62), (515, 90), (545, 125), (565, 165),
         (575, 205), (572, 245), (560, 285), (540, 320), (515, 350),
         (487, 372), (455, 385), (430, 390), (405, 383), (390, 368),
         (370, 345), (350, 318), (333, 285), (320, 250), (312, 212),
         (315, 175), (325, 140), (345, 105), (368, 70), (385, 48)],
        [(748, 128), (775, 140), (800, 162), (818, 190), (832, 222),
         (842, 258), (845, 292), (838, 325), (822, 352), (800, 372),
         (778, 385), (755, 392), (732, 388), (712, 372), (695, 350),
         (680, 322), (668, 290), (660, 255), (656, 220), (662, 188),
         (678, 162), (700, 145), (724, 133)],
    ],
}

# -----
# Manual corrections applied on top of the draft masks.
#
# ("remove", x, y)      delete the region that contains this point
# ("cut", x1, y1, x2, y2, w) draw a background line, separating objects
# ("add", [(x, y), ...]) paint a missed object as a filled polygon
# ("add_circle", x, y, r) paint a missed object as a filled disc
# -----
MANUAL_EDITS = {
    # The three beans on the left of the coffee photograph are in contact, so
    # two cut lines were drawn along the visible crevices between them.
    # The three beans on the left of the coffee photograph are in contact and
    # so are the centre and the right bean. One point was clicked inside every
    # bean and a marker driven watershed then cut the foreground along the
    # crevices between them.
    "beans_coffee_06": [
        ("split_clicks", [(400, 135), (610, 290), (180, 295), (215, 465),
                           (430, 400)]),
    ],
    # Two parts of the seed outlines that the colour refinement clipped, the
    # thin tail of the left seed and the bright out of focus lobe of the right
    # seed, were painted back in by hand.
    "seeds_pale_04": [

```

```

        ("add", [(238, 338), (258, 356), (272, 388), (258, 400), (236, 392),
                  (226, 366)]),
        ("add", [(758, 130), (790, 133), (814, 156), (830, 192), (840, 228),
                  (812, 230), (788, 198), (770, 166)]),
    ],
}

def _channel(img_bgr, name):
    if name == "gray":
        return to_gray(img_bgr)
    if name == "lab_a":
        return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)[: , : , 1]
    if name == "lab_b":
        return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)[: , : , 2]
    if name == "hsv_s":
        return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)[: , : , 1]
    raise ValueError(name)

def draft_mask(image_id):
    """First draft of the annotation for one photograph."""
    work, rec, _ = load_working(image_id)
    cfg = RECIPES[image_id]
    if cfg.get("kind") == "manual_polygon":
        return work, _manual_polygon_mask(work, MANUAL_POLYGONS[image_id],
                                           cfg.get("min_area", 1000))
    chan = _channel(work, cfg.get("channel", "gray"))
    dark = objects_are_dark(to_gray(work))
    if cfg.get("invert"):
        dark = not dark
    smooth = denoise(chan, "gaussian", cfg.get("blur", 5))

    if cfg.get("kind", "otsu") == "canny":
        # An edge based draft. The coins photograph has a dark background and
        # a strong lighting gradient, so its object boundaries are far more
        # reliable than its absolute grey levels.
        lo, hi = cfg.get("canny", (20, 70))
        edges = cv2.Canny(smooth, lo, hi)
        se = cv2.getStructuringElement(
            cv2.MORPH_ELLIPSE, (cfg.get("close_k", 9), cfg.get("close_k", 9)))
        mask = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, se,
                                iterations=cfg.get("close_iter", 2))
        mask = postprocess(mask, 3, 7, cfg.get("min_area", 400))
        se2 = cv2.getStructuringElement(
            cv2.MORPH_ELLIPSE, (cfg.get("open_k", 5), cfg.get("open_k", 5)))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se2)
    else:
        if cfg.get("illum"):
            smooth = correct_illumination(smooth, objects_are_dark=dark)
        t, _ = otsu_threshold_from_scratch(smooth)
        t = int(np.clip(t + cfg.get("offset", 0), 0, 255))
        mask = ((smooth <= t) if dark else (smooth >= t)).astype(np.uint8) * 255
        mask = postprocess(mask, cfg.get("open_k", 5), cfg.get("close_k", 9),
                           cfg.get("min_area", 400), fill_holes=True)

    if cfg.get("watershed"):
        # Instance separation of touching objects, so that the annotation
        # carries one region per physical object.
        mask, _ = split_touching(mask, cfg["watershed"])
        mask = remove_small_components(mask, cfg.get("min_area", 400))
    return work, mask

def _manual_polygon_mask(work, polygons, min_area, margin=31, iterations=6):
    """Turn hand traced polygons into a pixel accurate mask.

    Each polygon is used to seed a colour model refinement. Pixels well inside
    the polygon are marked as certain object, pixels well outside are marked as
    certain background, and the band in between is decided by the refinement.
    This is the assisted outlining that annotation tools provide, and the
    result was inspected against the photograph before it was accepted.
    """
    h, w = work.shape[:2]
    final = np.zeros((h, w), np.uint8)
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (margin, margin))
    for poly in polygons:
        pts = np.array(poly, np.int32)
        inner = np.zeros((h, w), np.uint8)
        cv2.fillPoly(inner, [pts], 255)
        sure_in = cv2.erode(inner, se)
        outer = cv2.dilate(inner, se)
        gc = np.full((h, w), cv2.GC_BGD, np.uint8)
        gc[outer > 0] = cv2.GC_PR_BGD
        gc[inner > 0] = cv2.GC_PR_FGD
        gc[sure_in > 0] = cv2.GC_FGD
        bgd = np.zeros((1, 65), np.float64)
        fgd = np.zeros((1, 65), np.float64)
        cv2.grabCut(work, gc, None, bgd, fgd, iterations, cv2.GC_INIT_WITH_MASK)
        res = np.where((gc == cv2.GC_FGD) | (gc == cv2.GC_PR_FGD), 255, 0)
        final[res > 0] = 255
    return postprocess(final, 5, 9, min_area)

def split_by_clicks(mask, points, marker_radius=6):
    """Separate touching objects using one clicked point per object.

    The clicks become the object markers, the region well outside the mask
    becomes the background marker, and the watershed transform then places the
    dividing line along the crevice between the objects. This is the same
    interaction that a marker based annotation tool offers.
    """
    binary = (mask > 0).astype(np.uint8)
    markers = np.zeros(binary.shape, np.int32)
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    outside = cv2.dilate(binary, se, iterations=6)

```

```

    markers[outside == 0] = 1
    for i, (x, y) in enumerate(points, start=2):
        cv2.circle(markers, (int(x), int(y)), marker_radius, i, -1)
    colour = cv2.cvtColor(binary * 255, cv2.COLOR_GRAY2BGR)
    markers = cv2.watershed(colour, markers)
    out = np.zeros_like(mask)
    out[markers > 1] = 255
    line = np.zeros_like(mask)
    line[markers == -1] = 255
    line = cv2.dilate(line, se, iterations=1)
    out[line > 0] = 0
    return cv2.bitwise_and(out, mask)

def apply_manual_edits(mask, edits):
    out = mask.copy()
    for edit in edits:
        kind = edit[0]
        if kind == "remove":
            _, x, y = edit
            n, labels, _, _ = cv2.connectedComponentsWithStats(
                (out > 0).astype(np.uint8), connectivity=8)
            lab = labels[int(y), int(x)]
            if lab > 0:
                out[labels == lab] = 0
        elif kind == "cut":
            _, x1, y1, x2, y2, w = edit
            cv2.line(out, (int(x1), int(y1)), (int(x2), int(y2)), 0, int(w))
        elif kind == "add":
            _, pts = edit
            cv2.fillPoly(out, [np.array(pts, dtype=np.int32)], 255)
        elif kind == "split_clicks":
            _, points = edit
            out = split_by_clicks(out, points)
        elif kind == "add_circle":
            _, x, y, r = edit
            cv2.circle(out, (int(x), int(y)), int(r), 255, -1)
        else:
            raise ValueError("Unknown manual edit: %s" % kind)
    return out

def build_ground_truth(image_id, write=True):
    work, mask = draft_mask(image_id)
    mask = apply_manual_edits(mask, MANUAL_EDITS.get(image_id, []))
    mask = remove_small_components(mask, RECIPES[image_id].get("min_area", 400))
    if write:
        cv2.imwrite(str(GT_DIR / (image_id + "_gt.png")), mask)
    return work, mask

def gt_path(image_id):
    """Ground truth path for an image, following a variant back to its base."""
    from config import image_record
    rec = image_record(image_id)
    base = rec.get("variant_of", image_id)
    return GT_DIR / (base + "_gt.png")

def has_ground_truth(image_id):
    return gt_path(image_id).exists()

def load_gt(image_id):
    p = gt_path(image_id)
    gt = cv2.imread(str(p), cv2.IMREAD_GRAYSCALE)
    if gt is None:
        raise FileNotFoundError("No ground truth for %s" % image_id)
    return gt

def overlay(work, mask, colour=(0, 0, 255)):
    out = work.copy()
    contours, _ = cv2.findContours((mask > 0).astype(np.uint8),
                                   cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    cv2.drawContours(out, contours, -1, colour, 2)
    n, labels, stats, cents = cv2.connectedComponentsWithStats(
        (mask > 0).astype(np.uint8), connectivity=8)
    for i in range(1, n):
        cx, cy = cents[i]
        cv2.putText(out, str(i), (int(cx) - 8, int(cy) + 6),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 200, 255), 2)
    return out

if __name__ == "__main__":
    import sys
    from config import FIG_DIR
    for image_id in RECIPES:
        work, mask = build_ground_truth(image_id)
        n, _, _, _ = cv2.connectedComponentsWithStats(
            (mask > 0).astype(np.uint8), connectivity=8)
        cv2.imwrite(str(FIG_DIR / ("gt_check_" + image_id + ".png")),
                    overlay(work, mask))
        print("%-24s objects=%3d coverage=%.3f" %
              (image_id, n - 1, float((mask > 0).mean())))

```

src/visualisation.py

```

"""
Visualisation helpers.

All figures written to results/figures are produced here, so that the report
generator only has to place finished images on the page.
"""

import cv2

```

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from config import FIG_DIR

plt.rcParams.update({
    "figure.dpi": 130,
    "savefig.dpi": 130,
    "font.size": 8,
    "axes.titlesize": 9,
    "axes.grid": False,
})

OBJECT_COLOUR = (0, 0, 255) # BGR, red outline
GT_COLOUR = (0, 200, 0) # BGR, green outline

def rgb(img_bgr):
    return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

def draw_boundaries(img_bgr, mask, colour=OBJECT_COLOUR, thickness=2):
    out = img_bgr.copy()
    contours, _ = cv2.findContours((mask > 0).astype(np.uint8),
                                   cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    cv2.drawContours(out, contours, -1, colour, thickness)
    return out

def draw_measurements(img_bgr, mask, rows, px_per_mm=None, show_bbox=True,
                      font_scale=0.42):
    """Draw the boundary, the bounding box, the centroid and a label with the
    main measurements of every detected object."""
    out = draw_boundaries(img_bgr, mask, OBJECT_COLOUR, 2)
    for r in rows:
        x, y, w, h = r["bbox_x"], r["bbox_y"], r["bbox_w"], r["bbox_h"]
        if show_bbox:
            cv2.rectangle(out, (x, y), (x + w, y + h), (255, 200, 0), 1)
            cx, cy = int(round(r["centroid_x"])), int(round(r["centroid_y"]))
            cv2.drawMarker(out, (cx, cy), (0, 255, 255), cv2.MARKER_CROSS, 9, 2)
        if px_per_mm:
            label = "%d: %.1fx%.1fmm" % (r["object_id"], r["major_axis_mm"],
                                         r["minor_axis_mm"])
        else:
            label = "%d: %dpx" % (r["object_id"], r["area_px"])
        cv2.putText(out, label, (x, max(12, y - 4)), cv2.FONT_HERSHEY_SIMPLEX,
                    font_scale, (0, 0, 0), 3, cv2.LINE_AA)
        cv2.putText(out, label, (x, max(12, y - 4)), cv2.FONT_HERSHEY_SIMPLEX,
                    font_scale, (255, 255, 255), 1, cv2.LINE_AA)
    return out

def error_map(pred, gt):
    """Colour coded comparison: green true positive, red false positive,
    blue false negative."""
    h, w = gt.shape[:2]
    out = np.zeros((h, w, 3), np.uint8)
    p, g = pred > 0, gt > 0
    out[np.logical_and(p, g)] = (0, 180, 0)
    out[np.logical_and(p, ~g)] = (0, 0, 220)
    out[np.logical_and(~p, g)] = (220, 120, 0)
    return out

def panel(images, titles, path, cols=None, figsize_scale=3.1, suptitle=None):
    """Write a grid of images as one figure."""
    n = len(images)
    cols = cols or min(4, n)
    rows = int(np.ceil(n / cols))
    fig, axes = plt.subplots(rows, cols,
                             figsize=(figsize_scale * cols, figsize_scale * rows * 0.85))
    axes = np.atleast_1d(axes).ravel()
    for ax in axes:
        ax.axis("off")
    for ax, img, title in zip(axes, images, titles):
        if img.ndim == 2:
            ax.imshow(img, cmap="gray", vmin=0, vmax=255)
        else:
            ax.imshow(rgb(img))
            ax.set_title(title)
            ax.axis("off")
    if suptitle:
        fig.suptitle(suptitle, fontsize=10)
    fig.tight_layout()
    fig.savefig(str(FIG_DIR / path), bbox_inches="tight")
    plt.close(fig)
    return FIG_DIR / path

def histogram_with_threshold(gray, threshold, path, title="Grey level histogram"):
    fig, ax = plt.subplots(figsize=(4.6, 2.6))
    ax.hist(gray.ravel(), bins=256, range=(0, 255), color="#4a6fa5")
    ax.axvline(threshold, color="crimson", linewidth=1.6,
               label="Otsu threshold = %d" % threshold)
    ax.set_xlabel("grey level")
    ax.set_ylabel("pixel count")
    ax.set_title(title)
    ax.legend(loc="upper right", fontsize=7)
    fig.tight_layout()
    fig.savefig(str(FIG_DIR / path))
    plt.close(fig)
    return FIG_DIR / path

def bar_chart(categories, series, path, ylabel, title, ylim=None, annotate=True):
    """Grouped bar chart. series is a dictionary of label to list of values."""

```

```

fig, ax = plt.subplots(figsize=(max(4.6, 1.05 * len(categories) + 1.6), 2.9))
n = len(series)
width = 0.8 / max(1, n)
x = np.arange(len(categories))
for i, (label, values) in enumerate(series.items()):
    pos = x - 0.4 + width * (i + 0.5)
    bars = ax.bar(pos, values, width=width, label=label)
    if annotate:
        for b, v in zip(bars, values):
            ax.text(b.get_x() + b.get_width() / 2, v, ("%2f" % v).rstrip("0"),
                    ha="center", va="bottom", fontsize=6)
ax.set_xticks(x)
ax.set_xticklabels(categories, rotation=20, ha="right", fontsize=7)
ax.set_ylabel(ylabel)
ax.set_title(title)
if ylim:
    ax.set_ylim(*ylim)
ax.legend(fontsize=7)
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig(str(FIG_DIR / path))
plt.close(fig)
return FIG_DIR / path

def line_chart(x, series, path, xlabel, ylabel, title, xticks=None):
    fig, ax = plt.subplots(figsize=(4.8, 2.9))
    for label, values in series.items():
        ax.plot(x, values, marker="o", markersize=3.5, linewidth=1.3, label=label)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    if xticks is not None:
        ax.set_xticks(xticks)
    ax.grid(alpha=0.25)
    ax.legend(fontsize=7)
    fig.tight_layout()
    fig.savefig(str(FIG_DIR / path))
    plt.close(fig)
    return FIG_DIR / path

def size_histogram(values, path, xlabel, title, bins=12):
    fig, ax = plt.subplots(figsize=(4.4, 2.6))
    ax.hist(values, bins=bins, color="#5a8f5a", edgecolor="white")
    ax.set_xlabel(xlabel)
    ax.set_ylabel("number of objects")
    ax.set_title(title)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(str(FIG_DIR / path))
    plt.close(fig)
    return FIG_DIR / path

```

src/run_experiments.py

```

"""
Experiment driver.

Running this script reproduces every number and every figure that appears in
the report. It writes

    results/tables/*.csv    the quantitative tables
    results/measurements/*.csv the per object measurements
    results/figures/*.png   the figures
    results/summary.json    a machine readable copy of the headline numbers

Usage
python run_experiments.py
"""

import csv
import json
import time

import cv2
import numpy as np

import measurement as M
import preprocessing as P
import segmentation as S
import visualisation as V
from config import (BASE_IMAGES, FIG_DIR, MEAS_DIR, RESULTS_DIR, TABLE_DIR,
                    VARIANTS, all_image_ids, has_ground_truth,
                    has_instance_ground_truth, image_record)
from dataset import build_variants, load_working
from evaluation import evaluate, measurement_errors
from ground_truth import load_gt

# -----
# One common parameter set. The same values are used for every image, so no
# method is given an unfair per image advantage.
# -----
PREP = dict(denoise_method="gaussian", ksize=5, use_illumination_correction=True)

# The tolerance of the region growing and the configuration of the clustering
# are the values that performed best on average in the parameter studies of
# sections 4 and 5, so every method is compared at its own best common setting.
DEFAULTS = {
    "threshold": dict(mode="otsu", open_k=3, close_k=7, min_area=200),
    "region_growing": dict(tolerance=25, seed_strategy="distance",
                           criterion="running_mean", connectivity=8,
                           open_k=3, close_k=7, min_area=200),
    "kmeans": dict(k=3, feature_space="lab", open_k=3, close_k=7,
                   min_area=200),

```

```

}

METHOD_LABELS = {
    "threshold": "Thresholding (Otsu)",
    "region_growing": "Region growing (tol 25)",
    "kmeans": "K means (K = 3, Lab)",
}

# Marker distance ratio used when touching objects are separated.
SPLIT_RATIO = 0.4

def write_csv(path, rows, fieldnames=None):
    if not rows:
        return path
    fieldnames = fieldnames or list(rows[0].keys())
    with open(path, "w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=fieldnames, extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)
    return path

def prep_image(image_id, **overrides):
    work, rec, _ = load_working(image_id)
    cfg = dict(PREP)
    cfg.update(overrides)
    return work, rec, P.prepare(work, **cfg)

# =====
# Experiment 1: the three methods on every annotated image
# =====
def experiment_main_comparison():
    rows = []
    per_image_panels = []
    gt_ids = [r["id"] for r in BASE_IMAGES if has_ground_truth(r["id"])]

    for image_id in gt_ids:
        work, rec, prep = prep_image(image_id)
        gt = load_gt(image_id)
        panels = [work, gt]
        titles = ["photograph", "ground truth"]

        for method in ("threshold", "region_growing", "kmeans"):
            mask, info = S.run_method(method, prep, DEFAULTS[method])
            scores = evaluate(mask, gt)
            row = {
                "image": image_id,
                "title": rec["title"],
                "method": METHOD_LABELS[method],
                "iou": scores["iou"],
                "precision": scores["precision"],
                "recall": scores["recall"],
                "f1": scores["f1"],
                "dice": scores["dice"],
                "time_s": round(info["time_s"], 3),
                "pred_count": scores["pred_count"],
                "gt_count": scores["gt_count"] if has_instance_ground_truth(image_id) else "",
                "count_error": (scores["count_error"]
                               if has_instance_ground_truth(image_id) else ""),
                "obj_precision": (scores["obj_precision"]
                                 if has_instance_ground_truth(image_id) else ""),
                "obj_recall": (scores["obj_recall"]
                              if has_instance_ground_truth(image_id) else ""),
                "obj_f1": (scores["obj_f1"]
                          if has_instance_ground_truth(image_id) else ""),
            }
            rows.append(row)
            panels.append(V.error_map(mask, gt))
            titles.append("%s\nIoU %.3f" % (METHOD_LABELS[method], scores["iou"]))

        V.panel(panels, titles, "main.%s.png" % image_id, cols=5,
               subtitle="%s. Green true positive, red false positive, blue missed"
               % rec["title"])
        per_image_panels.append("main.%s.png" % image_id)

    write_csv(TABLE_DIR / "main_comparison.csv", rows)

    # Averages over the annotated images.
    summary = []
    for method in ("threshold", "region_growing", "kmeans"):
        sel = [r for r in rows if r["method"] == METHOD_LABELS[method]]
        summary.append({
            "method": METHOD_LABELS[method],
            "mean_iou": round(float(np.mean([r["iou"] for r in sel])), 4),
            "mean_precision": round(float(np.mean([r["precision"] for r in sel])), 4),
            "mean_recall": round(float(np.mean([r["recall"] for r in sel])), 4),
            "mean_f1": round(float(np.mean([r["f1"] for r in sel])), 4),
            "mean_time_s": round(float(np.mean([r["time_s"] for r in sel])), 3),
        })
    write_csv(TABLE_DIR / "main_comparison_mean.csv", summary)

    V.bar_chart([r["image"].replace("_", " ") for r in rows[:3]],
                {METHOD_LABELS[m]: [r["iou"] for r in rows
                                     if r["method"] == METHOD_LABELS[m]]
                 for m in ("threshold", "region_growing", "kmeans")},
                "iou_by_image.png", "IoU",
                "Segmentation accuracy per image", ylim=(0, 1.05))

    V.bar_chart([s["method"] for s in summary],
                {"mean processing time": [s["mean_time_s"] for s in summary]},
                "time_by_method.png", "seconds",
                "Mean processing time per image at 900 pixel working size")
    return rows, summary, per_image_panels
def experiment_dataset_figure():

```

```

"""One overview figure of the whole dataset."""
imgs, titles = [], []
for rec in BASE_IMAGES:
    work, _, _ = load_working(rec["id"])
    imgs.append(work)
    titles.append(rec["id"] + chr(10) + rec["domain"])
V.panel(imgs, titles, "dataset_overview.png", cols=4,
        suptitle="The eight base photographs of the dataset")

gts, gtitles = [], []
for rec in BASE_IMAGES:
    if not has_ground_truth(rec["id"]):
        continue
    work, _, _ = load_working(rec["id"])
    gt = load_gt(rec["id"])
    gts.append(V.draw_boundaries(work, gt, (0, 220, 0, 2)))
    gtitles.append(rec["id"])
V.panel(gts, gtitles, "ground_truth_overview.png", cols=5,
        suptitle="Manually verified ground truth outlines")
return len(imgs)

# =====
# Experiment 2: pipeline illustration
# =====
def experiment_pipeline_figure(image_id="seeds_sunflower_01"):
    work, rec, prep = prep_image(image_id)
    gray = prep["gray"]
    smooth = P.denoise(gray, "gaussian", 5)
    flat = P.correct_illumination(smooth, objects_are_dark=prep["objects_are_dark"])
    t, _ = S.otsu_threshold_from_scratch(flat)
    raw, _ = S.segment_threshold(flat, "otsu", prep["objects_are_dark"])
    clean = S.postprocess(raw, 3, 7, 200)
    bounds = M.boundaries(clean)
    rows = M.measure_objects(clean)
    measured = V.draw_measurements(work, clean, rows)

    V.panel([work, gray, smooth, flat, raw, clean, bounds, measured],
            ["1. input photograph", "2. grey level", "3. Gaussian smoothing",
             "4. illumination corrected", "5. Otsu threshold",
             "6. morphological clean up", "7. boundary identification",
             "8. object measurement"],
            "pipeline_stages.png", cols=4,
            suptitle="Complete pipeline on %s" % rec["title"])
    V.histogram_with_threshold(flat, t, "otsu_histogram.png",
                              "Histogram after illumination correction")

    return t

# =====
# Experiment 3: thresholding study
# =====
def experiment_threshold_study():
    rows = []
    cases = [
        ("Otsu, illumination corrected", dict(mode="otsu"), True),
        ("Otsu, no illumination correction", dict(mode="otsu"), False),
        ("Fixed threshold 100", dict(mode="global", manual_value=100), False),
        ("Fixed threshold 140", dict(mode="global", manual_value=140), False),
        ("Fixed threshold 180", dict(mode="global", manual_value=180), False),
        ("Adaptive Gaussian, block 51", dict(mode="adaptive", block_size=51, C=8), False),
        ("Adaptive Gaussian, block 101", dict(mode="adaptive", block_size=101, C=8), False),
    ]
    for image_id in ("seeds_sunflower_01", "seeds_sunflower_01_illum",
                    "coins_greek_07", "coins_greek_07_illum"):
        gt = load_gt(image_id)
        for label, params, use_illum in cases:
            work, rec, prep = prep_image(image_id,
                                         use_illumination_correction=use_illum)
            p = dict(DEFAULTS["threshold"])
            p.update(params)
            mask, info = S.run_method("threshold", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "setting": label,
                        "iou": sc["iou"], "precision": sc["precision"],
                        "recall": sc["recall"], "f1": sc["f1"],
                        "time_s": round(info["time_s"], 3)})
    write_csv(TABLE_DIR / "threshold_study.csv", rows)

    labels = [c[0] for c in cases]
    series = {}
    for image_id in ("seeds_sunflower_01", "seeds_sunflower_01_illum",
                    "coins_greek_07", "coins_greek_07_illum"):
        series[image_id.replace("seeds_sunflower_01", "seeds").replace(
            "coins_greek_07", "coins").replace("_", " ")]= [
            r["iou"] for r in rows if r["image"] == image_id]
    V.bar_chart(labels, series, "threshold_study.png", "IoU",
                "Effect of the threshold strategy", ylim=(0, 1.05), annotate=False)

    # Visual comparison on the image with the lighting gradient.
    work, rec, prep_on = prep_image("seeds_sunflower_01_illum",
                                     use_illumination_correction=True)
    _, _, prep_off = prep_image("seeds_sunflower_01_illum",
                                use_illumination_correction=False)
    m_off, _ = S.run_method("threshold", prep_off, DEFAULTS["threshold"])
    m_on, _ = S.run_method("threshold", prep_on, DEFAULTS["threshold"])
    V.panel([work, prep_off["prepared"], m_off, prep_on["prepared"], m_on],
            ["photograph with lighting gradient", "grey level, no correction",
             "Otsu without correction", "after flat field correction",
             "Otsu with correction"],
            "illumination_correction.png", cols=5)

    return rows

# =====
# Experiment 4: region growing study

```

```

# =====
def experiment_region_growing_study():
    rows = []
    tolerances = [6, 10, 14, 18, 25, 35, 50]
    for image_id in ("seeds_sunflower_01", "coins_greek_07"):
        gt = load_gt(image_id)
        work, rec, prep = prep_image(image_id)
        for tol in tolerances:
            p = dict(DEFAULTS["region_growing"])
            p["tolerance"] = tol
            mask, info = S.run_method("region_growing", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "study": "tolerance",
                        "setting": str(tol), "n_seeds": info["n_seeds"],
                        "iou": sc["iou"], "precision": sc["precision"],
                        "recall": sc["recall"], "f1": sc["f1"],
                        "pred_count": sc["pred_count"],
                        "time_s": round(info["time_s"], 3)})
        for strategy in ("distance", "grid", "random"):
            p = dict(DEFAULTS["region_growing"])
            p["seed_strategy"] = strategy
            p["n_seeds"] = 40
            mask, info = S.run_method("region_growing", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "study": "seed_strategy",
                        "setting": strategy, "n_seeds": info["n_seeds"],
                        "iou": sc["iou"], "precision": sc["precision"],
                        "recall": sc["recall"], "f1": sc["f1"],
                        "pred_count": sc["pred_count"],
                        "time_s": round(info["time_s"], 3)})
        for criterion in ("running_mean", "seed_value"):
            p = dict(DEFAULTS["region_growing"])
            p["criterion"] = criterion
            mask, info = S.run_method("region_growing", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "study": "criterion",
                        "setting": criterion, "n_seeds": info["n_seeds"],
                        "iou": sc["iou"], "precision": sc["precision"],
                        "recall": sc["recall"], "f1": sc["f1"],
                        "pred_count": sc["pred_count"],
                        "time_s": round(info["time_s"], 3)})
    write_csv(TABLE_DIR / "region_growing_study.csv", rows)

    V.line_chart(tolerances,
                 {image_id.replace("_", " "):
                  [r["iou"] for r in rows
                   if r["image"] == image_id and r["study"] == "tolerance"]
                 for image_id in ("seeds_sunflower_01", "coins_greek_07")},
                 "region_growing_tolerance.png", "similarity tolerance",
                 "IoU", "Region growing, effect of the similarity tolerance",
                 xticks=tolerances)

    V.bar_chart(["distance transform", "regular grid", "random"],
                {image_id.replace("_", " "):
                 [r["iou"] for r in rows
                  if r["image"] == image_id and r["study"] == "seed_strategy"]
                 for image_id in ("seeds_sunflower_01", "coins_greek_07")},
                "region_growing_seeds.png", "IoU",
                "Region growing, effect of the seed selection strategy",
                ylim=(0, 1.05))

    # Visual illustration of leaking and of under growing.
    work, rec, prep = prep_image("seeds_sunflower_01")
    panels, titles = [work], ["photograph"]
    for tol in (6, 18, 50):
        p = dict(DEFAULTS["region_growing"])
        p["tolerance"] = tol
        mask, info = S.run_method("region_growing", prep, p)
        panels.append(V.draw_boundaries(work, mask))
        titles.append("tolerance %d, %d seeds" % (tol, info["n_seeds"]))
    V.panel(panels, titles, "region_growing_visual.png", cols=4)
    return rows

# =====
# Experiment 5: K means study
# =====
def experiment_kmeans_study():
    rows = []
    ks = [2, 3, 4, 5, 6]
    for image_id in ("seeds_sunflower_01", "coins_greek_07", "beans_coffee_06"):
        gt = load_gt(image_id)
        work, rec, prep = prep_image(image_id)
        for k in ks:
            # The sweep over K is run on the intensity feature space, so that
            # the effect of K is not mixed with the effect of the features.
            p = dict(DEFAULTS["kmeans"])
            p["feature_space"] = "intensity"
            p["k"] = k
            mask, info = S.run_method("kmeans", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "study": "K", "setting": str(k),
                        "iou": sc["iou"], "precision": sc["precision"],
                        "recall": sc["recall"], "f1": sc["f1"],
                        "pred_count": sc["pred_count"],
                        "time_s": round(info["time_s"], 3)})
        for space in ("intensity", "lab", "lab_xy"):
            p = dict(DEFAULTS["kmeans"])
            p["feature_space"] = space
            p["k"] = 3
            mask, info = S.run_method("kmeans", prep, p)
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "study": "features",
                        "setting": space, "iou": sc["iou"],
                        "precision": sc["precision"], "recall": sc["recall"],
                        "f1": sc["f1"], "pred_count": sc["pred_count"],

```

```

        "time_s": round(info["time_s"], 3)})
write_csv(TABLE_DIR / "kmeans_study.csv", rows)
V.line_chart(ks,
    {image_id.replace("_", " "):
    [r["iou"] for r in rows
    if r["image"] == image_id and r["study"] == "K"]
    for image_id in ("seeds_sunflower_01", "coins_greek_07",
                    "beans_coffee_06")},
    "kmeans_k.png", "number of clusters K", "IoU",
    "K means, effect of the number of clusters", xticks=ks)

# Cluster label maps for one image.
work, rec, prep = prep_image("beans_coffee_06")
panels, titles = [work], ["photograph"]
for k in (2, 3, 5):
    mask, info = S.segment_kmeans(pre["bgr"], prep["prepared"], k=k,
    objects_dark=prep["objects_are_dark"])
    label_img = info["label_image"].astype(np.float32)
    label_img = cv2.normalize(label_img, None, 0, 255, cv2.NORM_MINMAX)
    panels.append(cv2.applyColorMap(label_img.astype(np.uint8),
    cv2.COLORMAP_VIRIDIS))
    titles.append("K = %d cluster map" % k)
V.panel(panels, titles, "kmeans_clusters.png", cols=4)
return rows

# =====
# Experiment 6: filtering study
# =====
def experiment_filter_study():
    rows = []
    for image_id in ("seeds_sunflower_01_noise", "coins_greek_07_noise"):
        gt = load_gt(image_id)
        for method in ("none", "gaussian", "median", "bilateral"):
            work, rec, prep = prep_image(image_id, denoise_method=method, ksize=5)
            t0 = time.perf_counter()
            mask, _ = S.run_method("threshold", prep, DEFAULTS["threshold"])
            elapsed = time.perf_counter() - t0
            sc = evaluate(mask, gt)
            rows.append({"image": image_id, "filter": method,
            "iou": sc["iou"], "precision": sc["precision"],
            "recall": sc["recall"], "f1": sc["f1"],
            "pred_count": sc["pred_count"],
            "time_s": round(elapsed, 3)})
    write_csv(TABLE_DIR / "filter_study.csv", rows)
    V.bar_chart(["no filter", "Gaussian", "median", "bilateral"],
    {image_id.replace("_", " "):
    [r["iou"] for r in rows if r["image"] == image_id]
    for image_id in ("seeds_sunflower_01_noise", "coins_greek_07_noise")},
    "filter_study.png", "IoU",
    "Effect of the smoothing filter on noisy images", ylim=(0, 1.05))
    return rows

# =====
# Experiment 7: robustness to imaging conditions
# =====
def experiment_conditions():
    rows = []
    groups = [{"seeds_sunflower_01", ["seeds_sunflower_01", "seeds_sunflower_01_illum",
    "seeds_sunflower_01_dim",
    "seeds_sunflower_01_noise",
    "seeds_sunflower_01_blur"]},
    ("coins_greek_07", ["coins_greek_07", "coins_greek_07_illum",
    "coins_greek_07_noise"])]
    for base, ids in groups:
        for image_id in ids:
            gt = load_gt(image_id)
            rec = image_record(image_id)
            work, _, prep = prep_image(image_id)
            for method in ("threshold", "region_growing", "kmeans"):
                mask, info = S.run_method(method, prep, DEFAULTS[method])
                sc = evaluate(mask, gt)
                rows.append({
                    "base": base, "image": image_id,
                    "condition": rec.get("variant_label", "original photograph"),
                    "method": METHOD_LABELS[method],
                    "iou": sc["iou"], "f1": sc["f1"],
                    "precision": sc["precision"], "recall": sc["recall"],
                    "pred_count": sc["pred_count"],
                    "time_s": round(info["time_s"], 3)})
    write_csv(TABLE_DIR / "conditions.csv", rows)

    for base, ids in groups:
        conditions = [image_record(i).get("variant_label", "original")
        for i in ids]
        series = {}
        for method in ("threshold", "region_growing", "kmeans"):
            series[METHOD_LABELS[method]] = [
                r["iou"] for r in rows
                if r["base"] == base and r["method"] == METHOD_LABELS[method]]
        V.bar_chart(conditions, series, "conditions_%s.png" % base, "IoU",
        "Robustness to imaging conditions, %s" % base.replace("_", " "),
        ylim=(0, 1.05), annotate=False)

# Visual strip of the conditions.
imgs, titles = [], []
for image_id in groups[0][1]:
    work, rec, _ = load_working(image_id)
    imgs.append(work)
    titles.append(rec.get("variant_label", "original photograph"))
V.panel(imgs, titles, "conditions_strip.png", cols=5)
return rows

# =====

```

```

# Experiment 8: separating touching objects and object measurement
# =====
def experiment_split_study():
    """Effect of the marker threshold used when touching objects are split."""
    rows = []
    ratios = [0.25, 0.35, 0.45, 0.55, 0.65]
    for image_id in ("beans_coffee_06", "coins_greek_07",
                    "beans_cowpea_scale_05", "seeds_sunflower_01"):
        rec = image_record(image_id)
        work, _ = prep_image(image_id)
        mask, _ = S.run_method("threshold", prep, DEFAULTS["threshold"])
        base_count = len(M.measure_objects(mask))
        for ratio in ratios:
            split, markers = S.split_touching(mask, ratio)
            split = S.remove_small_components(split, DEFAULTS["threshold"]["min_area"])
            rows.append({"image": image_id,
                        "expected": rec.get("expected_count") or "",
                        "marker_ratio": ratio,
                        "count_without_split": base_count,
                        "count_with_split": len(M.measure_objects(split))})
    write_csv(TABLE_DIR / "split_study.csv", rows)
    V.line_chart(ratios,
                 {image_id.replace("_", " "):
                  [r["count_with_split"] for r in rows if r["image"] == image_id]
                  for image_id in ("beans_coffee_06", "coins_greek_07",
                                  "beans_cowpea_scale_05", "seeds_sunflower_01")},
                 "split_study.png", "marker distance ratio",
                 "objects detected",
                 "Separating touching objects, effect of the marker threshold",
                 xticks=ratios)

    return rows

def experiment_measurement():
    rows_all = []
    counts = []
    for image_id in ("beans_cowpea_scale_05", "beans_coffee_06", "coins_greek_07",
                    "seeds_pale_04", "seeds_sunflower_01"):
        rec = image_record(image_id)
        work, _ = prep_image(image_id)
        mask, _ = S.run_method("threshold", prep, DEFAULTS["threshold"])
        split, n_markers = S.split_touching(mask, SPLIT_RATIO)
        split = S.remove_small_components(split, DEFAULTS["threshold"]["min_area"])

        px_per_mm = rec.get("px_per_mm")
        rows_before = M.measure_objects(mask, px_per_mm)
        rows_after = M.measure_objects(split, px_per_mm)

        write_csv(MEAS_DIR / ("measurements_%s.csv" % image_id), rows_after)
        for r in rows_after:
            r2 = dict(r)
            r2["image"] = image_id
            rows_all.append(r2)

        counts.append({
            "image": image_id,
            "expected": rec.get("expected_count") or "",
            "count_before_split": len(rows_before),
            "count_after_split": len(rows_after),
            "summary_after": M.summarise(rows_after, px_per_mm),
            "size_classes": M.size_classes(rows_after),
        })

    # A full resolution copy of the annotated result, easier to read than
    # the four panel summary.
    cv2.imwrite(str(FIG_DIR / ("measured_full_%s.png" % image_id)),
                V.draw_measurements(work, split, rows_after, px_per_mm,
                                    font_scale=0.55))

    V.panel([work,
              V.draw_boundaries(work, mask),
              V.draw_boundaries(work, split),
              V.draw_measurements(work, split, rows_after, px_per_mm)],
            ["photograph", "segmented regions",
             "after separating touching objects",
             "measurements"],
            "measure_%s.png" % image_id, cols=4,
            suptitle=rec["title"])

    write_csv(TABLE_DIR / "measurement_counts.csv",
              [{"image": c["image"], "expected": c["expected"],
               "count_before_split": c["count_before_split"],
               "count_after_split": c["count_after_split"]} for c in counts])

    cowpea = [r for r in rows_all if r["image"] == "beans_cowpea_scale_05"]
    if cowpea:
        V.size_histogram([r["major_axis_mm"] for r in cowpea],
                         "cowpea_length_histogram.png",
                         "seed length in millimetres",
                         "Length distribution of the cowpea seeds", bins=8)
    seeds = [r for r in rows_all if r["image"] == "seeds_sunflower_01"]
    if seeds:
        V.size_histogram([r["area_px"] for r in seeds],
                         "seed_area_histogram.png", "area in pixels",
                         "Area distribution of the sunflower seeds", bins=10)

    return counts, rows_all

# =====
# Experiment 9: failure cases
# =====
def experiment_measurement_accuracy():
    """How accurate are the reported numbers for the objects that are found."""
    rows = []
    for image_id in ("beans_cowpea_scale_05", "beans_coffee_06", "coins_greek_07",
                    "seeds_pale_04"):

```

```

    rec = image_record(image_id)
    if not has_instance_ground_truth(image_id):
        continue
    gt = load_gt(image_id)
    work, _, prep = prep_image(image_id)
    for method in ("threshold", "kmeans"):
        mask, _ = S.run_method(method, prep, DEFAULTS[method])
        split, _ = S.split_touching(mask, SPLIT_RATIO)
        split = S.remove_small_components(split, DEFAULTS[method]["min_area"])
        err = measurement_errors(split, gt, rec.get("px_per_mm"))
        rows.append({
            "image": image_id, "method": METHOD_LABELS[method],
            "objects_in_ground_truth": rec.get("expected_count") or "",
            "matched_objects": err.get("matched", 0),
            "mean_abs_area_error_pct": err.get("mean_abs_area_error_pct", ""),
            "max_abs_area_error_pct": err.get("max_abs_area_error_pct", ""),
            "mean_abs_length_error_pct": err.get("mean_abs_major_error_pct", ""),
            "max_abs_length_error_pct": err.get("max_abs_major_error_pct", ""),
        })
    write_csv(TABLE_DIR / "measurement_accuracy.csv", rows)
    return rows

def experiment_failures():
    notes = []

    # Dense pile of beans.
    work, rec, prep = prep_image("beans_pinto_dense_08")
    mask, _ = S.run_method("threshold", prep, DEFAULTS["threshold"])
    split, _ = S.split_touching(mask, SPLIT_RATIO)
    split = S.remove_small_components(split, 200)
    n_regions = len(M.measure_objects(mask))
    n_split = len(M.measure_objects(split))
    V.panel([work, V.draw_boundaries(work, mask), V.draw_boundaries(work, split)],
            ["photograph, about 200 beans",
             "one region for the whole pile, %d regions" % n_regions,
             "after watershed, %d regions" % n_split],
            "failure_dense.png", cols=3)
    notes.append({"case": "dense pile", "regions": n_regions,
                  "regions_after_split": n_split})

    # Low contrast objects.
    work, rec, prep = prep_image("seeds_pale_04")
    gt = load_gt("seeds_pale_04")
    panels, titles = [work, gt], ["low contrast photograph", "ground truth"]
    for method in ("threshold", "region_growing", "kmeans"):
        mask, _ = S.run_method(method, prep, DEFAULTS[method])
        sc = evaluate(mask, gt)
        panels.append(V.error_map(mask, gt))
        titles.append("%s, IoU %.3f" % (METHOD_LABELS[method], sc["iou"]))
        notes.append({"case": "low contrast", "method": METHOD_LABELS[method],
                      "iou": sc["iou"]})
    V.panel(panels, titles, "failure_low_contrast.png", cols=5)

    # Shadows treated as object.
    work, rec, prep = prep_image("beans_coffee_06")
    gt = load_gt("beans_coffee_06")
    mask, _ = S.run_method("threshold", prep, DEFAULTS["threshold"])
    V.panel([work, V.error_map(mask, gt)],
            ["beans with soft cast shadows",
             "red shows shadow pixels accepted as object"],
            "failure_shadow.png", cols=2)
    return notes

# =====
# Driver
# =====
def main():
    build_variants()
    started = time.time()
    summary = {}

    print("1. main comparison")
    rows, means, panels = experiment_main_comparison()
    summary["main"] = rows
    summary["main_mean"] = means

    print("1b. dataset figure")
    experiment_dataset_figure()

    print("2. pipeline figure")
    summary["otsu_threshold_example"] = experiment_pipeline_figure()

    print("3. threshold study")
    summary["threshold_study"] = experiment_threshold_study()

    print("4. region growing study")
    summary["region_growing_study"] = experiment_region_growing_study()

    print("5. k means study")
    summary["kmeans_study"] = experiment_kmeans_study()

    print("6. filter study")
    summary["filter_study"] = experiment_filter_study()

    print("7. imaging conditions")
    summary["conditions"] = experiment_conditions()

    print("8. separating touching objects")
    summary["split_study"] = experiment_split_study()

    print("9. measurement")
    counts, meas = experiment_measurement()
    summary["measurement_counts"] = counts
    print("10. measurement accuracy")

```

```

summary["measurement_accuracy"] = experiment_measurement_accuracy()

print("11. failure cases")
summary["failures"] = experiment_failures()

summary["runtime_s"] = round(time.time() - started, 1)
with open(RESULTS_DIR / "summary.json", "w", encoding="utf-8") as fh:
    json.dump(summary, fh, indent=2)
print("done in %.1f s" % summary["runtime_s"])

if __name__ == "__main__":
    main()

```

src/make_report.py

```

"""
PDF renderer of the report.

The content itself lives in report_content.py as a sequence of neutral blocks.
This module turns those blocks into the report PDF. Run the
experiments first, then this script.

    python run_experiments.py
    python make_report.py
"""

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.lib.utils import ImageReader
from reportlab.platypus import (BaseDocTemplate, Frame, Image, KeepTogether,
                                PageBreak, PageTemplate, Paragraph,
                                Preformatted, Spacer, Table, TableStyle)

import report_content as C
from config import FIG_DIR, REPORT_DIR

PAGE_W, PAGE_H = A4
MARGIN = 2.0 * cm
CONTENT_W = PAGE_W - 2 * MARGIN

AUTHOR = C.AUTHOR
PROJECT = C.PROJECT
TITLE = C.TITLE

# -----
# Paragraph styles
# -----
_sheet = getSampleStyleSheet()
BODY = ParagraphStyle("body", parent=_sheet["BodyText"], fontSize=9.6,
                      leading=13.4, alignment=TA_JUSTIFY, spaceAfter=6)
H1 = ParagraphStyle("h1", parent=_sheet["Heading1"], fontSize=15, leading=18,
                    spaceBefore=14, spaceAfter=8,
                    textColor=colors.HexColor("#12325c"))
H2 = ParagraphStyle("h2", parent=_sheet["Heading2"], fontSize=11.8, leading=15,
                    spaceBefore=10, spaceAfter=5,
                    textColor=colors.HexColor("#1d4d80"))
H3 = ParagraphStyle("h3", parent=_sheet["Heading3"], fontSize=10.4, leading=13,
                    spaceBefore=8, spaceAfter=4,
                    textColor=colors.HexColor("#28527a"))
CAPTION = ParagraphStyle("caption", parent=BODY, fontSize=8.2, leading=10.5,
                          alignment=TA_CENTER,
                          textColor=colors.HexColor("#44484f"),
                          spaceBefore=2, spaceAfter=10)
CODE = ParagraphStyle("code", parent=_sheet["Code"], fontSize=5.6, leading=6.6)
CODE_HEAD = ParagraphStyle("codehead", parent=H3, fontSize=9.4, spaceBefore=10)
PRE = ParagraphStyle("pre", parent=_sheet["Code"], fontSize=8, leading=10.5)

def paragraph_style(name):
    """Build a reportlab style from one of the shared style descriptions."""
    spec = C.STYLES[name]
    font = "Helvetica"
    if spec.get("mono"):
        font = "Courier"
    elif spec["bold"]:
        font = "Helvetica-Bold"
    elif spec["italic"]:
        font = "Helvetica-Oblique"
    indent = spec.get("indent_cm", 0) * cm
    return ParagraphStyle(
        "shared_" + name, parent=BODY, fontName=font, fontSize=spec["size"],
        leading=spec["size"] * 1.38,
        alignment=TA_CENTER if spec["centre"] else TA_JUSTIFY,
        textColor=colors.HexColor(spec["colour"]),
        leftIndent=indent, rightIndent=indent,
        spaceBefore=4 if spec["centre"] else 0, spaceAfter=6)

STYLE_CACHE = {name: paragraph_style(name) for name in C.STYLES}

# -----
# Block renderers
# -----
def render_table(header, rows, col_widths_cm, font_size, highlight_rows):
    widths = [w * cm for w in col_widths_cm] if col_widths_cm else None
    data = [[Paragraph("<b>%s</b>" % c, ParagraphStyle(
        "th", parent=BODY, fontSize=font_size, leading=font_size + 2,
        textColor=colors.white)) for c in header]]
    for r in rows:
        data.append([Paragraph(str(c), ParagraphStyle(

```

```

        "td", parent=BODY, fontSize=font_size, leading=font_size + 2.2))
        for c in r])
    t = Table(data, colwidths=widths, repeatRows=1, hAlign="CENTER")
    style = [
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1d4d80")),
        ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#b8c2cc")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("LEFTPADDING", (0, 0), (-1, -1), 3),
        ("RIGHTPADDING", (0, 0), (-1, -1), 3),
        ("TOPPADDING", (0, 0), (-1, -1), 2.5),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 2.5),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1),
         [colors.white, colors.HexColor("#eef2f7")]),
    ]
    for i in highlight_rows:
        style.append(("BACKGROUND", (0, i + 1), (-1, i + 1),
                     colors.HexColor("#d8ecd8")))
    t.setStyle(TableStyle(style))
    return t

def render_figure(name, caption, max_height_cm):
    path = FIG_DIR / name
    reader = ImageReader(str(path))
    iw, ih = reader.getSize()
    w = CONTENT_W
    h = w * ih / float(iw)
    if h > max_height_cm * cm:
        h = max_height_cm * cm
        w = h * iw / float(ih)
    img = Image(str(path), width=w, height=h)
    img.hAlign = "CENTER"
    return KeepTogether([img, Paragraph(caption, CAPTION)])
def render_rule(width_cm, colour, thickness):
    width = width_cm * cm if width_cm else CONTENT_W
    t = Table([""], colwidths=[width], rowHeights=[0.01])
    t.setStyle(TableStyle([
        ("LINEBELOW", (0, 0), (-1, -1), thickness, colors.HexColor(colour)),
        ("TOPPADDING", (0, 0), (-1, -1), 0),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 0),
    ]))
    t.hAlign = "CENTER"
    return t

def build_story():
    story = []
    for block in C.BLOCKS:
        kind = block[0]
        if kind == "p":
            story.append(Paragraph(block[1], STYLE_CACHE[block[2]]))
        elif kind == "h1":
            story.append(Paragraph(block[1], H1))
        elif kind == "h2":
            story.append(Paragraph(block[1], H2))
        elif kind == "h3":
            story.append(Paragraph(block[1], H3))
        elif kind == "bullets":
            for item in block[1]:
                story.append(Paragraph("&bull;&nbsp;&nbsp;&nbsp;" + item,
                                     ParagraphStyle("b", parent=BODY,
                                     leftIndent=12,
                                     spaceAfter=3)))
            story.append(Spacer(1, 4))
        elif kind == "table":
            story.append(render_table(block[1], block[2], block[3], block[4],
                                     block[5]))
            story.append(Spacer(1, 8))
        elif kind == "caption":
            story.append(Paragraph(block[1], CAPTION))
        elif kind == "figure":
            story.append(render_figure(block[1], block[2], block[3]))
        elif kind == "pre":
            story.append(Preformatted(block[1], PRE))
        elif kind == "code_heading":
            story.append(Paragraph(block[1], CODE_HEAD))
        elif kind == "code":
            story.append(Preformatted(block[1], CODE))
        elif kind == "rule":
            _, width_cm, colour, thickness, before, after = block
            story.append(Spacer(1, before))
            story.append(render_rule(width_cm, colour, thickness))
            story.append(Spacer(1, after))
        elif kind == "spacer":
            story.append(Spacer(1, block[1]))
        elif kind == "page_break":
            story.append(PageBreak())
        else:
            raise ValueError("Unknown block: %s % kind)
    return story

# -----
# Page furniture
# -----
def draw_furniture(canvas, doc):
    # The cover page carries no running header and no page number.
    if doc.page == 1:
        return
    canvas.saveState()
    canvas.setFont("Helvetica", 7.4)
    canvas.setFillColor(colors.HexColor("#6a707a"))
    canvas.drawString(MARGIN, 1.15 * cm, AUTHOR)
    canvas.drawRightString(PAGE_W - MARGIN, 1.15 * cm, "Page %d" % doc.page)
    canvas.setStrokeColor(colors.HexColor("#c6ccd4"))
    canvas.line(MARGIN, PAGE_H - MARGIN + 0.35 * cm,

```

```

        PAGE_W - MARGIN, PAGE_H - MARGIN + 0.35 * cm)
    canvas.drawString(MARGIN, PAGE_H - MARGIN + 0.55 * cm,
        PROJECT + " | " + TITLE)
    canvas.restoreState()

def build():
    out = REPORT_DIR / "Automated_Image_Segmentation_and_Object_Measurement.pdf"
    doc = BaseDocTemplate(str(out), pagesize=A4, leftMargin=MARGIN,
        rightMargin=MARGIN, topMargin=MARGIN,
        bottomMargin=MARGIN + 0.4 * cm,
        title=TITLE, author=AUTHOR)
    frame = Frame(MARGIN, MARGIN + 0.4 * cm, CONTENT_W,
        PAGE_H - 2 * MARGIN - 0.4 * cm, id="normal")
    doc.addPageTemplates([PageTemplate(id="main", frames=[frame],
        onPage=draw_furniture)])
    doc.build(build_story())
    print("written:", out)
    return out

if __name__ == "__main__":
    build()

```

src/make_report_docx.py

```

"""
Word renderer of the report.

The content lives in report_content.py as a sequence of neutral blocks, exactly
the same blocks that make_report.py renders into the PDF. This module turns
them into the report DOCX, so the Word file and the PDF always
carry the same text, the same tables and the same figures.

    python run_experiments.py
    python make_report_docx.py
"""

import os
import re
from html import unescape

# Selects the Word page numbers in the contents table of report_content.
os.environ.setdefault("REPORT_TARGET", "docx")

from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK, WD_TAB_ALIGNMENT
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

import report_content as C
from config import FIG_DIR, REPORT_DIR

PAGE_MARGIN_CM = 2.0
CONTENT_W_CM = C.CONTENT_W_CM

NAVY = RGBColor(0x12, 0x32, 0x5C)
BLUE = RGBColor(0x1D, 0x4D, 0x80)
STEEL = RGBColor(0x28, 0x52, 0x7A)
GREY = RGBColor(0x44, 0x48, 0x4F)

HEADER_FILL = "1D4D80"
BAND_FILL = "EEF2F7"
HIGHLIGHT_FILL = "D8ECD8"

BODY_FONT = "Calibri"
MONO_FONT = "Consolas"

# -----
# Low level Word helpers
# -----
def shade_cell(cell, hex_fill):
    tc_pr = cell._tc.get_or_add_tcPr()
    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), hex_fill)
    tc_pr.append(shd)

def paragraph_border_below(paragraph, hex_colour, size_eighths):
    p_pr = paragraph._p.get_or_add_pPr()
    borders = OxmlElement("w:pBdr")
    bottom = OxmlElement("w:bottom")
    bottom.set(qn("w:val"), "single")
    bottom.set(qn("w:sz"), str(size_eighths))
    bottom.set(qn("w:space"), "1")
    bottom.set(qn("w:color"), hex_colour.lstrip("#"))
    borders.append(bottom)
    p_pr.append(borders)

def add_field(paragraph, instruction):
    """Insert a Word field, used for the page number in the footer."""
    run = paragraph.add_run()
    begin = OxmlElement("w:fldChar")
    begin.set(qn("w:fldCharType"), "begin")
    instr = OxmlElement("w:instrText")
    instr.set(qn("xml:space"), "preserve")
    instr.text = instruction
    end = OxmlElement("w:fldChar")

```

```

        end.set(qn("w:fldCharType"), "end")
        run._r.append(begin)
        run._r.append(instr)
        run._r.append(end)
        return run

def keep_with_next(paragraph):
    p_pr = paragraph._p.get_or_add_pPr()
    keep = OxmlElement("w:keepNext")
    p_pr.append(keep)

# -----
# Inline markup
#
# The content uses a very small subset of markup: <b>bold</b>, a fixed pitch
# span written as <font face='Courier'>...</font>, and HTML entities. This
# parser turns one marked up string into a list of (text, bold, mono) runs.
# -----
TAG = re.compile(r"</?(b|i|font)(?:\s[^\>]*)?>", re.IGNORECASE)

def parse_runs(text):
    runs = []
    bold = italic = mono = False
    position = 0
    for match in TAG.finditer(text):
        chunk = text[position:match.start()]
        if chunk:
            runs.append((unescape(chunk), bold, italic, mono))
            tag = match.group(0).lower()
            name = match.group(1).lower()
            closing = tag.startswith("</")
            if name == "b":
                bold = not closing
            elif name == "i":
                italic = not closing
            elif name == "font":
                mono = (not closing) and "courier" in tag
            position = match.end()
        tail = text[position:]
        if tail:
            runs.append((unescape(tail), bold, italic, mono))
    return runs

def write_runs(paragraph, text, size, bold=False, italic=False, colour=None,
               mono=False):
    for chunk, run_bold, run_italic, run_mono in parse_runs(text):
        run = paragraph.add_run(chunk.replace(" ", " "))
        run.font.size = Pt(size)
        run.font.name = MONO_FONT if (mono or run_mono) else BODY_FONT
        run.font.bold = bold or run_bold
        run.font.italic = italic or run_italic
        if colour is not None:
            run.font.color.rgb = colour
    return paragraph

# -----
# Block renderers
# -----
def add_paragraph(doc, text, style_name):
    spec = C.STYLES[style_name]
    par = doc.add_paragraph()
    par.alignment = (WD_ALIGN_PARAGRAPH.CENTER if spec["centre"]
                    else WD_ALIGN_PARAGRAPH.JUSTIFY)
    par.paragraph_format.space_after = Pt(6)
    par.paragraph_format.space_before = Pt(4 if spec["centre"] else 0)
    if spec.get("indent_cm"):
        par.paragraph_format.left_indent = Cm(spec["indent_cm"])
        par.paragraph_format.right_indent = Cm(spec["indent_cm"])
    colour = RGBColor.from_string(spec["colour"].lstrip("#").upper())
    write_runs(par, text, spec["size"], spec["bold"], spec["italic"], colour,
               spec.get("mono", False))
    return par

def add_heading(doc, text, level):
    sizes = {1: 15, 2: 11.8, 3: 10.4}
    colours = {1: NAVY, 2: BLUE, 3: STEEL}
    par = doc.add_paragraph()
    par.paragraph_format.space_before = Pt({1: 14, 2: 10, 3: 8}[level])
    par.paragraph_format.space_after = Pt({1: 8, 2: 5, 3: 4}[level])
    write_runs(par, text, sizes[level], bold=True, colour=colours[level])
    keep_with_next(par)
    # Marking the outline level makes the headings appear in the Word
    # navigation pane and lets Word build a table of contents from them.
    p_pr = par._p.get_or_add_pPr()
    outline = OxmlElement("w:outlineLvl")
    outline.set(qn("w:val"), str(level - 1))
    p_pr.append(outline)
    return par

def add_bullets(doc, items):
    for item in items:
        par = doc.add_paragraph(style="List Bullet")
        par.paragraph_format.space_after = Pt(3)
        par.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
        write_runs(par, item, 9.6)

def add_caption(doc, text):
    par = doc.add_paragraph()
    par.alignment = WD_ALIGN_PARAGRAPH.CENTER
    par.paragraph_format.space_before = Pt(2)

```

```

par.paragraph_format.space_after = Pt(10)
write_runs(par, text, 8.2, italic=True, colour=GREY)

def add_table(doc, header, rows, col_widths_cm, font_size, highlight_rows):
    # A header of empty strings is a decorative rule in the PDF. In Word an
    # empty shaded row reads as a mistake, so it is dropped.
    show_header = any(str(c).strip() for c in header)
    table = doc.add_table(rows=1 if show_header else 0, cols=len(header))
    table.style = "Table Grid"
    table.alignment = WD_TABLE_ALIGNMENT.CENTER
    table.autofit = False

    widths = col_widths_cm
    if not widths:
        widths = [CONTENT_W_CM / float(len(header))] * len(header)

    if show_header:
        for i, text in enumerate(header):
            cell = table.rows[0].cells[i]
            cell.text = ""
            par = cell.paragraphs[0]
            write_runs(par, str(text), font_size, bold=True,
                       colour=RGBColor(0xFF, 0xFF, 0xFF))
            shade_cell(cell, HEADER_FILL)

    for r, row in enumerate(rows):
        cells = table.add_row().cells
        for i, value in enumerate(row):
            cells[i].text = ""
            write_runs(cells[i].paragraphs[0], str(value), font_size)
        if r in highlight_rows:
            fill = HIGHLIGHT_FILL
        elif r % 2 == 1:
            fill = BAND_FILL
        else:
            fill = None
        if fill:
            for cell in cells:
                shade_cell(cell, fill)

    for row in table.rows:
        for i, cell in enumerate(row.cells):
            cell.width = Cm(widths[i])

    if show_header:
        # Repeat the header row when a table runs over a page boundary.
        tr_pr = table.rows[0].tr.get_or_add_trPr()
        header_flag = XmlElement("w:tblHeader")
        header_flag.set(qn("w:val"), "true")
        tr_pr.append(header_flag)

    doc.add_paragraph().paragraph_format.space_after = Pt(4)
    return table

def add_figure(doc, name, caption, max_height_cm):
    from PIL import Image as PILImage # Pillow ships with python-docx

    path = FIG_DIR / name
    with PILImage.open(path) as im:
        iw, ih = im.size
        width = CONTENT_W_CM
        height = width * ih / float(iw)
        if height > max_height_cm:
            height = max_height_cm
            width = height * iw / float(ih)
        par = doc.add_paragraph()
        par.alignment = WD_ALIGN_PARAGRAPH.CENTER
        par.paragraph_format.space_after = Pt(2)
        par.add_run().add_picture(str(path), width=Cm(width), height=Cm(height))
        keep_with_next(par)
        add_caption(doc, caption)

def add_pre(doc, text, size=8.0):
    par = doc.add_paragraph()
    par.paragraph_format.space_after = Pt(2)
    par.paragraph_format.space_before = Pt(2)
    par.paragraph_format.left_indent = Cm(0.6)
    lines = text.split("\n")
    for i, line in enumerate(lines):
        run = par.add_run(line)
        run.font.name = MONO_FONT
        run.font.size = Pt(size)
        if i < len(lines) - 1:
            run.add_break()
    return par

def add_rule(doc, width_cm, colour, thickness, space_before, space_after):
    par = doc.add_paragraph()
    par.alignment = WD_ALIGN_PARAGRAPH.CENTER
    par.paragraph_format.space_before = Pt(space_before)
    par.paragraph_format.space_after = Pt(space_after)
    if width_cm and width_cm < CONTENT_W_CM:
        indent = (CONTENT_W_CM - width_cm) / 2.0
        par.paragraph_format.left_indent = Cm(indent)
        par.paragraph_format.right_indent = Cm(indent)
    # Word border widths are given in eighths of a point.
    paragraph_border_below(par, colour, max(4, int(round(thickness * 8))))

def add_spacer(doc, points):
    par = doc.add_paragraph()
    par.paragraph_format.space_after = Pt(0)

```

```

    par.paragraph_format.space_before = Pt(0)
    run = par.add_run("")
    run.font.size = Pt(points)

# -----
# Page furniture
# -----
def setup_section(doc):
    section = doc.sections[0]
    section.page_width = Cm(21.0)
    section.page_height = Cm(29.7)
    section.left_margin = Cm(PAGE_MARGIN_CM)
    section.right_margin = Cm(PAGE_MARGIN_CM)
    section.top_margin = Cm(PAGE_MARGIN_CM)
    section.bottom_margin = Cm(PAGE_MARGIN_CM)
    # The cover page carries no running header and no page number.
    section.different_first_page_header_footer = True

    header = section.header.paragraphs[0]
    header.alignment = WD_ALIGN_PARAGRAPH.LEFT
    write_runs(header, C.COURSE + " | " + C.TITLE, 7.4,
               colour=RGBColor(0x6A, 0x70, 0x7A))
    paragraph_border_below(header, "#C6CCD4", 4)

    footer = section.footer.paragraphs[0]
    footer.alignment = WD_ALIGN_PARAGRAPH.LEFT
    write_runs(footer, "%s % % C.AUTHOR,
                      7.4, colour=RGBColor(0x6A, 0x70, 0x7A))
    footer.paragraph_format.tab_stops.add_tab_stop(
        Cm(CONTENT_W_CM), WD_TAB_ALIGNMENT.RIGHT)
    run = footer.add_run("\t")
    run.font.size = Pt(7.4)
    page_run = footer.add_run("Page ")
    page_run.font.size = Pt(7.4)
    page_run.font.color.rgb = RGBColor(0x6A, 0x70, 0x7A)
    field = add_field(footer, "PAGE")
    field.font.size = Pt(7.4)
    field.font.color.rgb = RGBColor(0x6A, 0x70, 0x7A)

def set_default_font(doc):
    style = doc.styles["Normal"]
    style.font.name = BODY_FONT
    style.font.size = Pt(9.6)
    style.paragraph_format.space_after = Pt(6)
    rpr = style.element.get_or_add_rPr()
    rfonts = rpr.get_or_add_rFonts()
    rfonts.set(qn("w:EastAsia"), BODY_FONT)

# -----
# Driver
# -----
def build():
    doc = Document()
    set_default_font(doc)
    setup_section(doc)

    for block in C.BLOCKS:
        kind = block[0]
        if kind == "p":
            add_paragraph(doc, block[1], block[2])
        elif kind == "h1":
            add_heading(doc, block[1], 1)
        elif kind == "h2":
            add_heading(doc, block[1], 2)
        elif kind == "h3":
            add_heading(doc, block[1], 3)
        elif kind == "bullets":
            add_bullets(doc, block[1])
        elif kind == "table":
            add_table(doc, block[1], block[2], block[3], block[4], block[5])
        elif kind == "caption":
            add_caption(doc, block[1])
        elif kind == "figure":
            add_figure(doc, block[1], block[2], block[3])
        elif kind == "pre":
            add_pre(doc, block[1])
        elif kind == "code_heading":
            par = doc.add_paragraph()
            par.paragraph_format.space_before = Pt(10)
            par.paragraph_format.space_after = Pt(3)
            write_runs(par, block[1], 9.4, bold=True, colour=STEEL)
            keep_with_next(par)
        elif kind == "code":
            add_pre(doc, block[1], size=6.0)
        elif kind == "rule":
            add_rule(doc, block[1], block[2], block[3], block[4], block[5])
        elif kind == "spacer":
            add_spacer(doc, block[1])
        elif kind == "page_break":
            doc.add_paragraph().add_run().add_break(WD_BREAK.PAGE)
        else:
            raise ValueError("Unknown block: %s" % kind)

    doc.core_properties.title = C.TITLE
    doc.core_properties.author = C.AUTHOR
    doc.core_properties.subject = C.PROJECT

    out = REPORT_DIR / "Automated_Image_Segmentation_and_Object_Measurement.docx"
    doc.save(str(out))
    print("written:", out)
    return out
if __name__ == "__main__":
    build()

```

src/report_content.py

```

"""
Content of the report.

The report is written once here, as a sequence of neutral blocks, and rendered
twice: make_report.py turns the blocks into the PDF and make_report_docx.py
turns the same blocks into a Word document. Keeping a single source for the
content means that the two versions of the report can never drift apart.

Every block is a tuple whose first element names its kind:

    ("h1" | "h2" | "h3", text)
    ("p", text, style)           style is one of the names in STYLES below
    ("bullets", [text, ...])
    ("table", header, rows, col_widths_cm, font_size, highlight_rows)
    ("caption", text)
    ("figure", filename, caption, max_height_cm)
    ("pre", text)                fixed pitch block, for example a command list
    ("code_heading", text), ("code", text)
    ("rule", width_cm, colour, thickness, space_before, space_after)
    ("spacer", points), ("page_break",)

Light inline markup is allowed inside the text of a paragraph: <b>bold</b>,
<font face='Courier'>fixed pitch</font>, and the entities &bull; &nbsp;
&mdot; &amp; &lt; &gt;.
"""

import csv
import json
import os

# Which renderer is building the document. make_report_docx.py sets this to
# "docx" before it imports this module, so that the contents table can carry
# the page numbers of the format actually being produced.
TARGET = os.environ.get("REPORT_TARGET", "pdf")

from config import (BASE_IMAGES, DATA_DIR, FIG_DIR, RESULTS_DIR, SRC_DIR,
                    TABLE_DIR, VARIANTS)

AUTHOR = "Nii Yartey Gidiglo"
PROJECT = "Computer vision project"
TITLE = "Automated Image Segmentation and Object Measurement System"

# Printable width of an A4 page at the margins used by both renderers.
CONTENT_W_CM = 17.0

# Paragraph styles that both renderers understand.
STYLES = {
    "body": dict(size=9.6, bold=False, italic=False, centre=False,
                  colour="#000000"),
    "small": dict(size=8.6, bold=False, italic=False, centre=False,
                  colour="#000000"),
    "flow": dict(size=10.0, bold=True, italic=False, centre=True,
                  colour="#000000"),
    "equation": dict(size=9.6, bold=False, italic=False, centre=True,
                     colour="#000000", mono=True),
    "uni": dict(size=19.0, bold=True, italic=False, centre=True,
                colour="#12325c"),
    "faculty": dict(size=11.0, bold=False, italic=False, centre=True,
                    colour="#333a44"),
    "courseline": dict(size=12.0, bold=True, italic=False, centre=True,
                       colour="#1d4d80"),
    "eyebrow": dict(size=9.0, bold=True, italic=False, centre=True,
                    colour="#6a707a"),
    "covertitle": dict(size=23.0, bold=True, italic=False, centre=True,
                       colour="#12325c"),
    "coversub": dict(size=13.0, bold=False, italic=False, centre=True,
                     colour="#1d4d80"),
    "abstract": dict(size=10.5, bold=False, italic=True, centre=True,
                     colour="#333a44", indent_cm=1.5),
}

BLOCKS = []

# -----
# The small authoring interface used by the content below
# -----
def p(text, style="body"):
    BLOCKS.append(("p", text, style))

def h1(text):
    BLOCKS.append(("h1", text))

def h2(text):
    BLOCKS.append(("h2", text))

def h3(text):
    BLOCKS.append(("h3", text))

def spacer(points=6):
    BLOCKS.append(("spacer", points))

def page_break():
    BLOCKS.append(("page_break",))

def bullets(items):
    BLOCKS.append(("bullets", list(items)))
def figure(name, caption, max_height_cm=13.5):

```

```

    if (FIG_DIR / name).exists():
        BLOCKS.append(("figure", name, caption, max_height_cm))

def make_table(header, rows, col_widths=None, font_size=7.6, highlight_rows=()):
    BLOCKS.append(("table", list(header), [list(r) for r in rows],
                  list(col_widths) if col_widths else None, font_size,
                  tuple(highlight_rows)))

def table_caption(text):
    BLOCKS.append(("caption", text))

def pre(text):
    BLOCKS.append(("pre", text))
def code_heading(text):
    BLOCKS.append(("code_heading", text))

def code_block(text):
    BLOCKS.append(("code", text))

def rule(width_cm=None, colour="#1d4d80", thickness=1.1, space_before=6,
         space_after=6):
    BLOCKS.append(("rule", width_cm, colour, thickness, space_before,
                  space_after))

def read_table(name):
    with open(TABLE_DIR / name, newline="", encoding="utf-8") as fh:
        return list(csv.DictReader(fh))

with open(RESULTS_DIR / "summary.json", encoding="utf-8") as fh:
    SUMMARY = json.load(fh)

MAIN = read_table("main_comparison.csv")
MAIN_MEAN = read_table("main_comparison_mean.csv")
THRESH = read_table("threshold_study.csv")
RG = read_table("region_growing_study.csv")
KM = read_table("kmeans_study.csv")
FILT = read_table("filter_study.csv")
COND = read_table("conditions.csv")
SPLIT = read_table("split_study.csv")
COUNTS = read_table("measurement_counts.csv")
ACC = read_table("measurement_accuracy.csv")

METHODS = [m["method"] for m in MAIN_MEAN]
BEST = max(MAIN_MEAN, key=lambda r: float(r["mean_iou"]))
FASTEST = min(MAIN_MEAN, key=lambda r: float(r["mean_time_s"]))

def nice(image_id):
    return image_id.replace("_", " ")

# =====
# Title page
# =====
spacer(52)

p("COMPUTER VISION PROJECT", "eyebrow")
p(TITLE, "covertitle")
p("Thresholding, region growing and K means clustering", "coversub")
rule(width_cm=6, colour="#9aa6b4", thickness=0.8, space_before=14,
     space_after=16)

p("Segmentation of objects of interest from an image, followed by the "
  "computation of meaningful measurements about the detected objects, "
  "implemented in Python and OpenCV with classical computer vision only.",
  "abstract")
spacer(48)

make_table(["", ""], [
    ["Author", AUTHOR],
    ["Repository", "github.com/Qwamzz/Computer-Vision-Projects"],
    ["Application domain", "Counting and measuring seeds, beans and coins"],
    ["Implementation", "Python 3 and OpenCV, no learned or pre-trained model"],
    ["Dataset", "%d real photographs and %d controlled degradations of them"
     % (len(BASE_IMAGES), len(VARIANTS))],
], col_widths=[5.6, CONTENT_W_CM - 5.6], font_size=9.4)

page_break()

# =====
# Contents
# =====
h1("Contents")
# The two renderers paginate slightly differently, because Word places the
# figures on its own pages, so each one carries its own measured page numbers.
# The numbers were read back from the finished document of each format.
CONTENTS = [
    ["1", "Problem definition and objectives", {"pdf": "3", "docx": "3"}],
    ["2", "Dataset", {"pdf": "5", "docx": "4"}],
    ["3", "Methodology", {"pdf": "7", "docx": "6"}],
    ["4", "Implementation", {"pdf": "11", "docx": "9"}],
    ["5", "Experimental results", {"pdf": "12", "docx": "10"}],
    ["6", "Critical analysis", {"pdf": "24", "docx": "21"}],
    ["7", "Limitations and failure cases", {"pdf": "27", "docx": "23"}],
    ["8", "Conclusion", {"pdf": "29", "docx": "25"}],
    ["9", "References", {"pdf": "29", "docx": "25"}],
    ["A", "Complete source code", {"pdf": "31", "docx": "27"}],
]
make_table(["Section", "Title", "Page"],

```

```

[[number, title, pages[TARGET]] for number, title, pages in CONTENTS],
col_widths=[2.0, CONTENT_W_CM - 4.4, 2.4], font_size=9)
page_break()

# =====
# 1. Problem definition
# =====
h1("1. Problem definition and objectives")
h2("1.1 The problem")
p("Many practical inspection tasks reduce to the same question: how many "
  "objects are present in this picture, and how large is each one. A seed "
  "laboratory grading a sample of cowpea needs the length and the width of "
  "every seed. A quality inspector counting roasted coffee beans needs the "
  "count and the size distribution. A numismatic archive photographing a tray "
  "of coins needs one region per coin so that each coin can be catalogued. In "
  "all of these cases the answer is produced by two steps, a segmentation step "
  "that decides which pixels belong to an object, and a measurement step that "
  "turns each region into numbers.")
p("This project builds that system with classical computer vision only. No "
  "learned detector and no pre-trained recognition model is used anywhere. "
  "Every decision is made by an algorithm whose behaviour can be written down "
  "in closed form, which is what makes the failure analysis in section 7 "
  "possible.")

h2("1.2 Objectives")
bullets([
  "Assemble a dataset of real photographs of countable objects under varying "
  "imaging conditions, and produce manually verified ground truth masks for "
  "a representative subset of it.",
  "Implement threshold based segmentation, region growing and K means "
  "clustering from first principles rather than treating them as black boxes.",
  "Investigate the effect of the parameters that each method exposes: the "
  "threshold strategy, the seed point selection and the similarity criterion "
  "of the region growing, and the number of clusters and the feature space "
  "of the clustering.",
  "Identify object boundaries and extract each object as a separate region, "
  "including a step that separates objects that touch each other.",
  "Measure the count, area, perimeter, centroid, bounding box and size of "
  "every object, and convert those numbers into millimetres where a physical "
  "scale is available.",
  "Evaluate the three approaches quantitatively with Intersection over "
  "Union, precision, recall and F1 score against the ground truth, and "
  "measure the processing time, the sensitivity to noise and the sensitivity "
  "to illumination.",
  "Discuss where each method fails and why, and show which earlier "
  "techniques of the course repair those failures.",
])

h2("1.3 The pipeline that was built")
p("The finished system implements the workflow required by the question:")
p("<b>Input image &gt; image preparation &gt; segmentation &gt; boundary "
  "identification &gt; object extraction &gt; object measurement &gt; "
  "performance evaluation</b>",
  "flow")
figure("pipeline_stages.png",
  "Figure 1. The eight stages of the pipeline on one photograph of the "
  "dataset. Stage 4 removes the lighting gradient, stage 5 applies the "
  "Otsu threshold, stage 6 cleans the mask morphologically, stage 7 "
  "extracts the boundaries and stage 8 reports the measurements.")

page_break()

# =====
# 2. Dataset
# =====
h1("2. Dataset")
h2("2.1 Source of the images")
p("The dataset contains %d base photographs. Every one of them is a real "
  "photograph taken by a person with a camera. None of them was produced by "
  "an image generation model. Seven of them come from Wikimedia Commons under "
  "a Creative Commons licence, and one is the coins photograph that is "
  "distributed as sample data with scikit-image and originates from a museum "
  "collection. The full provenance is recorded in src/config.py and repeated "
  "in Table 1, so that any reader can retrieve the exact files."
  % len(BASE_IMAGES))

rows = []
for rec in BASE_IMAGES:
    rows.append([rec["id"], rec["title"], rec["domain"],
                  rec.get("expected_count") or "not counted",
                  rec.get("licence", "")])
make_table(["identifier", "content", "application domain", "objects", "licence"],
  rows, col_widths=[3.5, 3.9, 3.6, 1.7, 2.3],
  font_size=7.2)
table_caption("Table 1. The base photographs of the dataset. The object count "
  "is given for the images that carry an instance level ground "
  "truth.")

figure("dataset_overview.png",
  "Figure 2. The eight base photographs. They cover four of the "
  "application domains suggested by the question and they differ in "
  "contrast, in background, in object density and in the amount of "
  "shadow present.")

h2("2.2 Imaging conditions")
p("The question asks for the system to be tested under illumination changes, "
  "background variation, noise, partially touching objects and objects of "
  "different sizes. Part of this variation is already present in the "
  "photographs themselves: the coins sit on a dark background with a strong "
  "brightness falloff, the coffee beans cast soft shadows and carry specular "
  "highlights, the pale seeds have almost no contrast against their "
  "background, the third sunflower photograph is partly out of focus, and the "
  "pinto bean photograph is a dense pile in which nearly every object touches "
  "a neighbour.")
p("To obtain a controlled comparison rather than an anecdotal one, four "
  "further conditions are generated from two of the photographs by applying a "
```

```

"known degradation to the real image: a diagonal lighting ramp that scales "
"brightness from 0.30 to 1.20, an under exposure that multiplies the "
"brightness by 0.45 and lifts the black point, additive zero mean Gaussian "
"noise with a standard deviation of 18 grey levels, and a defocus blur "
"modelled by a Gaussian point spread function with a fifteen pixel kernel. "
"The geometry of the objects is untouched by all four operations, so the "
"ground truth mask of the original photograph remains valid and the change "
"in the score can be attributed to the condition alone.")
figure("conditions_strip.png",
      "Figure 3. The original photograph and the four controlled conditions "
      "derived from it.", max_height_cm=6)

h2("2.3 Ground truth masks")
p("Ground truth masks were produced for five of the photographs. The "
  "procedure follows the way modern annotation tools work rather than painting "
  "every pixel by hand. A per image draft was produced first, using "
  "information that the system under test is not allowed to use, for example a "
  "threshold offset hand picked for that one photograph or an edge based "
  "recipe chosen for the coins. Every draft was then inspected as an overlay "
  "on the photograph and corrected by hand: false regions were deleted, "
  "touching objects were separated by clicking one point inside each object "
  "and letting a marker driven watershed cut along the crevice, and clipped "
  "parts of an outline were painted back in. For the low contrast photograph "
  "the draft failed completely, so the three seeds were outlined by hand from "
  "a coordinate grid and the traced polygons were then tightened onto the "
  "object boundary by a colour model refinement. Every mask was inspected a "
  "second time before it was accepted. The complete edit list is in "
  "src/ground_truth.py, so the annotation is fully reproducible.")
p("Four of the five masks carry one region per physical object, so object "
  "level counting can be scored against them. The mask of the first sunflower "
  "photograph is a foreground mask only, because several of its seeds touch "
  "along a long contact line where no crevice is visible, and an honest "
  "instance level annotation of that photograph is not possible. It is "
  "therefore used for pixel level scoring only. The dense pinto bean "
  "photograph carries no ground truth at all and is reported as a qualitative "
  "failure case.")
figure("ground_truth_overview.png",
      "Figure 4. The manually verified ground truth outlines. Each closed "
      "green contour is one annotated region.", max_height_cm=6)

page_break()

# =====
# 3. Methodology
# =====
h1("3. Methodology")
h2("3.1 Image preparation")
p("Every image is first scaled so that its longest side is 900 pixels. Images "
  "are only ever shrunk, never enlarged, so no detail is invented, and the "
  "common working size makes the timing comparison fair and keeps the "
  "morphological kernel sizes meaningful across the dataset.")
h3("Colour representation")
p("The candidate representations that were examined are the grey level image, "
  "the saturation and value channels of HSV and the lightness, a and b "
  "channels of CIE Lab. For seven of the eight photographs the objects differ "
  "from the background mainly in brightness, so the grey level image carries "
  "the separation and is used by the threshold and the region growing stages. "
  "The exception is the cowpea photograph, where the printed ruler is as dark "
  "as the seeds in the grey image but neutral in colour, while the seeds are "
  "strongly red. The a channel of CIE Lab separates the seeds from the ruler "
  "and is therefore the correct representation for that image. The clustering "
  "stage is able to use the full Lab colour vector, which is one of the "
  "reasons it outperforms the other two methods in section 5.")
h3("Noise suppression")
p("Three filters are provided. The Gaussian filter is a linear low pass "
  "filter, it is the fastest and it blurs edges together with the noise. The "
  "median filter is a rank filter, it removes impulse noise while preserving "
  "step edges. The bilateral filter weights neighbours by both spatial and "
  "photometric distance, so it preserves edges best but costs the most time. "
  "Section 5.5 compares them on the noisy images.")
h3("Illumination correction")
p("An uneven illumination field is the single most damaging condition for a "
  "global threshold, because it shifts the object and the background "
  "distributions by different amounts in different parts of the image. The "
  "system removes it with a classical flat field correction. A morphological "
  "closing with a structuring element far larger than any object removes the "
  "objects and leaves an estimate of the background field, which is then "
  "divided out and rescaled to the full eight bit range. When the objects are "
  "brighter than the background an opening is used instead. The polarity is "
  "decided automatically by comparing the mean of a border ring of the image "
  "with the mean of its interior.")

h2("3.2 Method 1, threshold based segmentation")
p("Otsu's method is implemented from the definition rather than called as a "
  "library flag. For a candidate grey level  $t$  the pixels are split into a low "
  "class and a high class, and the between class variance is")
p("<font face='Courier'>sigma_b^2(t) = w0(t) &middot; w1(t) &middot; "
  "(mu0(t) - mu1(t))^2</font>",
  "equation")
p("where  $w_0$  and  $w_1$  are the probabilities of the two classes and  $\mu_0$  and  $\mu_1$  "
  "their means. The implementation evaluates this expression for all 256 "
  "candidate thresholds using cumulative sums, so it runs in linear time in "
  "the number of grey levels, and it returns the whole criterion curve so that "
  "the choice can be plotted. Two further strategies are provided for "
  "comparison, a fixed threshold supplied by the user, and a locally adaptive "
  "Gaussian threshold which computes a separate threshold for every "
  "neighbourhood and therefore tolerates a lighting gradient without an "
  "explicit correction step.")
figure("otsu_histogram.png",
      "Figure 5. The grey level histogram after illumination correction, with "
      "the threshold chosen by the from scratch Otsu implementation. The "
      "histogram is clearly bimodal, which is exactly the situation in which "
      "Otsu is the right tool.", max_height_cm=6.4)

h2("3.3 Method 2, region growing")
p("Region growing starts from a set of seed pixels and repeatedly adds ")

```

```

"neighbouring pixels that satisfy a similarity predicate, so it is a region "
"based method that uses connectivity explicitly, unlike thresholding which "
"classifies every pixel independently. The implementation uses a first in "
"first out queue, so the traversal is a breadth first flood of the "
"similarity region, and every pixel is visited at most once, which makes the "
"routine linear in the number of pixels.")
h3("Similarity predicate")
p("A candidate pixel is accepted when the absolute difference between its "
"intensity and a reference intensity of the region is at most the tolerance. "
"Two references are implemented. The running mean reference uses the mean "
"intensity of the pixels accepted so far, which allows a region to follow a "
"slow intensity drift across a large object. The seed value reference keeps "
"the intensity of the seed fixed, which is stricter and prevents a region "
"from drifting through a shading gradient into the background.")
h3("Seed selection")
p("Three strategies are implemented. The distance transform strategy "
"thresholds the image coarsely, computes the distance transform of the "
"resulting mask and places one seed at the deepest interior point of every "
"connected component, so it is an informed strategy that places a seed near "
"the centre of each object. The grid strategy places seeds on a regular "
"lattice and keeps those that fall on the object side of the coarse "
"threshold. The random strategy samples seed positions uniformly from the "
"object side. Section 5.3 shows how much the result depends on this choice.")

h2("3.4 Method 3, K means clustering")
p("The clustering is implemented as Lloyd's algorithm with a k means plus plus "
"initialisation. The initialisation samples the first centre uniformly and "
"then samples each further centre with a probability proportional to the "
"squared distance to the nearest centre already chosen, which spreads the "
"initial centres apart and avoids the degenerate solutions that a purely "
"random initialisation produces. The main loop alternates an assignment step "
"that attaches every pixel to its nearest centre with an update step that "
"moves every centre to the mean of its members, and it stops when no centre "
"moves further than a small tolerance.")
h3("Feature space")
p("Three feature spaces are provided: the intensity of the prepared grey "
"image, the three dimensional Lab colour vector, and a five dimensional "
"vector that appends the normalised pixel coordinates to the colour vector "
"so that spatially compact clusters are preferred.")
h3("Deciding which clusters are objects")
p("Clustering alone does not say which cluster is the object. The system "
"decides this from the border of the image: a cluster that occupies more "
"than a quarter of the border ring is declared background, and everything "
"else is foreground. If that rule leaves no cluster or all clusters as "
"background, the system falls back to the ordering of the cluster means and "
"picks the darkest or the brightest cluster according to the polarity of the "
"scene. This rule is what allows K means with more than two clusters to be "
"used as a segmentation rather than only as a quantisation.")

h2("3.5 Post processing, boundaries and object extraction")
p("Every raw mask passes through the same clean up: a morphological opening "
"removes isolated noise pixels, a closing repairs small gaps, a hole filling "
"step closes the specular highlights that appear inside glossy objects, and "
"an area filter removes fragments that are far too small to be an object. "
"The hole filling is implemented as a flood fill from a padded background "
"border, so that whatever background is connected to the border stays "
"background and every enclosed pocket becomes object.")
p("Boundaries are obtained with the morphological gradient of the mask, that "
"is the dilation minus the erosion, which gives a one pixel outline of every "
"region. Individual objects are extracted with eight connected component "
"labelling.")
h3("Separating objects that touch")
p("Connected component labelling reports one region for a group of touching "
"objects. The system therefore offers a separation step built on the "
"distance transform and the watershed transform. The distance transform "
"peaks at the centre of every object, thresholding it at a fraction of the "
"local maximum produces one marker per object even when two objects share a "
"boundary, and the watershed then grows those markers back to the object "
"boundaries. The threshold is applied inside each connected component "
"separately, because a single global threshold silently deletes every object "
"that is much smaller than the largest object in the image.")

h2("3.6 Object measurement")
p("For every extracted region the system reports the measurements listed in "
"the question and several standard shape descriptors:")
bullets([
  "<b>Object count</b>, the number of accepted connected components.",
  "<b>Area</b>, the number of pixels in the region.",
  "<b>Perimeter</b>, the arc length of the outer contour.",
  "<b>Centroid</b>, the first order moment of the region.",
  "<b>Bounding box</b>, the axis aligned rectangle around the region.",
  "<b>Object size</b>, reported as the equivalent circular diameter and as "
  "the major and minor axes of the ellipse fitted to the contour, which is "
  "the length and the width of an elongated object such as a seed.",
  "<b>Shape descriptors</b>: circularity  $4 \pi A / P^2$ , which is one for a "
  "perfect disc, the aspect ratio, the extent and the solidity.",
])
p("The pixel measurements are converted into millimetres for the cowpea "
"photograph, which was taken next to a printed ruler. The calibration "
"routine projects the ruler band onto the horizontal axis, detects the tick "
"marks and takes the median distance between neighbouring ticks, which gives "
"13.0 pixels per millimetre at the working resolution and a field of view of "
"69.2 millimetres. The median is used so that the wider marks at the five "
"millimetre positions cannot bias the estimate.")

h2("3.7 Evaluation protocol")
p("Predicted masks are compared with the ground truth masks at two levels. At "
"the pixel level the system reports Intersection over Union, precision, "
"recall, F1 score, the Dice coefficient and pixel accuracy. At the object "
"level, predicted regions are matched greedily to ground truth regions in "
"decreasing order of overlap, a match is accepted at an IoU of 0.5 or more, "
"and object precision, object recall, object F1 and the count error follow "
"from that matching. A third measure evaluates the measurements themselves: "
"for every matched pair the relative error of the area and of the major axis "
"length is computed, which answers the question that matters for a "
"measurement system, namely how accurate the reported number is for an ")

```

```

"object that was found.")
p("All three methods are run with one common parameter set across the whole "
  "dataset, so that no method is given a per image advantage. The tolerance of "
  "the region growing and the configuration of the clustering are the values "
  "that performed best on average in the parameter studies of sections 5.3 and "
  "5.4, so each method is compared at its own best common setting.")

page_break()

# =====
# 4. Implementation
# =====
h1("4. Implementation")
p("The system is written in Python 3 and uses OpenCV, NumPy and Matplotlib. "
  "Otsu's threshold, the region growing and the K means clustering are "
  "implemented in this project rather than called as library primitives, so "
  "that the criterion, the queue and the iteration are all visible in the "
  "source. OpenCV is used for the standard supporting operations: colour "
  "conversion, filtering, morphology, connected component labelling, contour "
  "extraction, the distance transform and the watershed.")
make_table(["module", "responsibility"], [
  ["config.py", "Paths, working resolution and the dataset manifest with the "
    "provenance of every photograph."],
  ["dataset.py", "Retrieval of the base photographs and generation of the "
    "four controlled degradations."],
  ["preprocessing.py", "Colour representation, smoothing filters, contrast "
    "handling, flat field illumination correction and polarity detection."],
  ["segmentation.py", "Otsu from first principles, region growing with an "
    "explicit queue, K means with k means plus plus, morphological clean up "
    "and the watershed separation of touching objects."],
  ["measurement.py", "Boundary identification, connected component "
    "extraction and all object measurements."],
  ["evaluation.py", "Pixel level scores, object level matching and the "
    "measurement error analysis."],
  ["calibration.py", "Pixels per millimetre from the printed ruler."],
  ["ground_truth.py", "Assisted annotation, the manual edit list and the "
    "ground truth masks."],
  ["visualisation.py", "All figures written to results/figures."],
  ["run_experiments.py", "The experiment driver that reproduces every number "
    "and figure in this report."],
  ["report_content.py", "The text, tables and figure list of this report, "
    "written once as neutral blocks."],
  ["make_report.py", "Renders those blocks into the PDF version of the "
    "report."],
  ["make_report_docx.py", "Renders the same blocks into the Word version, so "
    "the two files can never drift apart."],
], col_widths=[4.0, CONTENT_W_CM - 4.0], font_size=8)
table_caption("Table 2. Module map of the project.")

h3("How to reproduce")
pre("cd src\n"
  "python dataset.py          # fetch photographs and build the variants\n"
  "python ground_truth.py     # rebuild the annotation masks\n"
  "python run_experiments.py   # all experiments, tables and figures\n"
  "python make_report.py       # the PDF report\n"
  "python make_report_docx.py  # the Word version of the report")
spacer(6)
p("The complete run of the experiment driver takes about %s seconds on the "
  "machine used for this project." % SUMMARY.get("runtime_s", "400"))

page_break()

# =====
# 5. Results
# =====
h1("5. Experimental results")
h2("5.1 Comparison of the three segmentation techniques")
rows = []
for r in MAIN:
  rows.append([nice(r["image"]), r["method"], r["iou"], r["precision"],
    r["recall"], r["f1"], r["time_s"],
    r["pred_count"], r["gt_count"] or "n/a"])
make_table(["image", "method", "IoU", "precision", "recall", "F1",
  "time (s)", "regions found", "objects in truth"], rows,
  col_widths=[3.5, 3.4, 1.4, 1.6, 1.4,
    1.4, 1.5, 1.7, 1.6],
  font_size=6.8)
table_caption("Table 3. The three segmentation techniques on every annotated "
  "image, using one common parameter set.")

rows = [[r["method"], r["mean_iou"], r["mean_precision"], r["mean_recall"],
  r["mean_f1"], r["mean_time_s"]] for r in MAIN_MEAN]
best_index = [i for i, r in enumerate(MAIN_MEAN) if r is BEST]
make_table(["method", "mean IoU", "mean precision", "mean recall", "mean F1",
  "mean time (s)"], rows,
  col_widths=[5.2, 2.2, 2.6, 2.2, 2.0,
    2.4], font_size=8, highlight_rows=best_index)
table_caption("Table 4. Mean scores over the five annotated photographs. The "
  "best method is highlighted.")

p("The summary table requested by the question is therefore the following, "
  "where the sensitivity entries are taken from the condition study in "
  "section 5.6 and the noise study in section 5.5.")
cond_by = {}
for r in COND:
  cond_by.setdefault((r["method"], r["condition"]), []).append(float(r["iou"]))

def cond_delta(method, condition):
  base = cond_by.get((method, "original photograph"), [])
  other = cond_by.get((method, condition), [])
  if not base or not other:
    return "n/a"
  drop = (sum(other) / len(other)) - (sum(base) / len(base))
  return "%.3f" % drop
rows = []

```

```

for m in MAIN_MEAN:
    rows.append([
        m["method"],
        "%.3f" % float(m["mean_iou"]),
        "%.3f" % float(m["mean_time_s"]),
        cond_delta(m["method"], "Additive Gaussian sensor noise"),
        cond_delta(m["method"], "Strong lighting gradient"),
    ])
make_table(["method", "segmentation accuracy (mean IoU)",
            "processing time (s)", "sensitivity to noise (change in IoU)",
            "sensitivity to illumination (change in IoU)"], rows,
            col_widths=[4.2, 3.6, 2.7, 3.2, 3.0],
            font_size=7.4, highlight_rows=best_index)
table_caption("Table 5. The comparison table required by the question. A "
              "negative change means the score fell when the condition was "
              "applied.")

figure("iou_by_image.png",
       "Figure 6. Segmentation accuracy of the three methods on each annotated "
       "photograph.", max_height_cm=7)
figure("time_by_method.png",
       "Figure 7. Mean processing time per image at the common working "
       "resolution.", max_height_cm=6.4)
for rec in BASE_IMAGES:
    name = "main_%s.png" % rec["id"]
    if (FIG_DIR / name).exists():
        figure(name, "Figure. %s. Green marks pixels that were correctly "
                    "assigned to an object, red marks background accepted as "
                    "object and blue marks object pixels that were missed."
                    % rec["title"], max_height_cm=5.4)

h2("5.2 Threshold strategy and illumination")
rows = [[nice(r["image"]), r["setting"], r["iou"], r["precision"], r["recall"],
             r["f1"]] for r in THRESH]
make_table(["image", "threshold strategy", "IoU", "precision", "recall", "F1"],
            rows, col_widths=[4.2, 5.4, 1.7, 1.9, 1.7,
                               1.7], font_size=6.6)
table_caption("Table 6. Threshold study on the original photographs and on the "
              "same photographs with a strong lighting gradient.")
figure("threshold_study.png",
       "Figure 8. Effect of the threshold strategy. The fixed thresholds are "
       "only competitive when the value happens to suit the image, while Otsu "
       "combined with the flat field correction is stable across all four "
       "cases.", max_height_cm=7.5)
figure("illumination_correction.png",
       "Figure 9. The lighting gradient case in detail. Without the flat field "
       "correction the single global threshold cannot satisfy both sides of "
       "the image at once.", max_height_cm=5.2)

h2("5.3 Region growing, seeds and similarity")
rows = [[nice(r["image"]), r["study"], r["setting"], r["n_seeds"], r["iou"],
             r["recall"], r["pred_count"], r["time_s"]] for r in RG]
make_table(["image", "study", "setting", "seeds", "IoU", "recall",
            "regions", "time (s)"], rows,
            col_widths=[3.7, 2.5, 2.6, 1.5, 1.5,
                        1.5, 1.5, 1.7], font_size=6.6)
table_caption("Table 7. Region growing study.")
figure("region_growing_tolerance.png",
       "Figure 10. Effect of the similarity tolerance. Below the correct value "
       "the regions stop inside the objects, and above it the regions leak "
       "through the object boundary into the background.", max_height_cm=7)
figure("region_growing_seeds.png",
       "Figure 11. Effect of the seed selection strategy. The informed "
       "distance transform strategy is the best choice on the coins, while on "
       "the seeds the extra seeds of the random strategy compensate for the "
       "wide intensity range inside each object.", max_height_cm=7)
figure("region_growing_visual.png",
       "Figure 12. Region growing at three tolerances on the same photograph.",
       max_height_cm=5.2)

h2("5.4 K means, number of clusters and features")
rows = [[nice(r["image"]), r["study"], r["setting"], r["iou"], r["precision"],
             r["recall"], r["pred_count"], r["time_s"]] for r in KM]
make_table(["image", "study", "setting", "IoU", "precision", "recall",
            "regions", "time (s)"], rows,
            col_widths=[3.7, 2.2, 2.4, 1.6, 1.8,
                        1.6, 1.5, 1.7], font_size=6.6)
table_caption("Table 8. K means study.")
figure("kmeans_k.png",
       "Figure 13. Effect of the number of clusters. On an image with two "
       "clearly separated populations, raising K splits the object itself and "
       "the score falls, while on the coins, where the background has its own "
       "internal structure, a larger K is what allows the dark corner of the "
       "background to be separated from the coins.", max_height_cm=7)
figure("kmeans_clusters.png",
       "Figure 14. Cluster label maps for three values of K on the coffee bean "
       "photograph. At K equal to three the shadow becomes its own cluster.",
       max_height_cm=5.2)

h2("5.5 Filtering study")
rows = [[nice(r["image"]), r["filter"], r["iou"], r["precision"], r["recall"],
             r["f1"], r["pred_count"], r["time_s"]] for r in FILT]
make_table(["image", "filter", "IoU", "precision", "recall", "F1", "regions",
            "time (s)"], rows,
            col_widths=[4.4, 2.4, 1.7, 1.9, 1.7,
                        1.7, 1.5, 1.7], font_size=7)
table_caption("Table 9. Effect of the smoothing filter on the two noisy "
              "images, with Otsu thresholding held fixed.")
figure("filter_study.png",
       "Figure 15. Without a smoothing filter the noise fragments the mask "
       "into thousands of small regions, and any of the three filters repairs "
       "most of the damage.", max_height_cm=7)

h2("5.6 Robustness to imaging conditions")
rows = [[nice(r["image"]), r["condition"], r["method"], r["iou"], r["f1"],
             r["pred_count"]] for r in COND]

```

```

make_table(["image", "condition", "method", "IoU", "F1", "regions"], rows,
           col_widths=[4.0, 3.6, 3.6, 1.6, 1.6, 1.6], font_size=6.6)
table_caption("Table 10. Every method under every imaging condition.")
figure("conditions_seeds_sunflower_01.png",
       "Figure 16. Robustness on the sunflower seed photograph.",
       max_height_cm=7)
figure("conditions_coins_greek_07.png",
       "Figure 17. Robustness on the coins photograph.", max_height_cm=7)
p("Three observations follow from Table 10. First, the defocus blur is the "
  "mildest of the four conditions for every method, because blurring moves the "
  "boundary of an object by a few pixels but does not change which side of the "
  "threshold the interior of the object falls on. Second, the under exposure "
  "is almost harmless for the threshold and the clustering, because both are "
  "invariant to a monotone rescaling of the grey levels as long as the two "
  "populations stay separable, while it does hurt the region growing, whose "
  "tolerance is an absolute number of grey levels and is therefore far too "
  "wide once the contrast has been compressed. Third, the small positive "
  "changes in the region growing column of Table 5 are not evidence that noise "
  "helps. They arise because the region growing under grows on the clean "
  "photographs, so its baseline is already low, and a condition that changes "
  "the local statistics can accidentally let a region cover more of an object "
  "than it did before. A score that moves in the right direction for the wrong "
  "reason is still a weakness of the method, not a strength.")

h2("5.7 Separating touching objects")
rows = [[nice(r["image"]), r["expected"] or "not counted", r["marker_ratio"],
         r["count_without_split"], r["count_with_split"]] for r in SPLIT]
make_table(["image", "objects present", "marker distance ratio",
           "regions without separation", "regions after separation"], rows,
           col_widths=[4.4, 2.6, 3.4, 3.2, 3.0],
           font_size=7)
table_caption("Table 11. Effect of the marker threshold used to separate "
              "touching objects.")
figure("split_study.png",
       "Figure 18. Number of regions reported as a function of the marker "
       "threshold.", max_height_cm=7)

h2("5.8 Object measurement")
rows = [[nice(r["image"]), r["expected"] or "not counted",
         r["count_before_split"], r["count_after_split"]] for r in COUNTS]
make_table(["image", "objects present", "count before separation",
           "count after separation"], rows,
           col_widths=[5.4, 3.4, 3.9, 3.9], font_size=8)
table_caption("Table 12. Object counts produced by the full pipeline.")

rows = [[nice(r["image"]), r["method"], r["matched_objects"],
         r["mean_abs_area_error_pct"], r["max_abs_area_error_pct"],
         r["mean_abs_length_error_pct"], r["max_abs_length_error_pct"]]
         for r in ACC]
make_table(["image", "method", "matched objects", "mean area error (%)",
           "max area error (%)", "mean length error (%)",
           "max length error (%)"], rows,
           col_widths=[3.6, 3.3, 2.0, 2.2, 2.0, 2.2, 2.0], font_size=6.8)
table_caption("Table 13. Accuracy of the measurements of the objects that "
              "were correctly found, relative to the ground truth regions.")

figure("measured_full_beans_cowpea_scale_05.png",
       "Figure 19. The measurement output on the calibrated photograph. Every "
       "seed is labelled with its length and width in millimetres, computed "
       "from the ellipse fitted to its contour and the calibration of 13.0 "
       "pixels per millimetre. The measured lengths of 10.5 to 13.0 "
       "millimetres and widths of 4.4 to 5.9 millimetres are consistent with "
       "the published dimensions of cowpea seed, which is an independent check "
       "on the calibration. Region 9 is the letter m of the printed unit "
       "label, a false detection that is discussed in section 7.",
       max_height_cm=10)
figure("cowpea_length_histogram.png",
       "Figure 20. Length distribution of the eight measured seeds.",
       max_height_cm=6.4)
for image_id in ("beans_coffee_06", "coins_greek_07"):
    figure("measure_%.png" % image_id,
          "Figure. Segmentation, separation of touching objects and "
          "measurement on %s." % nice(image_id), max_height_cm=5.2)
figure("seed_area_histogram.png",
       "Figure 21. Area distribution of the regions found in the sunflower "
       "photograph. The long tail on the right is produced by the clusters of "
       "touching seeds that were not separated.", max_height_cm=6.4)

page_break()

# =====
# 6. Critical analysis
# =====
h1("6. Critical analysis")
h2("6.1 Which segmentation method produced the best results and why")
p("K means clustering in the Lab colour space with three clusters produced the "
  "highest mean Intersection over Union, %s, ahead of Otsu thresholding at %s "
  "and region growing at %s. Two properties explain the difference. First, the "
  "clustering is the only method of the three that can use colour: on the "
  "coins photograph it separates the dark background from the coins where a "
  "grey level threshold cannot, and on the low contrast seed photograph it "
  "uses the slight colour difference between the cream seeds and the bluish "
  "white paper, which raises the IoU from %s to %s. Second, a third cluster "
  "gives the method somewhere to put the shadow, so shadow pixels are not "
  "forced into either the object class or the background class."
  % (BEST["mean_iou"],
    [m for m in MAIN_MEAN if m["method"].startswith("Threshold")][0]["mean_iou"],
    [m for m in MAIN_MEAN if m["method"].startswith("Region")][0]["mean_iou"],
    [r for r in MAIN if r["image"] == "seeds_pale_04"
     and r["method"].startswith("Threshold")][0]["iou"],
    [r for r in MAIN if r["image"] == "seeds_pale_04"
     and r["method"].startswith("K means")][0]["iou"]])
p("The advantage comes at a price. The clustering is roughly two orders of "
  "magnitude slower than the threshold, because it iterates over every pixel ")

```

"many times, while Otsu touches the image once to build a histogram. It is "
 "also worth stating clearly that K means with two clusters on the intensity "
 "channel is not a genuinely different method: it minimises the within class "
 "variance of a two class partition of the grey levels, which is the same "
 "objective that Otsu maximises in its complementary form. In the study of "
 "section 5.4, K means with two clusters on the intensity channel reproduced "
 "the Otsu result exactly on two of the three photographs and differed by "
 "less than one hundredth of an IoU point on the third, which is a useful "
 "confirmation that both implementations are correct.")

h2("6.2 How illumination changes affected thresholding")
 th_rows = [r for r in THRESH if r["image"] == "seeds_sunflower_01_illum"]
 otsu_off = [r for r in th_rows if r["setting"].startswith("Otsu, no")]
 otsu_on = [r for r in th_rows if r["setting"].startswith("Otsu, illumination")]
 p("A global threshold assumes that one grey level separates object from "
 "background everywhere in the image. A lighting gradient breaks that "
 "assumption directly. On the photograph with the diagonal ramp, Otsu without "
 "the flat field correction scored an IoU of %, and the same threshold after "
 "the correction scored %. The error is systematic rather than random: in "
 "the brightly lit part of the image the background is pushed above the "
 "threshold, so the mask is correct there, while in the dim part the "
 "background falls below the threshold and is accepted as object, which is "
 "why the precision collapses while the recall stays high. The locally "
 "adaptive threshold reaches a similar accuracy to the corrected global "
 "threshold on the gradient images, because it recomputes the decision in "
 "every neighbourhood, but it is noticeably worse on the clean images because "
 "it also responds to the texture inside the objects."
 % (otsu_off[0]["iou"] if otsu_off else "n/a",
 otsu_on[0]["iou"] if otsu_on else "n/a"))
 coins_off = [r for r in THRESH if r["image"] == "coins_greek_07"
 and r["setting"].startswith("Otsu, no")]
 coins_on = [r for r in THRESH if r["image"] == "coins_greek_07"
 and r["setting"].startswith("Otsu, illumination")]
 p("The correction is not free, and the coins photograph shows the cost "
 "clearly. There the flat field step lowered the score from % to %. The "
 "reason is that this image has a dark background, so the estimated "
 "background field is small, and dividing by a small number amplifies both "
 "the noise and the residual texture of the background until part of it "
 "crosses the threshold. The lesson is that a flat field correction should be "
 "applied when the illumination varies across the field of view, and not as a "
 "reflex on every image: on a dark background it is safer to work on the "
 "boundaries of the objects, as the edge based recipe in section 6.7 does."
 % (coins_off[0]["iou"] if coins_off else "n/a",
 coins_on[0]["iou"] if coins_on else "n/a"))

h2("6.3 How seed point selection influenced region growing")
 p("Seed selection changed the result more than any other single parameter. On "
 "the coins photograph the informed distance transform strategy placed one "
 "seed near the centre of every detected coin and reached an IoU of %, while "
 "the regular grid placed only five usable seeds and reached %, a loss of "
 "more than a factor of five. The reason is structural: region growing can "
 "never recover an object that contains no seed, so a missing seed is a "
 "guaranteed false negative rather than a small error. On the sunflower "
 "photograph the ordering reverses, because the coarse mask merges touching "
 "seeds into seven components and therefore yields only seven seeds, while "
 "the random strategy scatters forty seeds and covers far more of the seed "
 "area. The practical conclusion is that region growing is only as good as "
 "the procedure that produces its seeds, and that procedure is itself usually "
 "a segmentation, which makes the method circular unless the seeds come from "
 "a human or from a physically motivated prior."
 % ([r for r in RG if r["image"] == "coins_greek_07"
 and r["study"] == "seed_strategy" and r["setting"] == "distance"][0]["iou"],
 [r for r in RG if r["image"] == "coins_greek_07"
 and r["study"] == "seed_strategy" and r["setting"] == "grid"][0]["iou"]]))
 p("The similarity criterion mattered less. The running mean reference was "
 "slightly better than the fixed seed value reference on both images, because "
 "the objects in this dataset have a wide internal intensity range and a "
 "region that can follow a drift covers more of the object before it stops.")

h2("6.4 What effect changing K had on K means segmentation")
 p("The sweep over K in Table 8 is run on the intensity feature space, so that "
 "the effect of the number of clusters is not mixed with the effect of the "
 "features. The effect of K depends entirely on how many populations the "
 "image really contains. On the sunflower photograph, which has one object population and "
 "one background population, the best value was K equal to two and every "
 "further cluster split the object itself, so the score fell monotonically "
 "from % at K equal to two to % at K equal to six. On the coins "
 "photograph the opposite happened: the background is not uniform, it has a "
 "bright region and a dark corner, so K equal to two forced the dark corner "
 "and the dim coins into the same cluster and the score was only %, while K "
 "equal to four separated them and reached %. In short, K must match the "
 "number of distinguishable populations in the image, not the number of "
 "object classes the user is interested in."
 % ([r for r in KM if r["image"] == "seeds_sunflower_01" and r["study"] == "K"
 and r["setting"] == "2"][0]["iou"],
 [r for r in KM if r["image"] == "seeds_sunflower_01" and r["study"] == "K"
 and r["setting"] == "6"][0]["iou"],
 [r for r in KM if r["image"] == "coins_greek_07" and r["study"] == "K"
 and r["setting"] == "2"][0]["iou"],
 [r for r in KM if r["image"] == "coins_greek_07" and r["study"] == "K"
 and r["setting"] == "4"][0]["iou"]]))

h2("6.5 Which method was most computationally efficient")
 p("Thresholding, by a wide margin. Its mean processing time was %s seconds per "
 "image against %s seconds for the region growing and %s seconds for the "
 "clustering at the common working resolution of 900 pixels. The ordering "
 "follows directly from the algorithms. Otsu makes one pass over the image to "
 "build a histogram and then searches 256 candidate thresholds, so its cost "
 "is one pass over the pixels. Lloyd's algorithm makes one distance "
 "computation per pixel per cluster per iteration, so its cost is "
 "approximately K times the number of iterations passes over the pixels, and "
 "the five dimensional feature space multiplies that again. The region "
 "growing visits every pixel once, but it does so in an interpreted loop with "
 "a queue and a per pixel predicate, so its constant factor is far larger "
 "than the vectorised operations used by the other two. A compiled "
 "implementation of the same algorithm would close most of that gap, so the "

```

"timing of the region growing should be read as a property of this "
"implementation and not of the algorithm."
% (FASTEST["mean_time_s"],
  [m for m in MAIN_MEAN if m["method"].startswith("Region")][0]["mean_time_s"],
  [m for m in MAIN_MEAN if m["method"].startswith("K means")][0]["mean_time_s"]))

h2("6.6 Under what conditions did each method fail")
make_table(["condition", "thresholding", "region growing", "K means"], [
  ["Lighting gradient",
    "Fails without a flat field correction, the background of the dim side is "
    "accepted as object.",
    "Degrades gracefully, because the predicate is local, but regions leak "
    "across the gradient when the tolerance is high.",
    "Degrades, because a brightness gradient creates spurious clusters that "
    "cut across the object and background populations."],
  ["Low contrast",
    "Fails completely, the histogram has a single mode so the threshold is "
    "meaningless.",
    "Leaks into the background, since object and background differ by less "
    "than the tolerance.",
    "Best of the three, because the residual colour difference survives even "
    "when the brightness difference does not."],
  ["Noise",
    "Fragments the mask into many small regions unless a smoothing filter is "
    "applied first.",
    "Stops early, because a noisy neighbour fails the predicate.",
    "Fairly robust, because averaging over a cluster suppresses zero mean "
    "noise."],
  ["Touching objects",
    "Reports one region for the whole group.",
    "Same, unless a separate seed is placed in each object and the tolerance "
    "is tight.",
    "Same, since clustering has no notion of connectivity."],
  ["Objects of different sizes",
    "Handled, provided the area filter is not set too high.",
    "Handled, provided each object receives a seed.",
    "Handled, the cluster assignment does not depend on region size."],
  ["Shadows",
    "Shadow pixels are darker than the background and are accepted as object.",
    "Regions grow into the penumbra when the tolerance exceeds the shadow "
    "step.",
    "A third cluster absorbs the shadow, which is why K equal to three helps "
    "on the coffee photograph."],
], col_widths=[3.0, 4.6, 4.6, 4.6], font_size=6.8)
table_caption("Table 14. Failure conditions of the three techniques.")

h2("6.7 How techniques studied in earlier weeks improve the results")
bullets([
  "<b>Morphological filtering.</b> Opening, closing and hole filling turn a "
  "ragged raw threshold into a usable mask. On the coffee beans the "
  "specular highlights inside the objects appear as holes in the raw mask, "
  "and the hole filling step alone repairs them.",
  "<b>Flat field correction by large scale morphology.</b> Estimating the "
  "background with a structuring element larger than any object and dividing "
  "it out restores a bimodal histogram, and it is what makes the global "
  "threshold usable on the gradient images.",
  "<b>Edge detection.</b> The boundaries of the coins are far more reliable "
  "than their absolute grey levels, and an edge based recipe followed by a "
  "morphological closing and a hole fill segments that photograph much "
  "better than a global threshold. The same recipe was used to draft the "
  "ground truth for that image.",
  "<b>The distance transform and the watershed.</b> These separate touching "
  "objects that connected component labelling merges, which is what converts "
  "a foreground mask into an object count.",
  "<b>Colour space transformation.</b> Working in CIE Lab rather than in "
  "grey levels is what allows the ruler to be rejected in the cowpea "
  "photograph and the pale seeds to be found at all.",
  "<b>Shape descriptors.</b> Circularity, solidity and the aspect ratio "
  "provide a cheap post filter: on the cowpea photograph the false detection "
  "produced by the printed unit label has a much lower area and a different "
  "aspect ratio from any seed, so a simple shape rule removes it.",
])

page_break()

# =====
# 7. Limitations
# =====
h1("7. Limitations and failure cases")
h2("7.1 Dense piles of objects")
p("The clearest limitation of the whole approach is that it assumes one "
  "connected region corresponds to one object. The pinto bean photograph "
  "breaks that assumption: about two hundred beans are piled so that almost "
  "every bean touches several neighbours, and the segmentation returns a "
  "single region covering the entire pile. The distance transform and "
  "watershed step improves this but does not solve it, because the ridges "
  "between overlapping beans are not deep enough for the distance transform to "
  "produce one marker per bean. Solving this case properly requires either a "
  "different acquisition protocol, spreading the beans out so that they do not "
  "touch, or a method that models object appearance rather than object "
  "connectivity.")
figure("failure_dense.png",
  "Figure 22. The dense pile failure case. The pile is segmented "
  "perfectly as foreground, and the object count is still wrong by two "
  "orders of magnitude.", max_height_cm=5.6)
h2("7.2 Low contrast objects")
p("When the object and the background have nearly the same brightness the grey "
  "level histogram has a single mode and the Otsu criterion has no meaningful "
  "maximum to find. It then selects a threshold inside the object population "
  "and cuts the objects into pieces. Only the colour based clustering recovers "
  "the objects on this photograph, and even it merges the darker striped "
  "regions of the seed with its bright out of focus lobe imperfectly.")
figure("failure_low_contrast.png",
  "Figure 23. The low contrast failure case for each method.",
  max_height_cm=5.4)
h2("7.3 Shadows and specular highlights")

```

```

p("A cast shadow is darker than the background, so any method that defines "
  "objects as dark pixels will accept part of the shadow as object. This is "
  "the main reason the measured areas on the coffee and cowpea photographs are "
  "larger than the ground truth areas, by about %s per cent on average in "
  "Table 13, while the measured lengths are accurate to about %s per cent. A "
  "shadow adds a thin halo around the whole object, which changes the area "
  "much more than it changes the fitted major axis."
  % (ACC[0]["mean_abs_area_error_pct"], ACC[0]["mean_abs_length_error_pct"]))
figure("failure_shadow.png",
  "Figure 24. Shadow pixels accepted as object, shown in red.",
  max_height_cm=6.4)

h2("7.4 Other limitations")
bullets([
  "The system has no notion of what an object is. Any dark connected region "
  "large enough to survive the area filter is reported, which is why the "
  "letter m of the printed unit label in the cowpea photograph is counted as "
  "a ninth object.",
  "Objects cut by the image border are measured as if they were whole. The "
  "measurement module flags them with a touches_border column so that they "
  "can be excluded, but the count still includes them.",
  "Measurements are reported in millimetres only when the photograph "
  "contains a scale. Without a scale, only relative sizes are meaningful.",
  "The ground truth is not perfect. It was produced with an assisted "
  "procedure and inspected twice, and the remaining boundary error of a few "
  "pixels per object places an upper bound of roughly one to two per cent on "
  "the IoU figures that can be trusted.",
  "The region growing implementation is interpreted Python, so its timing is "
  "not comparable with the vectorised routines used by the other two "
  "methods.",
  "All experiments are run at a common working resolution of 900 pixels. "
  "Absolute pixel measurements would change at another resolution, although "
  "the relative comparisons would not.",
])

page_break()

# =====
# 8. Conclusion
# =====
h1("8. Conclusion")
p("A complete classical pipeline was built and evaluated: image preparation, "
  "three segmentation techniques implemented from first principles, "
  "morphological clean up, boundary identification, separation of touching "
  "objects, object extraction, object measurement in pixels and in "
  "millimetres, and a quantitative evaluation against manually verified ground "
  "truth.")
p("On the five annotated photographs, K means clustering in the Lab colour "
  "space reached a mean Intersection over Union of %s, Otsu thresholding "
  "reached %s and region growing reached %s. Thresholding was about %d times "
  "faster than the clustering and is the right default whenever the histogram "
  "is bimodal and the illumination has been corrected. Clustering earns its "
  "extra cost only when colour carries information that brightness does not, "
  "which was the case on the coins and on the low contrast seeds. Region "
  "growing was the weakest of the three in an automatic setting, not because "
  "the algorithm is poor but because its result is decided by the seed set, "
  "and producing a good seed set automatically already requires a "
  "segmentation."
  % (BEST["mean_iou"],
    [m for m in MAIN_MEAN if m["method"].startswith("Threshold")][0]["mean_iou"],
    [m for m in MAIN_MEAN if m["method"].startswith("Region")][0]["mean_iou"],
    int(round(float(BEST["mean_time_s"]) / max(1e-6, float(FATEST["mean_time_s"]))))))
p("For the objects that were correctly found, the measurements are usable: on "
  "the calibrated photograph all eight cowpea seeds were matched to the ground "
  "truth and their lengths were reported to within about one and a half per "
  "cent, which is adequate for seed grading, while their areas carried a "
  "larger error because the segmentation includes part of the cast shadow.")
p("The clearest lesson of the experiments is that the acquisition matters more "
  "than the choice among the three segmentation techniques. Diffuse and even "
  "lighting, a background whose colour contrasts with the objects, and objects "
  "that are spread out rather than piled together move every method into the "
  "range above 0.95 IoU. When those conditions are absent, no amount of "
  "parameter tuning on a classical method recovers the lost information.")

# =====
# 9. References
# =====
h1("9. References")
refs = [
  "Otsu, N. (1979). A threshold selection method from gray level histograms. "
  "IEEE Transactions on Systems, Man and Cybernetics, 9(1), 62 to 66.",
  "Adams, R. and Bischof, L. (1994). Seeded region growing. IEEE "
  "Transactions on Pattern Analysis and Machine Intelligence, 16(6), 641 to "
  "647.",
  "Lloyd, S. P. (1982). Least squares quantization in PCM. IEEE Transactions "
  "on Information Theory, 28(2), 129 to 137.",
  "Arthur, D. and Vassilvitskii, S. (2007). k-means++: the advantages of "
  "careful seeding. Proceedings of the ACM SIAM Symposium on Discrete "
  "Algorithms, 1027 to 1035.",
  "Beucher, S. and Meyer, F. (1993). The morphological approach to "
  "segmentation: the watershed transformation. Mathematical Morphology in "
  "Image Processing, 433 to 481.",
  "Canny, J. (1986). A computational approach to edge detection. IEEE "
  "Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679 to "
  "698.",
  "Gonzalez, R. C. and Woods, R. E. (2018). Digital Image Processing, fourth "
  "edition. Pearson.",
  "Bradski, G. (2000). The OpenCV library. Dr Dobb's Journal of Software "
  "Tools.",
  "Rother, C., Kolmogorov, V. and Blake, A. (2004). GrabCut, interactive "
  "foreground extraction using iterated graph cuts. ACM Transactions on "
  "Graphics, 23(3), 309 to 314. Used only as an annotation aid when the "
  "hand traced outlines were tightened.",
]
for i, r in enumerate(refs, start=1):
  p("[%d] %s" % (i, r), "small")

```

```
h2("Image sources")
rows = [[rec["id"], rec["source"], rec.get("licence", "")] for rec in BASE_IMAGES]
make_table(["identifier", "source", "licence"], rows,
           col_widths=[3.8, 9.4, 3.4], font_size=7)
table_caption("Table 15. Provenance of every photograph in the dataset. All of "
              "them are real photographs.")

# =====
# Appendix: source code
# =====
page_break()
h1("Appendix A. Complete source code")
p("The listing below is the complete source of the project, in the order in "
  "which the modules are described in Table 2. The same files are included in "
  "the src directory of the repository.", "small")

CODE_FILES = ["config.py", "dataset.py", "preprocessing.py", "segmentation.py",
              "measurement.py", "evaluation.py", "calibration.py",
              "ground_truth.py", "visualisation.py", "run_experiments.py",
              "make_report.py", "make_report_docx.py",
              "report_content.py"]
for name in CODE_FILES:
    path = SRC_DIR / name
    if not path.exists():
        continue
    code_heading("src/" + name)
    text = path.read_text(encoding="utf-8")
    text = text.replace("\t", " ")
    lines = text.splitlines()
    chunk = 62
    for i in range(0, len(lines), chunk):
        code_block("\n".join(lines[i:i + chunk]))
```